VLSI DESIGN VERIFICATION

SystemVerilog Interview Preparation

Module 5: Randomization and Constraints

Detailed study notes for Design Verification roles

Concepts from first principles · complete SystemVerilog examples · line-by-line explanations
comparison tables · common mistakes · interview questions with answers · coding exercises

Contents

Module 5 Overview 4

Part 1: Randomization Basics 6
- 1.1 Introduction to Randomization 6
- 1.2 rand and randc 9
- 1.3 The randomize() Method and Randomizing Class Properties 13
- 1.4 Randomizing Arrays 16

Part 2: Constraint Blocks and Constraint Expressions 22
- 2.1 Constraint Blocks and How the Solver Thinks 22
- 2.2 Relational and Arithmetic Constraint Expressions 26
- 2.3 The inside Operator 29
- 2.4 Conditional Constraints: Implication, if-else and Equivalence 32
- 2.5 foreach Constraints and Array Reduction 36
- 2.6 unique Constraints 40

Part 3: Controlling the Distribution 43
- 3.1 Distribution Constraints: dist and Weighted Distribution 43
- 3.2 Soft Constraints 48
- 3.3 solve...before: Variable Ordering 52
- 3.4 Functions in Constraints 56

Part 4: Controlling Randomization 59
- 4.1 pre_randomize() and post_randomize() 59
- 4.2 Turning Constraints On and Off: constraint_mode() 63
- 4.3 Turning Random Variables On and Off: rand_mode() 66
- 4.4 Inline Constraints: randomize() with, and Conditional Randomization 69
- 4.5 std::randomize(), System Random Functions and Seeding 73
- 4.6 Randomization Failure and Constraint Debugging 76
- 4.7 Constraint Inheritance, Overriding and Extension 80

Part 5: Worked Constraint Problems 84
- Problem 1: Address in two regions, word-aligned, with a reserved hole 85
- Problem 2: Even numbers, and multiples of 5 that are not multiples of 10 85
- Problem 3: Power of two 86
- Problem 4: Exactly N bits set, and N contiguous bits set 87
- Problem 5: Unique array elements without the unique keyword 88
- Problem 6: Array in ascending order 89
- Problem 7: No two consecutive elements equal; no three in a row 90
- Problem 8: Elements that add up to a given total 91
- Problem 9: Even values at even indices, odd values at odd indices 92
- Problem 10: No repeated value without randc 93
- Problem 11: Fixed patterns 95
- Problem 12: Payload length depends on the packet kind 96
- Problem 13: Exactly K elements equal to a given value 97
- Problem 14: Palindrome array 98
- Problem 15: Gray-code neighbours and fixed Hamming distance 99
- Problem 16: No two adjacent 1 bits 100
- Problem 17: A valid calendar date 101

Problem 18: Non-overlapping memory blocks 102
Problem 19: A matrix with unique rows and unique columns 103
Problem 20: ALU stimulus with a controlled overflow rate 104
Problem 21: Eight queens 105
Problem 22: A push/pop sequence that never overflows or underflows a FIFO 106
What the problems have in common 108

Part 6: Module 5 Revision and Practice 109
6.1 Revision Summary 109
6.2 Interview Questions and Answers 111
6.3 Output-Prediction Questions 113
6.4 Find-the-Error and Debugging Questions 116
6.5 Coding Exercises with Solutions 118
6.6 Concepts You Must Understand Before Moving Forward 123
6.7 What Comes Next 123

Module 5 Overview

Module 5 covers constrained-random stimulus generation: how SystemVerilog produces random values, how constraints restrict those values to legal and interesting ones, and how the constraint solver actually behaves. This is the most heavily examined topic in verification interviews after basic OOP. Interviewers ask you to write constraints on a whiteboard, to predict whether a `randomize()` call succeeds, and to explain why a distribution is skewed.

What this module contains

Part	Content
1	Why randomization, `rand` and `randc`, the `randomize()` method, randomizing class properties and arrays
2	Constraint blocks and constraint expressions: relational, arithmetic, `inside`, implication, `if-else`, `foreach`, `unique`
3	Controlling the distribution: `dist`, `soft`, `solve...before`, functions in constraints
4	Controlling randomization: `pre_randomize()`, `post_randomize()`, `constraint_mode()`, `rand_mode()`, `randomize() with`, `std::randomize`, seeding, failure and debugging, inheritance
5	Twenty-two worked constraint problems, from simple to advanced
6	Revision summary, interview questions, output-prediction and debugging questions, coding exercises, checklist

What you need from earlier modules

From	You must already know
Module 1	Two-state and four-state types, signed and unsigned rules, expression width, `enum`, `inside`, operators
Module 2	Fixed arrays, dynamic arrays, queues, associative arrays, `foreach`, array methods such as `sum()` and `size()`
Module 3	Functions, `automatic` lifetime, `void` functions
Module 4	Classes, objects, handles, `new()`, `this`, inheritance, `super`, virtual methods

The running example. The course DUT is the 8-bit ALU of Module 1 (`alu_pkg`, `alu`). Its transaction class is `alu_txn`, with `rand bit [7:0] a, b;` and `rand alu_op_e op;`. Examples that say "needs `alu_pkg.sv`" must be compiled after the `alu_pkg.sv` file from `examples/module-01/`. All other examples are self-contained.

> **Note:** No simulator was available when these notes were written. Random values depend on the seed and on the simulator, so they cannot be predicted. For random examples the notes therefore give a "What to expect" paragraph listing the properties that **every** run must satisfy, and any values shown are marked as sample output. Where the result does not depend on random values (a `randomize()` call that must fail, a mode query, a fully determined constraint), the exact output is given; those outputs were derived by hand from the IEEE 1800 rules. Run every example yourself.

Terms used throughout

Term	Meaning
Random variable	A class property declared `rand` or `randc` (or a variable passed to `std::randomize`)
State variable	Any non-random variable that appears in a constraint. The solver treats its current value as a constant
Constraint	A Boolean expression that must be true for the values the solver chooses
Constraint solver	The part of the simulator that finds values satisfying all active constraints
Solution	One complete assignment of values to all random variables that satisfies every constraint
Solution space	The set of all solutions
Seed	The number that initialises the random number generator. Same seed, same code, same simulator: same values
CRV	Constrained-random verification

Part 1: Randomization Basics

1.1 Introduction to Randomization

In plain English

A directed test says "drive `a` = 200, `b` = 100, `op` = add, and expect 300". You write every value yourself. That is fine for ten cases and impossible for ten million. The 8-bit ALU alone has 256 × 256 × 8 = 524,288 input combinations, and a real design has vastly more.

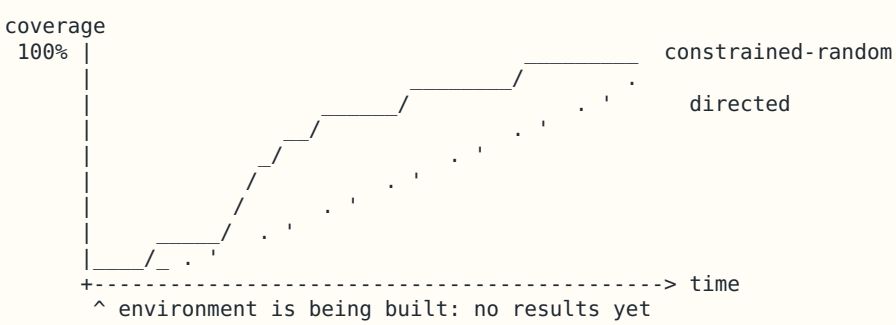
Randomization turns the job around. Instead of listing the stimulus, you describe **what legal stimulus looks like** (the constraints), and the simulator generates as many different legal stimuli as you ask for. You then measure with functional coverage (Module 10) which interesting cases were actually hit.

Technical definition

Constrained-random verification is a methodology in which stimulus is generated by a random number generator under the control of declarative **constraints**. In SystemVerilog the main mechanism is object-based: class properties are declared `rand` or `randc`, restrictions are written in `constraint` blocks, and the built-in `randomize()` method invokes the **constraint solver**, which assigns to all random properties a set of values that satisfies every active constraint.

Why it is needed

Directed testing	Constrained-random testing
Each test is written by hand	One test generates thousands of scenarios
Covers only what the engineer thought of	Also finds corner cases nobody thought of
Effort grows linearly with the number of scenarios	Effort is mostly up front (building the environment)
Progress is easy to see at first, then slows	Slow start, then coverage rises quickly
Expected results can be hard-coded	Needs a self-checking testbench (scoreboard) and functional coverage
Still needed for a few hard-to-reach cases	Used for the bulk of verification



Random stimulus alone is not enough. **Unconstrained** random values are mostly illegal (an opcode that does not exist, a packet length of four billion) or uninteresting. Constraints do two jobs:

1. **Legality:** generate only stimulus the DUT is specified to handle.
2. **Steering:** bias the stimulus toward interesting areas (boundaries, long bursts, error cases).

The sources of random values in SystemVerilog

Mechanism	Returns	Constraints	Typical use
`$random`	Signed 32-bit integer	None	Legacy Verilog. Avoid in new code
`$urandom`	Unsigned 32-bit integer	None	A quick random number in procedural code
`$urandom_range(max, min)`	Unsigned integer in [min, max]	Range only	Random delays, random counts
`obj.randomize()`	1 (success) or 0 (failure); assigns the object's `rand` / `randc` properties	Constraint blocks and `with`	Transactions, configuration objects. The main mechanism
`std::randomize(v1, v2) with {...}`	1 or 0; assigns the listed variables	Inline only	Randomizing local variables without a class
`randcase`	Chooses one branch by weight	Weights	Random choice between actions (Module 11)
`randsequence`	Generates a sequence from grammar rules	Weights	Structured random sequences (Module 11)

A first example

`tb_rand_intro.sv`

```
module tb_rand_intro;

  class simple_txn;
    rand bit [7:0] a;
    rand bit [7:0] b;

    constraint small_a_c { a < 10; }
  endclass

  simple_txn txn;

  initial begin
    txn = new();
    repeat (5) begin
      if (!txn.randomize()) $fatal(1, "randomize failed");
      $display("a=%0d b=%0d", txn.a, txn.b);
    end
    $display("$urandom_range(5, 1) = %0d", $urandom_range(5, 1));
  end

endmodule
```

What to expect. Five lines. In every line `a` is between 0 and 9 and `b` is anything from 0 to 255. The last line shows a number from 1 to 5. Sample output; actual values differ per seed and simulator:

```
a=3 b=201
a=9 b=14
a=0 b=77
a=6 b=250
a=3 b=92
$urandom_range(5, 1) = 4
```

Line by line

Line	Explanation
`class simple_txn;`	Random variables and constraints live in a class. Classes are covered in Module 4.
`rand bit [7:0] a;`	The keyword `rand` marks the property as a random variable. Without it, `randomize()` leaves the property alone.
`constraint small_a_c { a < 10; }`	A named constraint block. It contains one expression that must be true after randomization.
`txn = new();`	An object must exist before it can be randomized. Calling `randomize()` on a null handle is a run-time error.
`txn.randomize()`	Built-in method of every class. It calls the solver, assigns new values to `a` and `b`, and returns 1. It returns 0 if the constraints cannot be satisfied.
`if (!txn.randomize()) $fatal(...)`	The return value must always be checked. A failed randomization leaves the old values in place, and without the check the test continues with stale data.
`$urandom_range(5, 1)`	A random unsigned number between 1 and 5, both included.

How it works internally

A simulator contains a **pseudo-random number generator** (PRNG). It is deterministic: starting from the same **seed** it produces the same sequence of numbers. That is essential, because a failing random test must be reproducible by re-running it with the same seed.

```
   seed --> PRNG --> raw random numbers --+
                                          v
   rand variables  ----------------->  constraint  ---> one solution ---> values written
   active constraints -------------->    solver                          into the object
   state variable values ----------->
```

The solver does not "generate a value and test it". It analyses the constraints as a whole, determines the set of legal solutions, and picks one of them. Section 2.1 describes this in detail.

> **Key point:** Randomization in a testbench is pseudo-random and repeatable. A regression runs the same test with many different seeds; when one fails, the seed is recorded and the failure is reproduced exactly with that seed.

Interview questions

Q1. What is constrained-random verification? A verification approach in which stimulus is generated randomly but restricted by constraints to legal and interesting values. Combined with a self-checking testbench and functional coverage, it reaches far more scenarios than hand-written directed tests.

Q2. Why not use purely random stimulus? Most unconstrained values are illegal for the protocol or uninteresting. Constraints keep the stimulus legal and steer it toward corner cases.

Q3. What is the difference between `$random` and `$urandom`? `$random` returns a signed 32-bit value and is the old Verilog function; its sequence is not managed per thread and object. `$urandom` returns an unsigned 32-bit value and takes part in SystemVerilog's random stability scheme, where every thread and object has its own generator (Section 4.5). New code uses `$urandom` or `$urandom_range`.

Q4. If stimulus is random, how do you know what was tested? With functional coverage. Covergroups record which values and combinations occurred, so the random process is measured instead of assumed.

Q5. How do you reproduce a failure found by a random test? Re-run the same test with the same seed, the same code and the same simulator version. The pseudo-random sequence is then identical.

1.2 rand and randc

In plain English

Both keywords make a class property random. The difference is in how values repeat.

- `rand` is like **rolling a die**. Each roll is independent. You can get 4 three times in a row.
- `randc` is like **dealing cards from a shuffled deck**. You never see the same card twice until the whole deck has been dealt. Then the deck is reshuffled and dealing starts again.

Technical definition

Keyword	Definition
`rand`	Standard random variable. On each `randomize()` call the value is chosen with a **uniform distribution** over the values that satisfy the constraints. Successive calls are independent, so values may repeat at any time
`randc`	Random-**cyclic** variable. The solver iterates over all legal values in a random permutation. No value is repeated until every legal value has been produced once. Then a new permutation is started

Syntax

```
class cfg;
  rand  bit [3:0] len;       // 0 to 15, may repeat
  randc bit [2:0] id;        // 0 to 7, each value once per cycle of 8
  rand  alu_op_e  op;        // enum: only the named values are generated
  randc bit [1:0] slot[4];   // array: each element is its own randc variable
  bit   [7:0]     fixed;     // not random: randomize() never changes it
endclass
```

How randc works internally

```
  randc bit [1:0] id;      legal values: 0 1 2 3

  cycle 1:   shuffle -> 2 0 3 1     calls 1 to 4 return 2, 0, 3, 1
  cycle 2:   shuffle -> 1 3 2 0     calls 5 to 8 return 1, 3, 2, 0
  cycle 3:   shuffle -> ...

  A value may appear twice in a row only across a cycle boundary
  (here: call 4 returns 1 and call 5 returns 1).
```

Rules that follow from this mechanism:

- The simulator must remember, **per object**, which values have already been used. That history belongs to the object. A new object starts a new cycle.
- `randc` variables are **solved before** all `rand` variables. The solver first picks the next cyclic value, then solves the remaining variables around it.
- If constraints restrict a `randc` variable, the cycle runs over the **constrained** set of values only.
- If the constraints on the variable change between calls, or none of the remaining values of the current permutation satisfies the constraints, the permutation is recomputed (the cycle restarts).
- Because the history must be stored, a tool may limit the size of a `randc` variable. The standard requires that at least 8 bits be supported; commercial simulators support more. Large `randc` variables cost memory and time.

Restrictions on randc

Restriction	Reason
Only integral types: `bit` / `logic` vectors and enums	A cycle needs a finite, enumerable set of values
Cannot appear in `solve` ... `before`	`randc` variables are always solved first already
Cannot be the subject of a `dist`	A weighted distribution contradicts "each value exactly once per cycle"
Cannot have `soft` constraints	Soft constraints are defined only for `rand` variables
Cannot be declared for `std::randomize`	Local variables have no per-object history

Complete example

tb_rand_vs_randc.sv

```
module tb_rand_vs_randc;

  class dice;
    rand  bit [1:0] r;     // independent each time
    randc bit [1:0] c;     // cyclic
  endclass

  dice d;
  int  seen[4];            // how often each value of c was produced
  bit  ok;

  initial begin
    d = new();
    repeat (3) begin                    // three complete cycles
      seen = '{default: 0};
      repeat (4) begin
        if (!d.randomize()) $fatal(1, "randomize failed");
        $write("r=%0d c=%0d | ", d.r, d.c);
        seen[d.c]++;
      end
      ok = (seen[0] == 1) && (seen[1] == 1) && (seen[2] == 1) && (seen[3] == 1);
      $display(" every c value exactly once: %s", ok ? "yes" : "NO");
    end
  end

endmodule
```

What to expect. Three lines, each showing four randomizations. Within each line the four `c` values are a permutation of 0, 1, 2, 3, so every line ends with `yes`. The `r` values follow no such rule and will often repeat within a line. Sample output; actual values differ per seed and simulator:

```
r=3 c=2 | r=3 c=0 | r=1 c=3 | r=0 c=1 |  every c value exactly once: yes
r=2 c=1 | r=2 c=3 | r=2 c=2 | r=0 c=0 |  every c value exactly once: yes
r=1 c=0 | r=3 c=2 | r=3 c=1 | r=1 c=3 |  every c value exactly once: yes
```

Line by line

Line	Explanation
`rand bit [1:0] r;`	Four possible values, each with probability 1/4 on every call.
`randc bit [1:0] c;`	Four possible values, delivered as a shuffled permutation.
`int seen[4];`	A small histogram used to prove the cyclic property.
Outer `repeat (3)`	Because the first call starts the first cycle, calls 1 to 4, 5 to 8 and 9 to 12 are exactly three complete cycles.
`seen[d.c]++;`	Counts each value.
`ok = ...`	In a complete cycle every count must be exactly 1.

The classic randc bug: a new object every time

```
// WRONG: randc has no effect
repeat (8) begin
  cfg c = new();                 // a brand-new object: its randc history is empty
  void'(c.randomize());
  $display("%0d", c.id);         // values repeat
end

// RIGHT: one object, randomized repeatedly
cfg c = new();
repeat (8) begin
  void'(c.randomize());
  $display("%0d", c.id);         // 8 different values
end
```

The cyclic history is stored in the object. In a generator that creates a new transaction object for every item (the normal style, see Module 9), a `randc` field in the transaction therefore does **not** cycle. The fix is to keep the `randc` variable in a long-lived object (the generator itself or a configuration object) and copy its value into each transaction, or to randomize one "blueprint" object repeatedly and send copies.

rand versus randc

Property	`rand`	`randc`
Meaning	Random	Random-cyclic
Repetition	A value can repeat at any time	No repeat until all legal values have been used
Distribution in one call	Uniform over the legal values	Uniform over the values not yet used in this cycle
History kept between calls	None	Yes, per object

Property	`rand`	`randc`
Allowed types	Integral types, enums, packed structs, arrays, queues, object handles (and unpacked struct members declared `rand`)	Integral types and enums only (and arrays of them)
Size limit	None	Implementation-defined; at least 8 bits guaranteed
Solve order	Together with the other `rand` variables	Before all `rand` variables
`dist`, `soft`, `solve before`	Allowed	Not allowed
Cost	Low	Higher: history must be stored
Typical use	Data, addresses, lengths, delays	IDs, slots, ports, test-case selection, anything that should be visited exhaustively without repeats

Interview trap: "With `randc bit [1:0] x`, can the same value come twice in a row?" Yes, across a cycle boundary. The last value of one permutation can equal the first value of the next. `randc` guarantees no repetition *within* a cycle only.

Interview trap: "Is the order of a `randc` cycle the same in every cycle?" No. A new random permutation is computed for each cycle.

Common mistake: Using `randc` on a wide variable such as `randc bit [31:0] addr;`. The standard only guarantees 8 bits; tools that accept it need a great deal of memory or switch to an approximation. For "no repeated addresses" on wide values use a queue of used values and an `inside` constraint (Problem 10 in Part 5).

Verification application: `randc` is used where each value should be exercised once before any is repeated: selecting which of 16 ports to drive, which register to access in a register sweep, which interrupt to raise, or which test scenario to run. It reaches full coverage of that variable in the minimum number of transactions.

Interview questions

Q1. What is the difference between `rand` and `randc`? `rand` chooses each value independently with a uniform distribution, so values can repeat. `randc` is cyclic: it goes through all legal values in a random order and repeats none until all have been used; then it starts a new random permutation.

Q2. How many `randomize()` calls are needed to see all values of `randc bit [3:0] x`? Of `rand bit [3:0] x`? Exactly 16 for `randc`. For `rand` there is no guarantee; on average about 54 calls are needed to see all 16 values (the "coupon collector" result), and any particular run may need more.

Q3. Can a `randc` variable have constraints? Yes. The cycle then covers only the values that satisfy the constraints. If the constraints change, the cycle is recomputed.

Q4. Which is solved first, `rand` or `randc`? `randc` variables are solved first. As a consequence a `randc` value can be chosen that makes the remaining constraints unsatisfiable in cases where treating everything together would have found a solution.

Q5. Why does `randc` not work when I create a new object for each transaction? The record of already-used values is stored inside the object. A new object has an empty record, so each call is the first call of a new cycle.

Q6. Can a `real` or a `string` be declared `rand`? A `string` cannot. Up to IEEE 1800-2017 the standard solver handles integral types only, so `rand real` is not standard there; some simulators accept it as an extension and later revisions of the standard add real-valued random variables. In an interview, say "integral types only, tool support for reals varies".

Q7. What values does a `rand` enum variable get? Only the named members of the enumeration, each with equal probability if there are no other constraints. The solver never produces an out-of-range enum value.

Q8. What happens if a `rand logic [7:0]` variable is randomized? It receives two-state values only. The solver never generates X or Z.

1.3 The randomize() Method and Randomizing Class Properties

Definition

Every class has a built-in method with this prototype:

```
virtual function int randomize();
```

It cannot be overridden or redefined by the user. A call `obj.randomize()`:

1. finds new values for all active random variables of the object that satisfy all active constraints,
2. writes those values into the object,
3. returns **1** on success and **0** on failure.

What randomize() does, step by step

```
obj.randomize()
     |
     v
1. call obj.pre_randomize()              (and pre_randomize of rand member objects)
     |
     v
2. collect:  - rand / randc variables whose rand_mode is ON
             - constraint blocks whose constraint_mode is ON
             - inline constraints from "with { ... }"
             - current values of all state variables
     |
     v
3. solve:    - randc variables first
             - array sizes before array elements
             - arguments of functions called in constraints before their results
             - everything else simultaneously
     |
     +-----------------------------+
     | solution found              | no solution
     v                             v
4. assign the new values       4. leave ALL variables unchanged
5. call obj.post_randomize()   5. post_randomize() is NOT called
6. return 1                    6. return 0   (most tools also print a message)
```

Which properties are randomized

Property	Randomized by `obj.randomize()`?
Declared `rand` or `randc`, with `rand_mode` on (the default)	Yes
Declared `rand` / `randc` but switched off with `rand_mode(0)`	No. It becomes a state variable
Not declared `rand`	No. It is a state variable; its current value is used in constraints
`rand` handle to another object, handle is not null	Yes: the solver also randomizes that object's random variables and applies that object's constraints, all in one solve
`rand` handle that is null	Skipped. The solver never creates objects
Non-`rand` handle to another object	Not randomized, even if that class has `rand` properties
`static rand` property	Yes; the one shared copy is randomized on every call from any object

Complete example

This example needs `alu_pkg.sv` from Module 1.

tb_randomize_basic.sv

```
module tb_randomize_basic;
  import alu_pkg::*;

  class alu_txn;
    rand bit [7:0]  a, b;
    rand alu_op_e   op;
    logic [8:0]     result;          // not random: filled in later by the monitor
    int unsigned    id;              // not random: a state variable

    constraint id_link_c { a >= id; }   // a state variable used in a constraint

    function void print(string tag = "");
      $display("%s id=%0d a=%0d b=%0d op=%s", tag, id, a, b, op.name());
    endfunction
  endclass

  alu_txn txn;
  int     status;

  initial begin
    txn = new();

    txn.id = 250;
    status = txn.randomize();
    $display("status=%0d", status);
    txn.print("[1]");

    txn.id = 300;                       // a (8 bits) can never be >= 300
    status = txn.randomize();
    $display("status=%0d", status);
    txn.print("[2]");
  end

endmodule
```

What to expect. The first call succeeds: `status=1`, and `a` is between 250 and 255. `b` is any 8-bit value and `op` is one of the eight named opcodes. The second call must fail: `status=0`, and the `[2]` line shows `id=300` with exactly the same `a`, `b` and `op` as the `[1]` line, because a failed `randomize()` leaves the random variables

untouched. Most simulators also print a constraint-failure message. Sample output; the values of `a`, `b` and `op` differ per seed and simulator:

```
status=1
[1] id=250 a=253 b=41 op=ALU_XOR
status=0
[2] id=300 a=253 b=41 op=ALU_XOR
```

Line by line

Line	Explanation
`rand bit [7:0] a, b;`	Two random 8-bit fields.
`rand alu_op_e op;`	A random enum. Only `ALU_ADD` to `ALU_SHR` can be produced.
`logic [8:0] result;`	Not `rand`. It is the DUT's response and must never be randomized.
`int unsigned id;`	Not `rand`. Set by the generator.
`constraint id_link_c { a >= id; }`	`id` is a state variable. At the moment of the call its value (250, later 300) is read and treated as a constant.
First `randomize()`	The solver must find `a` in 250 to 255. Six values are legal.
`txn.id = 300;`	Now the constraint demands `a >= 300`, which an 8-bit variable cannot satisfy.
Second `randomize()`	No solution exists. The method returns 0 and writes nothing.

How to call randomize()

Style	Verdict
`if (!txn.randomize()) $fatal(1, "randomize failed");`	Recommended. Always executes, always checks
`if (!txn.randomize()) $error(...);`	Fine when the test should continue
`assert (txn.randomize()) else $fatal(1, "...");`	Common, but risky: if assertions are disabled (for example with `$assertoff` or a simulator option), some tools do not evaluate the expression at all, so the object is **not randomized**
`void'(txn.randomize());`	Discards the status. Acceptable only in throwaway code
`txn.randomize();`	Calling a function and ignoring its result gives a warning in most tools, and failures go unnoticed

Interview trap: "What happens to the random variables when `randomize()` fails?" They keep their previous values. Nothing is partially assigned. `post_randomize()` is not called. Candidates often answer "they become 0" or "they become X".

Common mistake: Forgetting `rand`. A property without `rand` is never changed by `randomize()`. If it is still mentioned in a constraint it acts as a constant, and a constraint such as `len < 10` with a non-random `len` of 0 is silently satisfied, or silently causes failures when `len` is 20.

Common mistake: Randomizing a null handle. `alu_txn txn; txn.randomize();` is a null-object access. Call `new()` first.

Interview questions

Q1. What does `randomize()` return? An `int`: 1 if the solver found a solution and assigned it, 0 if the constraints cannot be satisfied.

Q2. Can you override `randomize()`? No. It is a built-in virtual method. User code is added through the hooks `pre_randomize()` and `post_randomize()`.

Q3. What is a state variable? Any variable used in a constraint that is not being randomized in this call: a non-`rand` property, a `rand` property whose `rand_mode` is off, or a variable from an enclosing scope. The solver reads its current value and treats it as a constant.

Q4. Does `randomize()` create objects for `rand` handles? No. A `rand` handle that is null is skipped. If it is not null, the object it points to is randomized as part of the same solve.

Q5. Why should the return value of `randomize()` be checked? On failure the old values stay in place and no error is raised by the language itself. Without a check, the testbench silently re-sends the previous transaction or uses uninitialised values.

Q6. Why is `assert(obj.randomize())` discouraged? An immediate assertion can be switched off. In that case the expression inside it may not be evaluated, so the randomization does not happen and the test behaves differently with assertions on and off.

1.4 Randomizing Arrays

In plain English

A packet has a payload of several bytes; a burst has several beats. Stimulus therefore often needs a random **collection** of values, and frequently the **number** of elements must be random as well. Declaring an array `rand` makes every element random. For dynamic arrays and queues the size can also be random.

Rules by array type

Array type	Declared `rand` means	Size
Fixed-size array `rand bit [7:0] d[4];`	Every element is randomized	Fixed
Dynamic array `rand bit [7:0] d[];`	Every element is randomized	Randomized **only if** `d.size()` appears in a constraint. Otherwise the current size is kept
Queue `rand bit [7:0] q[$];`	Same as a dynamic array	Same as a dynamic array
Associative array `rand int aa[string];`	The elements that already exist are randomized; keys are not created or changed	Not changed
Array of object handles `rand item it[];`	Each non-null element's object is randomized. Null elements are skipped; objects are never created	As above

> **Key point:** If the size of a `rand` dynamic array or queue is not constrained, the array is **not resized**. An array that is empty before `randomize()` is still empty afterwards. This is the most common surprise with random arrays.

How it works internally

```
rand bit [7:0] payload[];
constraint len_c { payload.size() inside {[2:5]}; }

Solve step 1:  payload.size()  is treated as a random variable -> say 4
Solve step 2:  the array is resized to 4 elements
Solve step 3:  payload[0..3] are random variables; foreach constraints are
               expanded for exactly these 4 elements and solved
```

The size is always solved **before** the elements. This is an implicit ordering similar to `solve ... before` (Section 3.3): a constraint on the elements cannot "push back" on the choice of size. If the size is chosen so that the element constraints cannot be met, randomization fails. Write size constraints that are safe by themselves.

Syntax

```
class pkt;
  rand bit [7:0] hdr[4];                // fixed array
  rand bit [7:0] payload[];             // dynamic array
  rand int       gaps[$];               // queue

  constraint size_c {
    payload.size() inside {[1:8]};      // size is random, 1 to 8
    gaps.size() == payload.size();      // sizes can be related to each other
  }
  constraint value_c {
    foreach (payload[i]) payload[i] != 8'h00;     // constraint on every element
    foreach (gaps[i])    gaps[i] inside {[0:3]};
  }
endclass
```

`foreach` inside a constraint is covered in Section 2.5.

Complete example

tb_rand_array.sv

```
module tb_rand_array;

  class pkt;
    rand bit [7:0] hdr[2];
    rand bit [7:0] payload[];
    rand bit [3:0] nosize[];            // size is never constrained

    constraint size_c  { payload.size() inside {[2:5]}; }
    constraint value_c { foreach (payload[i]) payload[i] inside {[8'h10:8'h1F]}; }
  endclass

  pkt p;

  initial begin
    p = new();
    $display("before : payload.size=%0d nosize.size=%0d", p.payload.size(), p.nosize.size());

    if (!p.randomize()) $fatal(1, "randomize failed");
    $display("after  : payload.size=%0d nosize.size=%0d", p.payload.size(), p.nosize.size());
    $display("hdr=%p payload=%p", p.hdr, p.payload);

    p.nosize = new[3];                  // give it a size by hand
    if (!p.randomize()) $fatal(1, "randomize failed");
    $display("again  : payload.size=%0d nosize.size=%0d nosize=%p",
             p.payload.size(), p.nosize.size(), p.nosize);
  end

endmodule
```

What to expect. The `before` line is exact: both sizes are 0. After the first randomization `payload.size` is between 2 and 5 and every payload element is between `10` and `1f` hexadecimal, while `nosize.size` is still 0 because its size is unconstrained. After `new[3]`, `nosize` keeps the size 3 and its three elements are random 4-bit values. Sample output; actual values differ per seed and simulator, and the layout printed by `%p` varies:

```
before : payload.size=0 nosize.size=0
after  : payload.size=4 nosize.size=0
hdr='{'h3a, 'hc1} payload='{'h17, 'h10, 'h1e, 'h12}
again  : payload.size=2 nosize.size=3 nosize='{'h9, 'h0, 'hd}
```

Line by line

Line	Explanation
`rand bit [7:0] hdr[2];`	Fixed array. Both elements are randomized on every call.
`rand bit [7:0] payload[];`	Dynamic array, initially empty.
`payload.size() inside {[2:5]};`	Makes the size random. The solver resizes the array.
`foreach (payload[i]) ...`	Applied to each of the elements that exist after the size has been chosen.
`rand bit [3:0] nosize[];`	No size constraint: the solver keeps whatever size the array has.
`p.nosize = new[3];`	The size is set procedurally. From now on three elements are randomized.

Always bound the size

```
rand int data[];
constraint bad_c  { data.size() > 0; }             // upper bound: about 2 billion elements
constraint good_c { data.size() inside {[1:64]}; } // always give an upper bound
```

With only a lower bound, the solver is free to choose a size in the millions. The simulation then appears to hang or runs out of memory. Always constrain a random size from both sides.

Arrays of objects

tb_rand_obj_array.sv

```
module tb_rand_obj_array;

  class beat;
    rand bit [7:0] data;
    constraint c { data inside {[1:9]}; }
  endclass

  class burst;
    rand beat beats[];                      // array of handles

    function new(int n);
      beats = new[n];
      foreach (beats[i]) beats[i] = new();   // the objects must be constructed by hand
    endfunction
  endclass

  burst b;

  initial begin
    b = new(3);
    if (!b.randomize()) $fatal(1, "randomize failed");
    foreach (b.beats[i]) $display("beat %0d: data=%0d", i, b.beats[i].data);
  end

endmodule
```

What to expect. Three lines, `beat 0` to `beat 2`, each with `data` between 1 and 9.

The array holds handles. `rand` on the array means "randomize the objects these handles point to". The solver never calls `new()`. If the size were constrained and the solver enlarged the array, the new elements would be null handles and would simply be skipped. The standard pattern is therefore: allocate the maximum number of objects in the constructor or in `pre_randomize()`, and constrain a separate `rand int n_beats` that says how many of them are used.

Multidimensional arrays

tb_rand_2d.sv

```
module tb_rand_2d;

  class grid;
    rand bit [3:0] m[3][4];                // fixed 3 x 4
    rand int       d[][];                  // dynamic rows of dynamic columns

    constraint m_c {
      foreach (m[i, j]) m[i][j] inside {[1:9]};
    }
    constraint d_size_c {
      d.size() == 2;                       // number of rows
      foreach (d[i]) d[i].size() inside {[1:3]};   // length of each row
    }
    constraint d_val_c {
      foreach (d[i, j]) d[i][j] inside {[100:109]};
    }
  endclass

  grid g;

  initial begin
    g = new();
    if (!g.randomize()) $fatal(1, "randomize failed");
    foreach (g.m[i]) $display("m[%0d] = %p", i, g.m[i]);
    foreach (g.d[i]) $display("d[%0d] = %p", i, g.d[i]);
  end

endmodule
```

What to expect. Three lines `m[0]` to `m[2]`, each a row of four values between 1 and 9. Then exactly two lines `d[0]` and `d[1]`, each with one to three values between 100 and 109; the two rows may have different lengths.

Line by line

Line	Explanation
`rand bit [3:0] m[3][4];`	Twelve random elements.
`foreach (m[i, j])`	One `foreach` with two loop variables visits every element of a two-dimensional array. Note the syntax: `m[i, j]` in the `foreach` header, `m[i][j]` in the body.
`rand int d[][];`	A dynamic array of dynamic arrays: a "jagged" array whose rows can differ in length.
`d.size() == 2;`	Size of the first dimension.
`foreach (d[i]) d[i].size() inside {[1:3]};`	Size of each row. Each row size is its own random variable.
`foreach (d[i, j]) ...`	Element constraints, expanded after all sizes are known.

> **Note:** Constraining the sizes of a multidimensional dynamic array as shown is supported by the major commercial simulators, but it is one of the areas where tool behaviour has differed between versions. If a tool rejects it, allocate the array in `pre_randomize()` using separately randomized size variables.

Common mistakes

> **Common mistake:** Expecting an unconstrained-size dynamic array to receive a random size. It keeps its current size; an empty array stays empty.

> **Common mistake:** Writing a constraint on `payload[0]` when the size may be 0. If the solver picks size 0, the index is out of range and the randomization fails or reports an error. Guard it: `payload.size() > 0 -> payload[0] == 8'hA5;` or make the minimum size 1.

> **Common mistake:** Declaring `rand item arr[];` and expecting the solver to create the `item` objects. It does not.

> **Verification application:** A packet class typically has `rand bit [7:0] payload[];` with `payload.size()` constrained to the legal packet lengths, a `dist` on the size to favour minimum and maximum lengths, and a `crc` field computed from the payload in `post_randomize()`.

Interview questions

Q1. How do you randomize the size of a dynamic array? Declare the array `rand` and constrain `arr.size()`, for example `arr.size() inside {[1:10]};`. The solver chooses the size first, resizes the array, then randomizes the elements.

Q2. What happens if a `rand` dynamic array has no size constraint? Its size is left unchanged. The existing elements are randomized. If the array is empty, nothing happens.

Q3. Which is solved first, the size of an array or its elements? The size. Element constraints are then expanded for that size.

Q4. How are the elements of a `rand` associative array randomized? Only the entries that already exist are randomized. No keys are added or removed.

Q5. How do you randomize an array of class objects? Declare the array of handles `rand`, construct every object before calling `randomize()` (in the constructor or in `pre_randomize()`), and then randomize the containing object. The solver randomizes each non-null element and never constructs objects.

Q6. How do you constrain each element of a two-dimensional array? With a `foreach` that has one loop variable per dimension: `foreach (m[i, j]) m[i][j] < 10;`.

Q7. Why must a random array size have an upper bound? Without one, the solver may choose a very large size, consuming memory and time. A size constraint such as `size() > 0` permits sizes up to the maximum of a 32-bit integer.

Part 2: Constraint Blocks and Constraint Expressions

2.1 Constraint Blocks and How the Solver Thinks

In plain English

A constraint is a rule that the random values must obey: "the length is less than 64", "the address is word-aligned", "if the packet is a read, there is no payload". You write the rules; the simulator finds values that obey **all of them at once**.

The most important idea in this whole module is that constraints are **rules, not instructions**. They are not executed from top to bottom like statements. A constraint does not "assign" anything. The solver looks at the complete set of rules and picks one combination of values for which every rule is true.

Technical definition

A **constraint block** is a named class member introduced by the keyword `constraint`. It contains a list of **constraint expressions**. Each expression is a Boolean condition over random variables, state variables and constants. A `randomize()` call succeeds only if values exist that make every expression of every active constraint block true.

Constraints are:

- **Declarative.** They state what must be true, not how to compute it.
- **Unordered.** The order of expressions in a block, and of blocks in a class, has no effect on which solutions are legal.
- **Bidirectional.** A relation between two variables restricts both of them. `a < b` limits `a` as much as it limits `b`.
- **Solved simultaneously.** All active constraints, from all blocks, base classes and inline `with` clauses, form one problem.

Syntax

```
class bus_txn;
  rand bit [15:0] addr;
  rand bit [7:0]  len;
  rand bit        write;

  // constraint <name> { <expression>; <expression>; ... }
  constraint addr_c {
    addr[1:0] == 2'b00;            // each expression ends with a semicolon
    addr inside {[16'h1000:16'h1FFF]};
  }
  constraint len_c { len > 0; len <= 64; }

  constraint empty_c { }           // legal: an empty block restricts nothing

  constraint ext_c;                // prototype: body is written outside the class
  static constraint shared_c { len != 13; }
endclass

constraint bus_txn::ext_c { write == 1 -> len < 16; }   // external body
```

What may appear inside a constraint block

Item	Example	Section
Relational or arithmetic expression	`len <= 64;` `a + b == 100;`	2.2
Set membership	`addr inside {[0:15], 32};`	2.3
Implication	`write -> len < 16;`	2.4
`if` ... `else`	`if (write) len < 16; else len == 0;`	2.4
Iteration	`foreach (d[i]) d[i] > 0;`	2.5
Uniqueness	`unique {a, b, c};`	2.6
Distribution	`len dist {1 := 5, [2:10] := 1};`	3.1
Soft constraint	`soft len == 4;`	3.2
Ordering	`solve write before len;`	3.3
Function call	`len == count_ones(mask);`	3.4

What may **not** appear: assignments (`a = 5;`), procedural statements (`for`, `while`, `case`, `begin`...`end`), delays, tasks, and operators with side effects (`++`, `+=`). Expressions must be integral: real numbers and strings are not allowed in standard constraints.

> **Interview trap:** `constraint c { a = 5; }` is a syntax error. Inside a constraint, "equal to" is always `==`.

How the solver works

Think of every possible combination of values of the random variables as a point. Each constraint removes the points that violate it. What remains is the **solution space**. The solver picks one point from it. The standard requires that, in the absence of `dist` and `solve`...`before`, **every solution is equally likely**.

```
rand bit [1:0] x, y;
constraint c { x < y; }

All 16 combinations (x across, y down); "S" = solution, "." = removed by the constraint

           x=0   x=1   x=2   x=3
    y=0     .     .     .     .
    y=1     S     .     .     .
    y=2     S     S     .     .
    y=3     S     S     S     .

6 solutions, each chosen with probability 1/6.

Per-variable probabilities that result:
    x = 0 : 3/6      y = 0 : 0
    x = 1 : 2/6      y = 1 : 1/6
    x = 2 : 1/6      y = 2 : 2/6
    x = 3 : 0        y = 3 : 3/6
```

Two lessons from this small picture:

1. The constraint mentions `x` and `y` symmetrically, and it restricts both. `x` can never be 3 and `y` can never be 0, although neither is said explicitly.

2. **Uniform over solutions does not mean uniform per variable.** `x` is 0 half of the time. When a variable looks "skewed" in a regression, the cause is usually this: some of its values simply appear in more solutions than others. `solve ... before` (Section 3.3) exists to change this.

Complete example: bidirectionality

tb_constraint_bidir.sv

```
module tb_constraint_bidir;

  class pair;
    rand bit [3:0] x, y, z;

    constraint rel_c  { y == x + 2; }      // looks like "compute y from x"
    constraint pin_c  { y == 7; }          // but this fixes y, which forces x
    constraint z_lo_c { z > y; }
    constraint z_hi_c { z < 9; }
  endclass

  pair p;

  initial begin
    p = new();
    repeat (3) begin
      if (!p.randomize()) $fatal(1, "randomize failed");
      $display("x=%0d y=%0d z=%0d", p.x, p.y, p.z);
    end
  end

endmodule
```

EXPECTED OUTPUT

```
x=5 y=7 z=8
x=5 y=7 z=8
x=5 y=7 z=8
```

Line by line

Line	Explanation
`y == x + 2;`	A relation between `x` and `y`. It is not an assignment and has no direction.
`y == 7;`	Fixes `y`. Because the relation must also hold, `x` must be 5. The solver worked "backwards" from `y` to `x`.
`z > y;` and `z < 9;`	Two separate blocks. All blocks are combined: `z` must be greater than 7 and less than 9, so `z` is 8.
Output	There is exactly one solution, so every call returns it. This is one of the few random examples with a fully predictable output.

Constraint block details

Feature	Behaviour
Several blocks in one class	All are ANDed together. Splitting constraints into small named blocks is good practice because each can be switched off separately with `constraint_mode()`
Block names	Must be unique within the class. A derived class may reuse a base-class name to override it (Section 4.7)

Feature	Behaviour
External constraint	A prototype `constraint name;` (or `extern constraint name;`) inside the class, and a body `constraint class_name::name { ... }` outside. If no body is ever provided, the prototype form without `extern` is treated as an empty constraint (tools warn)
`static constraint`	The constraint itself is not different, but its on/off state is shared: `constraint_mode()` on one object changes it for all objects of the class
State variables	Read once, when `randomize()` is called
Four-state values	Constraints are solved in two-state. A state variable that contains X or Z when it is used in a constraint is an error

Common mistakes

> **Common mistake:** Reading constraints as a program. In `constraint c { a < 10; b == a + 1; a > 5; }` the solver does not "first choose `a` below 10, then compute `b`". It finds any `(a, b)` pair for which all three conditions are true.

> **Common mistake:** Expecting each variable to be uniformly distributed because there is "only one small constraint". See the `x < y` picture above.

> **Common mistake:** Putting two contradictory constraints in different blocks (or in a base and a derived class) and wondering why randomization fails. Blocks are not alternatives; all active blocks must hold together.

Interview questions

Q1. What is a constraint block? A named member of a class, declared with the keyword `constraint`, that contains Boolean expressions which the random variables of the object must satisfy after `randomize()`.

Q2. Are constraints executed in order? No. Constraints are declarative and are solved simultaneously. The textual order of expressions and blocks does not affect the set of legal solutions.

Q3. What does "constraints are bidirectional" mean? A constraint relating several variables restricts all of them, regardless of how it is written. `y == x + 2` together with `y == 7` forces `x` to 5. Likewise `a -> b` also means "if `b` is false then `a` must be false".

Q4. If there are no `dist` or `solve before` constraints, how are solutions chosen? Every legal combination of values (every solution) has the same probability. Individual variables are therefore not necessarily uniformly distributed.

Q5. What is an external constraint? A constraint whose prototype is declared in the class and whose body is defined outside it with the scope operator, `constraint cls::name { ... }`. It allows the body to be placed in a different file, for example one file per test.

Q6. What is a static constraint? A constraint declared `static`. Calling `constraint_mode()` on it through any object switches it on or off for every object of that class.

Q7. Can a constraint contain a non-random variable? Yes. It is a state variable: its current value is used as a constant. This is how a test "steers" a transaction: it sets a non-random knob and constraints refer to it.

2.2 Relational and Arithmetic Constraint Expressions

Definition

A constraint expression is any integral expression. The solver requires it to be **true** (non-zero). The usual forms are:

- **Relational constraints:** comparisons using `<`, `<=`, `>`, `>=`, `==`, `!=`.
- **Arithmetic constraints:** comparisons whose operands contain `+`, `-`, `*`, `/`, `%`, `<<`, `>>`.
- **Logical combinations:** using `&&`, `||`, `!`.
- **Bit-level constraints:** using bit-selects, part-selects, concatenation, bitwise and reduction operators, and system functions such as `$countones`, `$onehot`, `$clog2`.

Syntax and examples

```
class expr_demo;
  rand bit [7:0]  a, b, c;
  rand bit [15:0] addr;
  rand int        delay;

  constraint relational_c {
    a > 10;
    a <= 100;
    b != a;
    c >= a && c <= b || c == 0;        // && binds tighter than ||
  }
  constraint arithmetic_c {
    a + b < 200;
    a % 4 == 0;                        // multiple of 4
    b == 2 * c + 1;
    delay inside {[0:20]};
  }
  constraint bit_level_c {
    addr[1:0] == 2'b00;                // word aligned
    addr[15:12] != 4'hF;               // not in the top 4K region
    $countones(a) == 3;                // exactly three bits set
    (addr & 16'h00FF) != 0;            // low byte not zero
  }
endclass
```

Operators in constraints

Operator group	Allowed	Notes
`== != < <= > >=`	Yes	The basic building blocks
`&& || !`	Yes	`a && b` in a constraint is the same as writing `a;` and `b;` as two expressions
`+ - *`	Yes	Watch widths and overflow (below)
`/ %`	Yes	Slower to solve. Division by a random variable must never have a zero divisor
`<< >> >>> <<<`	Yes	
`& | ^ ~`, reduction operators	Yes	
`?:`	Yes	`len == (write ? 8 : 0);`

Operator group	Allowed	Notes
`inside`, `dist`, `->`, `<->`	Yes	Dedicated sections
`**`	Tool-dependent	Often restricted to constant operands. Prefer a shift for powers of two
`===`, `!==`, `==?` with X/Z	No	Constraints are two-state
`++`, `--`, assignment operators	No	No side effects are allowed

The traps: these are what interviewers test

Constraint expressions follow exactly the same width and sign rules as any other SystemVerilog expression (Module 1, Sections 2.6 and 4.4). The solver does not "know what you meant". Six traps cover most questions.

Trap 1: chained comparison

```
rand bit [7:0] x;
constraint wrong_c { 10 < x < 20; }          // does NOT mean "x between 10 and 20"
constraint right_c { x > 10; x < 20; }       // or: x inside {[11:19]};
```

`10 < x < 20` is parsed as `(10 < x) < 20`. `(10 < x)` is 0 or 1, and both are less than 20. The constraint is always true, so `x` is unconstrained.

Trap 2: overflow in a narrow expression

```
rand bit [7:0] a, b;
constraint narrow_c { a + b == 8'd10; }      // evaluated in 8 bits
constraint wide_c   { a + b == 10; }         // evaluated in 32 bits
```

In `narrow_c` every operand is 8 bits wide, so the sum is computed in 8 bits and wraps around. `a` = 200, `b` = 66 is a legal solution because 266 modulo 256 is 10. In `wide_c` the literal `10` is a 32-bit integer, the operands are extended to 32 bits first, and only true sums of 10 are accepted. When the mathematical value matters, make sure something in the expression is wide enough, or cast: `int'(a) + int'(b) == 10`.

Trap 3: unsigned subtraction

```
rand bit [3:0] a, b;
constraint wrong_c { a - b > 0; }            // intended: a greater than b
constraint right_c { a > b; }
```

`a` and `b` are unsigned, so the whole expression is unsigned (one unsigned operand makes everything unsigned). For `a` = 2, `b` = 5 the result is not -3 but a huge positive 32-bit number, which is greater than 0. The constraint `a - b > 0` only excludes `a == b`. Never express an ordering through a subtraction of unsigned values.

Trap 4: signed random variables

```
rand int  delay;
rand byte offset;
constraint c1 { delay < 100; }               // also allows -2,147,483,648 to -1
constraint c2 { offset < 50; }               // also allows -128 to -1
```

`int`, `byte`, `shortint` and `longint` are signed. A constraint with only an upper bound leaves all negative values legal, and there are far more of those. Either use unsigned types (`int unsigned`, `bit [7:0]`) or always give both bounds: `delay inside {[0:99]};`.

Trap 5: a non-random variable

```
bit [7:0] len;                              // "rand" forgotten
constraint c { len inside {[1:10]}; }
```

`len` is a state variable with value 0. The constraint is false, and nothing can be changed to make it true: `randomize()` fails. If `len` happened to hold a legal value, the call would succeed and `len` would silently never change.

Trap 6: operator precedence and `!`

```
constraint wrong_c { !x inside {[1:5]}; }    // parsed as (!x) inside {[1:5]}
constraint right_c { !(x inside {[1:5]}); }
```

Unary `!` binds tighter than `inside`. `!x` is 0 or 1, so the wrong version means "x is zero".

Complete example

tb_constraint_traps.sv

```
module tb_constraint_traps;

  class traps;
    rand bit [7:0] a, b;        // for the overflow trap
    rand bit [3:0] p, q;        // for the subtraction trap
    rand bit [7:0] x;           // for the chained comparison

    constraint narrow_c { a + b == 8'd10; }
    constraint sub_c    { p - q > 0; }
    constraint chain_c  { 10 < x < 20; }
  endclass

  traps t;
  int   wrapped, p_less, x_outside;

  initial begin
    t = new();
    repeat (1000) begin
      if (!t.randomize()) $fatal(1, "randomize failed");
      if (int'(t.a) + int'(t.b) != 10) wrapped++;
      if (t.p < t.q)                   p_less++;
      if (!(t.x inside {[11:19]}))     x_outside++;
    end
    $display("a + b was not really 10 : %0d of 1000", wrapped);
    $display("p was less than q       : %0d of 1000", p_less);
    $display("x was outside 11..19    : %0d of 1000", x_outside);
  end

endmodule
```

What to expect. All three counts are large, which proves that none of the three constraints does what it appears to say. There are 256 solutions of `a + b == 8'd10` in 8-bit arithmetic and only 11 of them have a true sum of 10, so the first count is about 957. `p - q > 0` excludes only `p == q`; of the 240 remaining pairs half have `p < q`, so the second count is about 500. `x` is unconstrained and only 9 of 256 values lie in 11 to 19, so the third count is about 965. Exact counts vary per seed and simulator.

Tips for writing solvable constraints

Guideline	Reason
Prefer `inside` and simple comparisons to arithmetic	Easier for the solver and for the reader
Avoid `/` and `%` on wide variables; use bit-selects for powers of two (`addr[1:0] == 0` instead of `addr % 4 == 0`)	Division and modulus are expensive
Avoid multiplying two wide random variables	The solver may be slow or give up
Use unsigned types for quantities that cannot be negative	Removes the signed traps
Give every variable both a lower and an upper bound	Keeps values legal and solving fast
Keep constraint blocks small and named by purpose	They can then be disabled or overridden individually

Interview questions

Q1. What is wrong with `constraint c { 0 < x < 10; }`? It is parsed as `(0 < x) < 10`. The inner comparison is 0 or 1, which is always less than 10, so the constraint restricts nothing. Write `x > 0; x < 10;` or `x inside {[1:9]};`.

Q2. `rand bit [7:0] a, b; constraint c { a + b == 8'd255; }` versus `{ a + b == 255; }`: is there a difference? For 255 there is no difference in the solutions, because an 8-bit wrapped sum equal to 255 needs a true sum of 255 or 511, and 511 cannot be reached by two 8-bit values (maximum 510). For most other targets there is a difference: with an 8-bit literal the sum wraps, so true sums of target + 256 are also accepted. The general rule is to make the expression at least as wide as the largest possible sum.

Q3. `rand int x; constraint c { x < 5; }` What values can `x` take? Any value from -2,147,483,648 to 4. `int` is signed. Nearly every result will be a large negative number.

Q4. How do you constrain a 32-bit address to be 4-byte aligned? `addr[1:0] == 2'b00;`. `addr % 4 == 0` is equivalent but more expensive to solve.

Q5. Can you use `$countones` in a constraint? Yes. `$countones(x) == 1` generates one-hot values; `$onehot(x)` does the same. These bit-vector system functions are understood by the solver and remain bidirectional.

Q6. Why does `constraint c { a - b > 0; }` not guarantee `a > b` for unsigned `a` and `b`? The subtraction is performed in unsigned arithmetic and wraps to a large positive value when `a < b`. Only `a == b` gives 0. The correct constraint is `a > b`.

2.3 The inside Operator

In plain English

`inside` asks "is this value one of the following?". The list can contain single values, ranges and whole arrays. In a constraint it means "choose the value from this set".

Syntax

```
x inside {1, 2, 5};                    // a list of values
x inside {[10:20]};                    // a range, both ends included
x inside {[0:9], 50, [100:199]};       // a mixture
x inside {[lo:hi]};                    // bounds may be variables or expressions
x inside {[$:10]};                     // $ = lowest value of x's type
x inside {[250:$]};                    // $ = highest value of x's type
x inside {legal_q};                    // every element of an array or queue
x inside {legal_q, 255, [16:31]};      // arrays can be mixed with values and ranges
!(x inside {[10:20]});                 // exclusion: note the parentheses
```

Behaviour

Point	Explanation
Probability	Every value **in the resulting set** is equally likely. A range does not count as "one item": in `{0, [10:19]}` the value 0 has probability 1/11, not 1/2
Duplicates	Listing a value twice, or overlapping ranges, does not increase its probability. `inside` describes a set
Empty range	`[hi:lo]` with the low bound greater than the high bound contributes no values
Bounds	May be random variables. `x inside {[lo:hi]}` then also restricts `lo` and `hi` (bidirectional)
Arrays	`x inside {arr}` means `x` equals at least one element. If `arr` is itself `rand`, this relates `x` to the array elements
Negation	`!(x inside {...})` excludes the set
Matching	Uses `==?`, so X/Z/ `?` in a set value are wildcards (Module 1, Section 4.6)

Complete example

tb_inside.sv

```
module tb_inside;

  class sel;
    rand bit [7:0] x, y, z;
    bit [7:0] legal_q[$] = '{8'd3, 8'd7, 8'd42};   // state variable: a list of legal values
    bit [7:0] used_q[$]  = '{8'd100, 8'd101};

    constraint x_c { x inside {0, [10:19], 255}; }
    constraint y_c { y inside {legal_q}; }
    constraint z_c { z inside {[100:103]}; !(z inside {used_q}); }
  endclass

  sel s;
  int x_zero;

  initial begin
    s = new();
    repeat (1200) begin
      if (!s.randomize()) $fatal(1, "randomize failed");
      if (s.x == 0) x_zero++;
      if (!(s.y inside {3, 7, 42}))  $error("y illegal: %0d", s.y);
      if (!(s.z inside {102, 103}))  $error("z illegal: %0d", s.z);
    end
    $display("last: x=%0d y=%0d z=%0d", s.x, s.y, s.z);
    $display("x was 0 in %0d of 1200 calls", x_zero);
  end

endmodule
```

What to expect. No `$error` messages. In the `last` line `x` is 0, 255 or between 10 and 19; `y` is 3, 7 or 42; `z` is 102 or 103. The set for `x` has 12 members (0, the ten values 10 to 19, and 255), each equally likely, so `x` is 0 in about 100 of the 1200 calls, not in 400.

Line by line

Line	Explanation
`bit [7:0] legal_q[$] = '{...};`	A non-random queue. It can be filled or changed by the test at run time.
`x inside {0, [10:19], 255};`	Twelve legal values, uniformly distributed.
`y inside {legal_q};`	The value must be one of the queue's elements. Changing the queue changes the legal set without touching the class.
`!(z inside {used_q});`	Excludes values already used. Combined with `[100:103]`, only 102 and 103 remain.

inside compared with other ways of writing a set

Form	Meaning	Distribution
`x inside {[1:3], 9};`	x is 1, 2, 3 or 9	Uniform over the 4 values
`x == 1 || x == 2 || x == 3 || x == 9;`	Same set	Same
`x >= 1; x <= 3;`	Range only	Uniform
`x dist {[1:3] := 1, 9 := 1};`	Same set	Weighted; here also uniform over the 4 values (Section 3.1)
`x dist {[1:3] :/ 1, 9 :/ 1};`	Same set	9 has probability 1/2; 1, 2, 3 share the other half

> **Interview trap:** `x inside {[1:100], 200}` does **not** give 200 a 50 percent chance. Each of the 101 values has probability 1/101. To weight ranges against single values, `dist` is required.

> **Common mistake:** `x inside {[10:1]}`. The range is empty, so if this is the only item the constraint can never be satisfied and randomization fails. Ranges are always written `[low:high]`.

> **Verification application:** A test keeps a queue of valid register addresses read from the register map and constrains `addr inside {valid_addr_q};`. An error test inverts it with `!(addr inside {valid_addr_q});` to generate accesses to unmapped addresses.

Interview questions

Q1. How do you constrain a variable to a set of values and ranges? With `inside`: `x inside {1, [5:10], 20};`.

Q2. How do you exclude a set of values? `!(x inside {[5:10]});`. The parentheses are required because `!` has higher precedence than `inside`.

Q3. What is the probability of 200 in `x inside {[1:99], 200}`? 1/100. All 100 values of the set are equally likely.

Q4. Can the bounds of an `inside` range be random variables? Yes: `x inside {[lo:hi]};` with `rand` `lo` and `hi`. The solver then chooses all three so that `lo <= x <= hi`.

Q5. What is the difference between `inside` and `dist`? `inside` only restricts the value to a set, with a uniform distribution over the set. `dist` restricts to a set and also assigns weights that control how often each value or range is chosen.

2.4 Conditional Constraints: Implication, if-else and Equivalence

In plain English

Many rules only apply in certain situations: "**if** the transfer is a read, the payload is empty", "**if** the packet is long, the length is at least 1000". Conditional constraints express this. SystemVerilog offers two notations with the same meaning, the implication operator `->` and `if`...`else`.

The implication operator ->

`A -> B` reads "A implies B": **if A is true, B must be true. If A is false, B is unrestricted.** It is exactly equivalent to `!A || B`.

A	B	`A -> B`	Allowed by the constraint?
0	0	1	Yes
0	1	1	Yes
1	0	0	**No**
1	1	1	Yes

Only one combination is forbidden: A true and B false. Read the table again from B's side: if B is false, A cannot be true. So `A -> B` also means "**not B implies not A**". This is the bidirectional behaviour of implication, and it is the point interviewers probe.

```
constraint c1 { write -> len < 16; }                // single consequence
constraint c2 { (kind == LONG) -> { len > 1000; len < 1500; } }   // a set of consequences
constraint c3 { (a > 5 && b == 0) -> c inside {[1:3]}; }
```

if-else constraints

```
constraint len_c {
  if (kind == SHORT)       len inside {[1:63]};
  else if (kind == MEDIUM) len inside {[64:1023]};
  else                     len inside {[1024:1500]};
}

constraint multi_c {
  if (write) {
    len > 0;
    addr[1:0] == 0;
  } else {
    len == 0;
  }
}
```

- Braces `{ }` group several constraint expressions, not `begin`...`end`.
- `if (A) X; else Y;` is exactly equivalent to `(A -> X) && (!A -> Y)`.
- An `if` without `else` is exactly `A -> X`.
- An `else` belongs to the nearest unmatched `if`, as in procedural code.

Despite looking like procedural code, `if-else` in a constraint is still declarative and bidirectional. The solver does not "evaluate the condition first". It finds values for the condition and the branches together.

Equivalence

Sometimes the relation must hold in both directions: "the length is over 1000 **if and only if** the packet kind is LONG". Implication alone is one-way: `(kind == LONG) -> (len > 1000)` still allows a SHORT packet with length 1200.

```
// Three ways to say "A if and only if B"
constraint eq1_c { (kind == LONG) -> (len > 1000);  (len > 1000) -> (kind == LONG); }
constraint eq2_c { (kind == LONG) == (len > 1000); }      // both sides are 1-bit results
constraint eq3_c { (kind == LONG) <-> (len > 1000); }     // equivalence operator
```

The `<->` operator was added to the language in IEEE 1800-2009; if an older tool does not accept it inside a constraint, forms 1 and 2 always work.

How conditional constraints affect the distribution

```
rand bit       mode;
rand bit [1:0] len;
constraint imp_c { (mode == 1) -> (len == 3); }

Solutions (mode, len):
    (0,0) (0,1) (0,2) (0,3)        mode = 0 : len is free      -> 4 solutions
    (1,3)                          mode = 1 : len must be 3    -> 1 solution

5 solutions, each with probability 1/5:
    P(mode = 1) = 1/5       P(mode = 0) = 4/5
    P(len = 3)  = 2/5       P(len = 0)  = P(len = 1) = P(len = 2) = 1/5
```

Although `mode` is a single bit with "no constraint of its own", it is 1 only 20 percent of the time. With a 32-bit `len`, `mode` would be 1 about once in four billion calls: effectively never. This is the standard motivation for `solve mode before len` (Section 3.3).

Complete example

tb_implication.sv

```
module tb_implication;

  class cfg;
    rand bit       mode;
    rand bit [1:0] len;
    constraint imp_c { (mode == 1) -> (len == 3); }
  endclass

  cfg c;
  int mode_one;

  initial begin
    c = new();

    // Forward direction: force the condition
    if (!c.randomize() with { mode == 1; }) $fatal(1, "randomize failed");
    $display("forced mode=1 : mode=%0d len=%0d", c.mode, c.len);

    // Backward direction: force the consequence to be false
    if (!c.randomize() with { len == 2; }) $fatal(1, "randomize failed");
    $display("forced len=2  : mode=%0d len=%0d", c.mode, c.len);

    // Distribution
    repeat (1000) begin
      if (!c.randomize()) $fatal(1, "randomize failed");
      if (c.mode == 1) mode_one++;
    end
    $display("mode was 1 in %0d of 1000 calls", mode_one);
  end

endmodule
```

What to expect. The first two lines are fully determined by the constraints:

EXPECTED OUTPUT

```
forced mode=1 : mode=1 len=3
forced len=2  : mode=0 len=2
```

The third line reports a count of about 200 (one fifth of 1000), not 500. The exact count varies per seed and simulator.

Line by line

Line	Explanation
`(mode == 1) -> (len == 3);`	Forbids exactly one situation: `mode` = 1 with `len` not equal to 3.
`randomize() with { mode == 1; }`	An inline constraint (Section 4.4) added for this call only. With `mode` = 1 the implication forces `len` = 3.
`randomize() with { len == 2; }`	Now the consequence is false. The only way to keep the implication true is `mode` = 0. The constraint has worked from right to left.
The loop	Shows that `mode` is not 50/50. Four of the five solutions have `mode` = 0.

Implication versus if-else

Property	Implication `A -> B`	`if (A) B; else C;`
Meaning	`!A || B`	`(A -> B) && (!A -> C)`
"Otherwise" branch	None. Needs a second implication `!A -> C`	Built in
Several consequences	`A -> { B1; B2; }`	`if (A) { B1; B2; }`
Chains of cases	One implication per case	`else if` chain; conditions are prioritised, so later branches implicitly include "and none of the earlier conditions"
Bidirectional	Yes	Yes
Effect on distribution	Identical for equivalent constraints	Identical
Is an expression	Yes: can be combined, for example `(a -> b) && c`	No: it is a statement-like constraint item
Typical use	Short one-line dependencies	Multi-way selection that reads like a specification table

Interview trap: "Does `a -> b` restrict `a`?" Yes. If other constraints make `b` false, `a` must be false. And even without that, `a` becomes less likely when few values of the other variables satisfy `b`.

Interview trap: "In `if (x == 0) y == 0;`, is `x` chosen first?" No. There is no order. If another constraint says `y == 5`, then `x` can never be 0.

Common mistake: Using `begin` ... `end` inside a constraint `if`. Constraint sets are grouped with braces.

Common mistake: Writing an implication when an equivalence is meant. `err_inject -> crc_bad` lets `crc_bad` be 1 even when `err_inject` is 0. If the bad CRC must appear only when injection is on, add the reverse implication or use `==`.

Verification application: Conditional constraints encode protocol tables directly. For the ALU: `(op == ALU_NOT) -> (b == 0);` because the NOT operation ignores `b`, or, to stress overflow, `if (op == ALU_ADD) a + b > 255;`. Tests then switch such blocks on and off with `constraint_mode()`.

Interview questions

Q1. What does the implication operator do in a constraint? `A -> B` means that whenever A is true, B must be true. When A is false, B is unconstrained. It is equivalent to `!A || B`.

Q2. Is implication bidirectional? Yes. Because only "A true and B false" is forbidden, B being false forces A to be false. The solver uses the constraint in both directions.

Q3. What is the difference between `->` and `if-else` in constraints? They express the same logic. `if (A) B;` equals `A -> B`. `if-else` adds an "otherwise" part, which with implication needs a second constraint `!A -> C`. `if-else` is more readable for multi-way choices; `->` is compact and is an expression that can be combined with other operators.

Q4. `rand bit a; rand bit [3:0] b; constraint c { a -> b == 0; }` **What is the probability that** `a` **is 1?** There are 16 solutions with `a` = 0 and one with `a` = 1, so 1/17.

Q5. How do you make `a` **in the previous question 1 half of the time?** Add `solve a before b;`. The solver then chooses `a` first with equal probability for 0 and 1, and then chooses `b`.

Q6. How do you write "A if and only if B" as a constraint? `A <-> B`, or `A == B` when both are 1-bit conditions, or the two implications `A -> B; B -> A;`.

Q7. Given `constraint c { (a == 1) -> (b == 1); }` **and an inline constraint** `b == 0`**, what is** `a`**?** `a` is any value except 1.

2.5 foreach Constraints and Array Reduction

In plain English

A `foreach` constraint applies the same rule to every element of an array: "every payload byte is non-zero", "each element is greater than the one before it". Array reduction methods such as `sum()` let a single constraint talk about the array as a whole: "the elements add up to 100".

Syntax

```
class arr_demo;
  rand bit [7:0] d[];

  constraint size_c { d.size() inside {[4:8]}; }

  constraint each_c {                       // a rule for every element
    foreach (d[i]) d[i] inside {[1:50]};
  }
  constraint neighbour_c {                  // a rule between neighbours
    foreach (d[i]) if (i > 0) d[i] != d[i-1];
  }
  constraint index_c {                      // the index may be used in the rule
    foreach (d[i]) (i % 2 == 0) -> d[i][0] == 0;     // even positions hold even values
  }
  constraint total_c {                      // a rule for the whole array
    d.sum() with (int'(item)) == 100;
  }
endclass
```

How foreach works internally

A `foreach` constraint is **unrolled**. After the array size is known, the solver replaces the loop by one copy of the body for each index. The loop variable is a constant in each copy.

```
   d.size() == 3
   foreach (d[i]) if (i > 0) d[i] > d[i-1];

   is solved as:

      i = 0 :   if (0 > 0) ...            -> condition is constant false, no constraint
      i = 1 :   d[1] > d[0];
      i = 2 :   d[2] > d[1];
```

Consequences:

- The guard `if (i > 0)` is evaluated at unrolling time. It costs nothing and it is what prevents the illegal index `d[-1]`.
- A reference to `d[i+1]` or `d[i-1]` without a guard indexes outside the array for the last or first element. That is an error and the randomization fails.
- A large array with a nested `foreach` produces a large number of constraints (an array of 1000 elements compared pairwise gives about half a million). Solving time grows accordingly.

Array reduction methods in constraints

Method	Returns	Example constraint
`sum()`	Sum of the elements	`d.sum() with (int'(item)) == 100;`
`product()`	Product of the elements	`d.product() with (int'(item)) < 1000;`
`and()`	Bitwise AND of all elements	`d.and() == 0;`
`or()`	Bitwise OR of all elements	`d.or() == 8'hFF;`
`xor()`	Bitwise XOR of all elements	`d.xor() == 0;`

Only these reduction methods may be used in constraints. Locator and ordering methods (`find`, `sort`, `unique`, `min`, `max`...) are procedural and are not allowed; they belong in `post_randomize()`.

The optional `with (expression)` clause replaces each element by the value of the expression before reducing. `item` is the current element and `item.index` is its index.

```
d.sum() with (int'(item))                // sum computed in 32 bits
d.sum() with (int'(item > 10))           // how many elements are greater than 10
d.sum() with (item.index < 4 ? int'(item) : 0)     // sum of the first four elements
```

The sum() width trap

`sum()` returns a value **of the element type**. For an array of `bit [7:0]`, the sum is computed in 8 bits and wraps around.

```
rand bit [7:0] d[4];
constraint c { d.sum() == 20; }

Accepted by the solver:   5 +   5 +   5 +   5 =  20      (20 mod 256 = 20)
Also accepted:          100 + 100 +  50 +  26 = 276      (276 mod 256 = 20)
                        200 + 200 + 100 +  32 = 532      (532 mod 256 = 20)

Fix:   constraint c { d.sum() with (int'(item)) == 20; }
```

The same trap, in a worse form, applies to counting. `item > 10` is a **1-bit** value, so `d.sum() with (item > 10)` adds 1-bit numbers in 1-bit arithmetic: the result is only the parity of the count. Always cast inside the `with` clause: `d.sum() with (int'(item > 10)) == 3`.

Complete example

tb_foreach_sum.sv

```
module tb_foreach_sum;

  class arr_demo;
    rand bit [7:0] d[];
    rand bit [7:0] w[4];

    constraint size_c  { d.size() inside {[4:6]}; }
    constraint asc_c   { foreach (d[i]) if (i > 0) d[i] > d[i-1]; }
    constraint total_c { d.sum() with (int'(item)) == 100; }

    constraint wrap_c  { w.sum() == 20; }           // the trap: 8-bit sum
  endclass

  arr_demo a;
  int      true_sum, wrapped;

  initial begin
    a = new();
    repeat (200) begin
      if (!a.randomize()) $fatal(1, "randomize failed");

      // check d: strictly ascending, true sum 100
      true_sum = 0;
      foreach (a.d[i]) begin
        true_sum += a.d[i];
        if (i > 0 && a.d[i] <= a.d[i-1]) $error("d is not ascending: %p", a.d);
      end
      if (true_sum != 100) $error("sum of d is %0d", true_sum);

      // check w: count how often the real sum is not 20
      true_sum = 0;
      foreach (a.w[i]) true_sum += a.w[i];
      if (true_sum != 20) wrapped++;
    end
    $display("last d = %p", a.d);
    $display("w.sum() == 20 was satisfied by a wrapped sum in %0d of 200 calls", wrapped);
  end

endmodule
```

What to expect. No `$error` messages: `d` always has 4 to 6 strictly ascending elements with a true total of 100. The last line shows a count close to 200. Almost every solution of `w.sum() == 20` in 8-bit arithmetic has a real sum of 276, 532 or 788, because there are vastly more such combinations than combinations that really add up to 20.

Line by line

Line	Explanation
`d.size() inside {[4:6]};`	Solved first.
`foreach (d[i]) if (i > 0) d[i] > d[i-1];`	Unrolled into size - 1 comparisons. The guard removes the copy for `i` = 0.
`d.sum() with (int'(item)) == 100;`	Each element is widened to 32 bits before being added, so the constraint means the true sum.
`w.sum() == 20;`	Each element is 8 bits, so the sum is 8 bits. It wraps.
Checker code	Recomputes the sums in a 32-bit `int`. This kind of independent check is how you convince yourself that a constraint means what you intended.

Common mistakes

Common mistake: `foreach (d[i]) d[i] < d[i+1];` with no guard. For the last element `d[i+1]` does not exist. Write `if (i < d.size() - 1) d[i] < d[i+1];` or use the `i > 0` form with `d[i-1]`.

Common mistake: Forgetting the cast in `sum()`, or casting the result instead of the items. `int'(d.sum()) == 100` does not help: the wrap has already happened.

Common mistake: Using a `for` loop inside a constraint. Only `foreach` is allowed.

Interview trap: "The array size is random and the `foreach` constraint needs at least two elements. What happens when the solver picks size 1?" Nothing goes wrong if the body is guarded: the unrolled copies simply disappear. But remember that size is solved first. If size 1 makes some other constraint (for example a sum) impossible, the call fails; the solver does not go back and choose another size.

Verification application: Burst and packet classes use `foreach` for per-beat rules (strobe patterns, incrementing addresses: `foreach (addr[i]) if (i > 0) addr[i] == addr[i-1] + 4;`) and `sum()` to distribute a total over several fields, such as a frame's total length divided into a random number of fragments.

Interview questions

Q1. How do you constrain every element of an array? With a `foreach` constraint: `foreach (arr[i]) arr[i] < 100;`.

Q2. Write a constraint for an array in ascending order. `foreach (arr[i]) if (i > 0) arr[i] > arr[i-1];`. Use `>=` if equal neighbours are allowed.

Q3. Why is the guard `if (i > 0)` needed? For `i` = 0 the expression `arr[i-1]` would index element -1, which does not exist. The guard is resolved when the loop is unrolled, so that copy of the constraint is dropped.

Q4. `rand bit [3:0] a[10]; constraint c { a.sum() == 50; }` Does this guarantee the elements add up to 50? No. `sum()` returns a value of the element type, here 4 bits. A 4-bit sum can only be 0 to 15 and can never equal 50, so the randomization fails. With a target below 16 it would succeed, but it would also accept wrapped sums (a true sum of 21 looks like 5). The correct form is `a.sum() with (int'(item)) == 50`.

Q5. How do you constrain exactly three elements of an array to be zero? `arr.sum() with (int'(item == 0)) == 3;`. The comparison gives 1 for each zero element, and the cast makes the sum 32 bits wide.

Q6. Which array methods can be used inside constraints? The reduction methods `sum`, `product`, `and`, `or` and `xor`, optionally with a `with` clause, plus `size()`. Other array methods are not allowed.

Q7. How does the solver handle `foreach`? It first solves the array size, then unrolls the loop into one constraint per element, with the index as a constant in each copy, and solves all of them together.

2.6 unique Constraints

In plain English

`unique` says "all of these must be different from one another". It is the direct way to ask for an array with no repeated values, or for several variables that must not collide.

Syntax

```
class uniq_demo;
  rand bit [3:0] a, b, c;
  rand bit [7:0] ids[6];
  rand bit [2:0] perm[8];

  constraint scalars_c { unique {a, b, c}; }         // three different values
  constraint array_c   { unique {ids}; }             // all six elements differ
  constraint mixed_c   { unique {ids[0:2], a}; }     // a slice and a scalar together
  constraint perm_c    { unique {perm}; }            // 8 different 3-bit values
endclass
```

Technical definition

`unique { list }` is a constraint item (added in IEEE 1800-2012) whose list contains random variables, arrays and array slices. After randomization, no two members of the expanded list have the same value. It is equivalent to writing `!=` between every pair of members.

Rule	Detail
Members	Scalars, whole arrays, array slices. All must have equivalent types
Not allowed in the list	`randc` variables
Number of values	If the list has more members than the type has values (for example five `bit [1:0]` variables), the constraint cannot be satisfied
Scope	It only requires uniqueness **within one `randomize()` call**. It says nothing about values produced by earlier calls; that is what `randc` does
Cost	For N members it is equivalent to N × (N - 1) / 2 inequalities

Complete example

tb_unique.sv

```
module tb_unique;

  class uniq_demo;
    rand bit [2:0] perm[8];
    rand bit [3:0] a, b, c;

    constraint perm_c    { unique {perm}; }
    constraint scalars_c { unique {a, b, c}; a < 3; b < 3; c < 3; }
  endclass

  uniq_demo u;
  int       seen[8];
  bit       ok;

  initial begin
    u = new();
    repeat (3) begin
      if (!u.randomize()) $fatal(1, "randomize failed");

      seen = '{default: 0};
      foreach (u.perm[i]) seen[u.perm[i]]++;
      ok = 1;
      foreach (seen[v]) if (seen[v] != 1) ok = 0;

      $display("perm=%p  is a permutation of 0..7: %s", u.perm, ok ? "yes" : "NO");
      $display("a+b+c=%0d", u.a + u.b + u.c);
    end
  end

endmodule
```

What to expect. In each of the three iterations `perm` contains the values 0 to 7 in some order: eight different values of a 3-bit type must use every value exactly once, so the line always ends in `yes`. `a`, `b` and `c` are three different values below 3, which can only be 0, 1 and 2 in some order, so the second line is always exactly `a+b+c=3`.

Line by line

Line	Explanation
`unique {perm};`	Eight elements, eight possible values: the result is a random permutation. This is the standard way to generate a random ordering.
`unique {a, b, c}; a < 3; ...`	Three different values chosen from {0, 1, 2}. The sum is always 3, which gives a deterministic check.
`seen[...]++`	The checker counts each value and requires every count to be 1.

unique versus randc versus hand-written inequalities

Requirement	Use
Several variables or array elements differ **within one randomization**	`unique { ... }`
One variable does not repeat **across successive randomizations** until all values are used	`randc`
A tool or coding standard that predates `unique`	Nested `foreach` with `!=` (Problem 5 in Part 5)

Requirement	Use
A random permutation of 0 to N-1	`unique` on an array of N elements constrained to `[0:N-1]`, or `arr.shuffle()` in `post_randomize()`

Interview trap: "Does `unique {arr}` prevent a value from appearing again in the next `randomize()` call?" No. Uniqueness is only among the listed members in the current solution.

Common mistake: Asking for more unique values than exist: `rand bit [1:0] v[5]; constraint c { unique {v}; }` has no solution, because a 2-bit element has only four values.

Verification application: Choosing N different slave ports for parallel traffic, generating a set of non-colliding IDs for outstanding transactions, or building a random order in which to program registers.

Interview questions

Q1. What does the `unique` constraint do? It forces all members of its list (scalar variables, arrays, slices) to have different values in the solution.

Q2. How would you generate an array of unique values without the `unique` keyword? With nested `foreach` loops: `foreach (a[i]) foreach (a[j]) if (i != j) a[i] != a[j];`. Other options are to fill the array with distinct values and call `shuffle()` in `post_randomize()`, or to use a `randc` helper variable.

Q3. What is the difference between `unique` and `randc`? `unique` gives distinct values among several variables in one `randomize()` call. `randc` gives non-repeating values for one variable over successive calls.

Q4. How do you generate a random permutation of 0 to 15? `rand bit [3:0] p[16]; constraint c { unique {p}; }`. Sixteen distinct 4-bit values must be exactly the numbers 0 to 15.

Q5. What happens with `rand bit [1:0] v[5]; constraint c { unique {v}; }`? `randomize()` fails and returns 0, because five distinct values cannot be chosen from a set of four.

Part 3: Controlling the Distribution

The constraints of Part 2 decide which values are **legal**. This part is about how **often** each legal value appears (`dist`, `solve ... before`), and about constraints that give way when they conflict (`soft`).

3.1 Distribution Constraints: dist and Weighted Distribution

In plain English

By default all legal solutions are equally likely. Often that is not what a test needs. Short packets and maximum-length packets find more bugs than the thousands of lengths in between; an error should be injected in 5 percent of the transfers, not 50 percent. A `dist` constraint attaches **weights** to values so that some are chosen more often than others.

Technical definition

A **distribution constraint** has the form `expression dist { item, item, ... };`. Each item is a value or a range with an optional weight. It does two things:

1. **Membership:** the expression must be equal to one of the listed values or lie in one of the listed ranges (like `inside`).
2. **Weighting:** when nothing else interferes, the probability of each value is proportional to its weight.

Syntax

```
x dist { 0 := 40, [1:3] := 60 };        // := gives the weight to EACH value of the range
x dist { 0 :/ 40, [1:3] :/ 60 };        // :/ DIVIDES the weight among the values of the range
x dist { 0, [1:3] };                    // no weight given: default is := 1
x dist { 0 := w_zero, 255 := w_max, [1:254] :/ w_mid };   // weights may be variables
```

The two weight operators

Operator	Name	Meaning for a single value	Meaning for a range of N values
`:=`	"per value"	The value has this weight	**Each** value in the range has this weight (range total = N × weight)
`:/`	"per range"	The value has this weight	The range **as a whole** has this weight (each value gets weight / N)

For a single value the two operators are identical. They differ only for ranges.

Worked weights

```
x dist { 0 := 40, [1:3] := 60 };

   value    weight    probability
     0        40       40/220 = 18.2 %
     1        60       60/220 = 27.3 %
     2        60       60/220 = 27.3 %
     3        60       60/220 = 27.3 %
            -----
   total     220

x dist { 0 :/ 40, [1:3] :/ 60 };

   value    weight    probability
     0        40       40/100 = 40 %
     1        20       20/100 = 20 %       (60 divided among 3 values)
     2        20       20/100 = 20 %
     3        20       20/100 = 20 %
            -----
   total     100
```

Weights are **relative**. They do not have to add up to 100. `{0 := 1, 1 := 3}` and `{0 := 25, 1 := 75}` describe the same distribution.

Rules

Rule	Detail
Weights	Any integral expression, including variables. Must not be negative
Weight 0	The value is never chosen
Default weight	`:= 1`
Values not listed	Illegal: `dist` is also a membership constraint
Subject of the `dist`	Must involve at least one `rand` variable. `randc` variables are not allowed
Interaction with other constraints	Other constraints may remove some listed values. The remaining values keep their relative weights as far as possible, but the standard only guarantees the exact proportions "absent any other constraints"
Negation	A `dist` cannot be negated or used as a sub-expression of `->`, `||` and so on in the way `inside` can. It can be the consequence of an implication or the body of an `if`

Complete example

tb_dist.sv

```
module tb_dist;

  class per_value;
    rand bit [1:0] x;
    constraint c { x dist {0 := 40, [1:3] := 60}; }
  endclass

  class per_range;
    rand bit [1:0] x;
    constraint c { x dist {0 :/ 40, [1:3] :/ 60}; }
  endclass

  per_value pv;
  per_range pr;
  int       hist_v[4], hist_r[4];

  initial begin
    pv = new();
    pr = new();
    repeat (2200) begin
      if (!pv.randomize()) $fatal(1, "randomize failed");
      hist_v[pv.x]++;
    end
    repeat (1000) begin
      if (!pr.randomize()) $fatal(1, "randomize failed");
      hist_r[pr.x]++;
    end
    $display(":=  over 2200 calls : %p", hist_v);
    $display(":/  over 1000 calls : %p", hist_r);
  end

endmodule
```

What to expect. The first histogram is close to 400, 600, 600, 600 (weights 40, 60, 60, 60 out of 220, times 2200 calls). The second is close to 400, 200, 200, 200 (weights 40, 20, 20, 20 out of 100, times 1000 calls). The numbers fluctuate by a few percent from run to run. Sample output; actual values differ per seed and simulator:

```
:=  over 2200 calls : '{391, 607, 598, 604}
:/  over 1000 calls : '{412, 195, 188, 205}
```

Line by line

Line	Explanation
`x dist {0 := 40, [1:3] := 60};`	Each of 1, 2, 3 gets weight 60. The range is "three times as heavy" as its written weight.
`x dist {0 :/ 40, [1:3] :/ 60};`	The range gets 60 in total, 20 per value. Here the written numbers are directly the percentages for the two items.
`hist_v[pv.x]++;`	Builds a histogram. Measuring a distribution always needs many samples; a handful of calls proves nothing.

A realistic use: packet length and opcode mix

This example needs `alu_pkg.sv` from Module 1.

tb_dist_knobs.sv

```
module tb_dist_knobs;
  import alu_pkg::*;
```

```
  class alu_txn;
    rand bit [7:0] a, b;
    rand alu_op_e  op;

    // weights are ordinary variables: a test can change them at run time
    int unsigned w_arith = 6;
    int unsigned w_logic = 3;
    int unsigned w_shift = 1;

    constraint op_dist_c {
      op dist { ALU_ADD := w_arith, ALU_SUB := w_arith,
                ALU_AND := w_logic, ALU_OR  := w_logic, ALU_XOR := w_logic, ALU_NOT := w_logic,
                ALU_SHL := w_shift, ALU_SHR := w_shift };
    }
    constraint corner_c {
      a dist { 8'h00 := 10, 8'hFF := 10, [8'h01:8'hFE] :/ 80 };   // 10% / 10% / 80%
    }
  endclass

  alu_txn txn;
  int     n_shift, n_a_min, n_a_max;

  initial begin
    txn = new();
    repeat (1000) begin
      if (!txn.randomize()) $fatal(1, "randomize failed");
      if (txn.op inside {ALU_SHL, ALU_SHR}) n_shift++;
      if (txn.a == 8'h00) n_a_min++;
      if (txn.a == 8'hFF) n_a_max++;
    end
    $display("default weights : shifts=%0d a_min=%0d a_max=%0d", n_shift, n_a_min, n_a_max);

    txn.w_arith = 0;                  // a "shift only" test: remove the other opcodes
    txn.w_logic = 0;
    n_shift = 0;
    repeat (1000) begin
      if (!txn.randomize()) $fatal(1, "randomize failed");
      if (txn.op inside {ALU_SHL, ALU_SHR}) n_shift++;
    end
    $display("shift-only test : shifts=%0d", n_shift);
  end

endmodule
```

What to expect. With the default weights the total opcode weight is 6+6+3+3+3+3+1+1 = 26, of which the shifts have 2, so `shifts` is about 77 of 1000. `a_min` and `a_max` are each about 100. Without the `dist`, `a` would be `00` only about 4 times in 1000 calls; the `dist` is what makes the corner values common. In the second loop every other weight is 0, so the result is exactly `shifts=1000`.

Line	Explanation
`int unsigned w_arith = 6;`	A state variable used as a weight: a "knob".
`op dist {...}`	One weight per opcode. Equal weights within a group.
`[8'h01:8'hFE] :/ 80`	254 values share a weight of 80. With `:=` the range would have a total weight of 254 × 80 and the corners would almost never appear.
`txn.w_arith = 0;`	Zero weight removes those opcodes. If **all** weights were set to 0, no value would be legal and `randomize()` would fail.

Constraint expressions versus distribution constraints

Property	Constraint expression (`<`, `==`, `inside`, ...)	Distribution constraint (`dist`)
Purpose	Define what is **legal**	Define what is legal **and how often**
Probability	All solutions equally likely	Proportional to the weights
Negation `!( ... )`	Allowed	Not allowed
Usable inside larger expressions	Yes	No, only as a constraint item
On `randc` variables	Allowed	Not allowed
Bidirectional	Yes	The membership part is; the weights are a preference of the solver
Guarantee	Always satisfied (hard)	Membership always satisfied; proportions exact only without interfering constraints
Typical use	Protocol legality	Steering toward corners; mixing traffic types

dist versus inside

	`x inside {0, [1:3]}`	`x dist {0 := 1, [1:3] := 1}`	`x dist {0 :/ 1, [1:3] :/ 1}`
Legal values	0, 1, 2, 3	0, 1, 2, 3	0, 1, 2, 3
P(x = 0)	1/4	1/4	1/2
P(x = 1)	1/4	1/4	1/6

Interview trap: "`x dist {[1:100] := 1, 200 := 50}`: is 200 chosen 50 percent of the time?" No. The range has 100 values of weight 1 each, total 100. 200 has weight 50. P(200) = 50/150 = 33 percent. With `[1:100] :/ 1` the answer would be 50/51.

Interview trap: "What is the difference between `:=` and `:/`?" For a single value, none. For a range, `:=` gives the weight to each value, `:/` splits the weight across the values.

Common mistake: Combining a `dist` on a variable with another constraint that removes most of the listed values, and then expecting the written percentages. Once other constraints interfere, the observed ratio can differ from the written one.

Common mistake: Expecting exact counts. Weights are probabilities. In 100 calls a 10 percent item appears "about 10" times, sometimes 5, sometimes 16.

Verification application: Coverage closure. When the coverage report shows holes at the boundaries, the engineer adds or adjusts `dist` weights (or sets weight knobs from the test) to push stimulus toward the uncovered bins.

Interview questions

Q1. What is a `dist` constraint? A constraint that restricts an expression to a set of values and ranges and assigns each a weight, so that the values are chosen with probabilities proportional to the weights.

Q2. Explain `:=` and `:/` with an example. `x dist {[1:4] := 20, 5 := 20}`: each of 1 to 4 has weight 20 and 5 has weight 20, so 5 is chosen 20/100 = 20 percent of the time. `x dist {[1:4] :/ 20, 5 :/ 20}`: the range shares 20 (5 each) and 5 has 20, so 5 is chosen 20/40 = 50 percent of the time.

Q3. Do the weights have to add up to 100? No. They are relative. Only the ratios matter.

Q4. What happens to values that are not listed in the `dist`? They are never generated. A `dist` restricts the variable to its listed values.

Q5. Can a `dist` be applied to a `randc` variable? No. A weighted distribution conflicts with the cyclic "each value once" behaviour.

Q6. Can weights be variables? Yes. Weights can be any integral expressions, including class properties set by the test. A weight of 0 disables that value.

Q7. How do you generate a signal that is 1 about 5 percent of the time? `err dist {1 := 5, 0 := 95};`

3.2 Soft Constraints

In plain English

A normal constraint is a law: if it cannot be obeyed, randomization fails. A **soft** constraint is a default: "use this unless someone asks for something else". If a soft constraint conflicts with a normal constraint, the soft one is quietly dropped and randomization still succeeds.

Technical definition

A constraint expression preceded by the keyword `soft` is a **soft constraint**. All other constraints are **hard constraints**. The solver must satisfy every hard constraint. It satisfies each soft constraint **if doing so is possible** given the hard constraints and the soft constraints of higher priority; otherwise that soft constraint is discarded for this call. Discarding a soft constraint is not an error and produces no message.

Why it is needed

Before soft constraints existed, a transaction class with a default such as `len == 8` forced every test that wanted a different length to switch the block off with `constraint_mode(0)` or to override it in a derived class. With `soft len == 8`, a test simply writes `txn.randomize() with { len == 20; }`. The default disappears by itself. This makes base classes reusable: the class author supplies sensible defaults, and the test writer overrides them without knowing the names of the constraint blocks.

Syntax

```
class pkt;
  rand bit [7:0] len;
  rand bit [7:0] gap;

  constraint range_c { len inside {[1:64]}; }          // hard: the protocol limit
  constraint dflt_c  { soft len == 8; }                // soft: the default
  constraint gap_c   { soft gap inside {[1:3]}; }      // soft constraints can be any expression
endclass

p.randomize() with { soft len == 3; };                 // soft constraints can also be inline
p.randomize() with { disable soft len; };              // drop the soft constraints on len
```

Priority between soft constraints

When two soft constraints conflict with each other, the one with the **higher priority** wins. Priority follows a simple idea: **the more specific or later the declaration, the higher the priority.**

Priority	Where the soft constraint is written
Highest	Inline, in `randomize() with { ... }`
	In a derived class
	In the base class
Lowest	In the class of a `rand` member object (the container's constraints outrank the contained object's)

Within one class, a soft constraint written **later** (in a later block, or later in the same block) has higher priority than one written earlier.

`disable soft x;` discards all soft constraints of lower priority that refer to `x`. It is used when a test wants `x` fully random again without specifying a replacement value.

How it works internally

```
randomize()
   1. Collect hard constraints H and soft constraints S1 (highest priority) ... Sn (lowest)
   2. If H alone has no solution                -> return 0 (failure)
   3. For each soft constraint, highest priority first:
          if H + (softs kept so far) + this one still has a solution  -> keep it
          else                                                        -> drop it
   4. Pick a solution of H + the kept soft constraints
```

A soft constraint is **all or nothing**. It is not a probability or a weight. If it can be honoured it is always honoured in full.

Complete example

tb_soft.sv

```
module tb_soft;

  class pkt;
    rand bit [7:0] len;
    rand bit [7:0] gap;

    constraint range_c { len inside {[1:64]}; }             // hard
    constraint dflt_c  { soft len == 8; }                   // soft default
    constraint gap_c   { soft gap == 1; soft gap == 2; }    // two soft constraints in conflict
  endclass

  pkt p;
  int status;

  initial begin
    p = new();

    status = p.randomize();
    $display("1. no override          : status=%0d len=%0d gap=%0d", status, p.len, p.gap);

    status = p.randomize() with { len == 20; };
    $display("2. hard inline len==20  : status=%0d len=%0d", status, p.len);

    status = p.randomize() with { soft len == 3; };
    $display("3. soft inline len==3   : status=%0d len=%0d", status, p.len);

    status = p.randomize() with { len > 60; };
    $display("4. hard inline len>60   : status=%0d len in 61..64: %s",
             status, (p.len inside {[61:64]}) ? "yes" : "NO");

    status = p.randomize() with { len == 100; };
    $display("5. hard inline len==100 : status=%0d", status);
  end

endmodule
```

EXPECTED OUTPUT

```
1. no override          : status=1 len=8 gap=2
2. hard inline len==20  : status=1 len=20
3. soft inline len==3   : status=1 len=3
4. hard inline len>60   : status=1 len in 61..64: yes
5. hard inline len==100 : status=0
```

Most simulators print an additional constraint-failure message for call 5.

Line by line

Call	Explanation
1	Nothing conflicts with `soft len == 8`, so it is honoured: `len` is 8. For `gap`, the two soft constraints contradict each other. The later one (`gap == 2`) has higher priority and is kept; `gap == 1` is dropped.
2	The hard inline constraint `len == 20` conflicts with `soft len == 8`. The soft constraint is dropped. No failure.
3	Two soft constraints on `len`: the class default (8) and the inline one (3). Inline has the highest priority, so `len` is 3.
4	`len > 60` conflicts with the soft default, which is dropped. The hard range constraint still applies, so `len` is 61 to 64.
5	`len == 100` conflicts with the **hard** constraint `len inside {[1:64]}`. Hard against hard cannot be resolved: `randomize()` returns 0. Soft constraints never rescue a conflict between hard constraints.

Soft versus hard constraints

Property	Hard constraint	Soft constraint
Keyword	None (the default)	`soft`
Must be satisfied	Always	Only if possible
Conflict with a hard constraint	`randomize()` fails	The soft constraint is dropped silently
Conflict with another soft constraint	Not applicable	The higher-priority one wins
Priority	None: all are equal and all must hold	Inline > derived class > base class; later > earlier
Can be removed with	`constraint_mode(0)` on the block	`constraint_mode(0)` on the block, or `disable soft var;`
Allowed on	`rand` and `randc` variables	`rand` variables only
Typical use	Protocol rules that must never be broken	Default values and typical-case preferences that tests may override

Interview trap: "If a soft constraint conflicts with a hard constraint, does `randomize()` fail?" No. The soft constraint is ignored and the call succeeds.

Interview trap: "Is a soft constraint satisfied with some probability?" No. It is satisfied whenever it does not conflict; otherwise it is dropped entirely.

Common mistake: Making protocol rules soft. If `soft addr[1:0] == 0` is the alignment rule, any test constraint that happens to conflict with it silently produces misaligned addresses. Rules are hard; defaults are soft.

Common mistake: Expecting a soft constraint to be dropped only "as much as necessary". With `soft len inside {[1:10]}` and a hard `len > 8`, there is no conflict: `len` is 9 or 10. With a hard `len > 20`, the entire soft constraint is dropped and `len` can be anything above 20.

Verification application: A reusable transaction ships with soft defaults: `soft delay == 0;`, `soft kind == NORMAL;`, `soft err_inject == 0;`. A directed error test overrides one of them inline: `randomize() with { err_inject == 1; }`, and inherits the remaining defaults unchanged.

Interview questions

Q1. What is a soft constraint? A constraint marked `soft` that the solver satisfies when possible and silently discards when it conflicts with a hard constraint or with a higher-priority soft constraint.

Q2. Why were soft constraints introduced? To let a class provide default values and behaviours that tests can override with inline or derived constraints, without switching constraint blocks off and without randomization failures.

Q3. Two soft constraints conflict. Which one is used? The one with higher priority: inline `with` constraints over class constraints, derived-class over base-class, and within a class the later-declared over the earlier-declared.

Q4. What does `disable soft x;` do? It removes all lower-priority soft constraints that involve the variable `x`, so that `x` is restricted only by hard constraints and by soft constraints of higher priority.

Q5. `constraint c1 { soft x == 5; }` and inline `with { x inside {[10:20]}; }`. What is `x`? A value from 10 to 20. The soft constraint conflicts with the hard inline constraint and is dropped.

Q6. `constraint c1 { x == 5; }` and inline `with { soft x == 10; }`. What is `x`? 5. The inline constraint is soft and conflicts with the hard class constraint, so the inline one is dropped. Being inline gives priority only among soft constraints.

Q7. Can `randc` variables have soft constraints? No. Soft constraints may be specified only for `rand` variables.

3.3 solve...before: Variable Ordering

In plain English

Section 2.4 showed a surprise: with `(mode == 1) -> (len == 3)`, the one-bit variable `mode` was 1 only one time in five, because four of the five legal combinations have `mode` = 0. `solve mode before len` tells the solver: "choose `mode` first, giving each of its legal values an equal chance, and only then choose `len`". The set of legal combinations stays exactly the same. Only the probabilities change.

Technical definition

`solve a before b;` is a **variable-ordering** constraint. It instructs the solver to pick the value of `a` before the value of `b`. `a` is chosen with a uniform distribution over the values of `a` **that occur in at least one solution**; then `b` is chosen uniformly among the values that complete a solution with that `a`.

Two statements define `solve`...`before` completely:

1. It **does not change the solution space.** Every combination that was legal is still legal, and none that was illegal becomes legal.
2. It **does change the probability** of each solution.

Syntax

```
constraint order_c {
  solve mode before len;                 // one before one
  solve kind, burst before addr, len;    // several before several
}
```

Worked probability tables

The class used in all three cases:

```
rand bit       mode;
rand bit [1:0] len;
constraint imp_c { (mode == 1) -> (len == 3); }
```

The five solutions are (0,0), (0,1), (0,2), (0,3) and (1,3).

```
Case A: no ordering                each solution is equally likely

   (mode,len)    probability
     (0,0)          1/5
     (0,1)          1/5
     (0,2)          1/5
     (0,3)          1/5
     (1,3)          1/5           P(mode=1) = 1/5       P(len=3) = 2/5

Case B: solve mode before len      mode first: 0 or 1, each 1/2

   (mode,len)    probability
     (0,0)       1/2 x 1/4 = 1/8
     (0,1)       1/2 x 1/4 = 1/8
     (0,2)       1/2 x 1/4 = 1/8
     (0,3)       1/2 x 1/4 = 1/8
     (1,3)       1/2 x 1   = 1/2  P(mode=1) = 1/2       P(len=3) = 5/8

Case C: solve len before mode      len first: 0, 1, 2 or 3, each 1/4

   (mode,len)    probability
     (0,0)       1/4 x 1   = 1/4
     (0,1)       1/4 x 1   = 1/4
     (0,2)       1/4 x 1   = 1/4
     (0,3)       1/4 x 1/2 = 1/8
     (1,3)       1/4 x 1/2 = 1/8  P(mode=1) = 1/8       P(len=3) = 1/4
```

The same five solutions appear in all three tables. The ordering only moves probability between them.

Complete example

tb_solve_before.sv

```
module tb_solve_before;

  class no_order;
    rand bit       mode;
    rand bit [1:0] len;
    constraint imp_c { (mode == 1) -> (len == 3); }
  endclass

  class mode_first extends no_order;
    constraint order_c { solve mode before len; }
  endclass

  class len_first extends no_order;
    constraint order_c { solve len before mode; }
  endclass

  function automatic int count_mode_one(no_order h);
    int n = 0;
    repeat (1000) begin
      if (!h.randomize()) $fatal(1, "randomize failed");
      if (h.mode == 1) begin
        n++;
        if (h.len != 3) $error("illegal solution mode=1 len=%0d", h.len);
      end
    end
    return n;
  endfunction

  no_order   a;
  mode_first b;
  len_first  c;

  initial begin
    a = new();  b = new();  c = new();
    $display("no ordering           : mode=1 in %0d of 1000", count_mode_one(a));
    $display("solve mode before len : mode=1 in %0d of 1000", count_mode_one(b));
    $display("solve len before mode : mode=1 in %0d of 1000", count_mode_one(c));
  end

endmodule
```

What to expect. The three counts are close to 200, 500 and 125 (probabilities 1/5, 1/2 and 1/8). No `$error` message appears in any case, because the legal set is the same for all three classes. Exact counts vary per seed and simulator.

Line by line

Line	Explanation
`class mode_first extends no_order;`	Inherits the variables and the implication, and adds only the ordering.
`solve mode before len;`	`mode` is picked first. Both 0 and 1 are part of at least one solution, so each gets probability 1/2.
`count_mode_one(no_order h)`	The argument is a base-class handle. Because `randomize()` is virtual, the constraints of the actual object (including `order_c`) are used.
`if (h.len != 3) $error(...)`	Checks that the ordering has not created an illegal solution.

Restrictions and implicit orderings

Rule	Detail
Variables	Must be `rand` integral variables. `randc` variables are not allowed (they are always solved first anyway)
Circular ordering	`solve a before b; solve b before a;` is an error
Cannot fix a failure	If the constraints have no solution, no ordering will create one
Cannot cause a failure	The solver only picks values of the first variable that are part of a complete solution
Not needed when	The relation between the variables is one-to-one (for example `b == a + 1`), because then every value already has the same number of solutions

Besides explicit `solve` ... `before`, the language has **implicit** orderings:

Implicit ordering	Can it cause a failure?
`randc` variables are solved before `rand` variables	Yes: the cyclic value is fixed first and the rest must fit around it
The size of a dynamic array or queue is solved before its elements	Yes, if the chosen size makes the element constraints impossible
Arguments of a function called in a constraint are solved before the function's result is used (Section 3.4)	Yes

Key point: Explicit `solve` ... `before` is a pure probability control and is always safe. The three implicit orderings are real "decide this first" steps and can turn a solvable problem into a failure.

Interview trap: "Does `solve a before b` change which values are legal?" No. It changes only how often each legal combination is selected.

Interview trap: "My `randomize()` fails. Will `solve before` fix it?" No. If the solution space is empty, nothing is left to order.

Common mistake: Reading `solve a before b` as "apply the constraints on `a` first and ignore `b`". The solver still considers all constraints together when deciding which values of `a` are possible.

Verification application: The usual pattern is a "kind" or "type" variable that determines the range of a data variable: `solve kind before len;`. Without it, kinds with large length ranges dominate and kinds with a single legal length (for example a zero-length control packet) almost never occur. An alternative that needs no ordering is to put a `dist` on `kind`.

Interview questions

Q1. What does `solve` ... `before` do? It sets the order in which the solver assigns the variables, which changes the probability distribution of the solutions. It does not change the set of legal solutions.

Q2. `rand bit a; rand bit [1:0] b; constraint c { a -> b == 0; }` **Give P(a=1) without ordering, with** `solve a before b`**, and with** `solve b before a`**.** Solutions: (0,0), (0,1), (0,2), (0,3), (1,0). Without ordering: 1/5. With `solve a before b`: 1/2. With `solve b before a`: `b` is 0 with probability 1/4, and then `a` is 1 with probability 1/2, so 1/8.

Q3. Can `solve ... before` **be used with** `randc`**?** No. `randc` variables are not permitted in `solve ... before`; they are always solved first.

Q4. When is `solve ... before` **needed?** When one variable's value determines how many values another variable can take, typically through an implication or `if-else`, and the first variable should have a balanced (or at least non-negligible) distribution.

Q5. Is there a way to get the same effect without `solve ... before`**?** Yes: put an explicit `dist` on the controlling variable, or randomize in two steps (randomize the controlling variable first, then the rest with it as a state variable). The two-step approach, unlike `solve ... before`, is a real ordering and can fail if the first choice leaves no solution.

Q6. Does the order in which constraints are written have the same effect as `solve ... before`**?** No. Textual order has no effect at all.

3.4 Functions in Constraints

In plain English

Some relationships are awkward or impossible to write as a single expression: a lookup in a table, a checksum, a calculation with a loop. A constraint may call a **function** to compute such a value. There is a price: the solver cannot see inside the function, so it cannot solve "backwards" through it.

Rules

Rule	Detail
What can be called	Functions only (no tasks). Usually a class method or a package function
Lifetime	The function must be `automatic` (class methods are automatic by default)
Arguments	`input` or `const ref` only. No `output`, `inout` or non-const `ref`
Side effects	None. The function must not modify class properties, and must not call `rand_mode()` or `constraint_mode()`
Number of calls	Unspecified. The solver may call the function any number of times, in any order

How it works internally: an implicit ordering

When a random variable is used as an **argument** of a function in a constraint, the solver:

1. solves the argument variables **first**, using only the constraints that involve those variables and no lower-priority ones,
2. calls the function with the chosen values,
3. treats the returned value as a **constant** while solving the remaining variables.

```
   constraint c { y == twice(x);  y < 6; }

   Step 1:  x is solved on its own. No constraint mentions only x -> x is any value 0..15.
   Step 2:  twice(x) is evaluated, say x = 9 -> 18.
   Step 3:  solve y with  y == 18  and  y < 6   -> impossible -> randomize() returns 0.

   The solver does NOT go back and choose a smaller x.
```

The function call has broken the bidirectional link between `x` and `y`. Information flows only from the arguments to the result.

Complete example

tb_func_constraint.sv

```
module tb_func_constraint;

  class with_function;
    rand bit [3:0] x;
    rand bit [4:0] y;

    function automatic bit [4:0] twice(bit [3:0] v);
      return 2 * v;
    endfunction

    constraint c { y == twice(x); y < 6; }
  endclass

  class with_expression;
    rand bit [3:0] x;
    rand bit [4:0] y;
    constraint c { y == 2 * x; y < 6; }
  endclass

  with_function   f;
  with_expression e;
  int             f_fail, e_fail;

  initial begin
    f = new();
    e = new();
    repeat (1000) begin
      if (!f.randomize()) f_fail++;
      if (!e.randomize()) e_fail++;
    end
    $display("with function   : %0d failures in 1000 calls", f_fail);
    $display("with expression : %0d failures in 1000 calls", e_fail);
  end

endmodule
```

What to expect. The expression version never fails: the line reads exactly `with expression : 0 failures in 1000 calls`, because the solver restricts `x` to 0, 1 or 2 so that `y` is 0, 2 or 4. The function version fails whenever the independently chosen `x` is 3 or more, which is 13 of its 16 values, so it reports about 810 failures (the simulator will also print a failure message for each one unless such messages are switched off).

Line by line

Line	Explanation
`function automatic bit [4:0] twice(...)`	A side-effect-free function with an `input` argument.
`y == twice(x); y < 6;`	`x` is a function argument and is solved first, alone. `y < 6` cannot influence it.

Line	Explanation
`y == 2 * x; y < 6;`	The same relation as an expression. The solver sees the whole relation and restricts `x` accordingly.

When to use a function, and what to use instead

Situation	Recommendation
The relation can be written as an expression	Write the expression. It stays bidirectional
A derived value (CRC, parity, checksum, length field) that nothing else depends on	Compute it in `post_randomize()` (Section 4.1), not in a constraint
A value computed from **state** variables only	A function is fine: nothing random flows through it
A complex mapping from one random variable to a bound for another (for example "maximum length allowed for this kind")	A function is acceptable; remember that the argument is solved first and is not influenced by the other variable

Interview trap: "Are constraints with function calls bidirectional?" No. The arguments are solved first and the return value is then a constant. This is the main exception to the "all constraints are solved together" rule.

Common mistake: A function in a constraint that modifies a class property, prints, or counts calls. The solver may call it many times or not at all.

Interview questions

Q1. Can you call a function inside a constraint? Yes, if the function is automatic, has only input (or const ref) arguments and has no side effects.

Q2. How does the solver treat a function call in a constraint? It first solves the random variables used as arguments, then evaluates the function, and then uses the result as a constant when solving the remaining variables. This creates an implicit ordering and removes bidirectionality across the call.

Q3. Why can a constraint with a function fail where the equivalent expression would succeed? Because the argument variables are chosen without regard to the constraints on the function's result. If the chosen arguments produce a result that violates another constraint, the solver cannot go back.

Q4. Where should a CRC or checksum field be computed? In `post_randomize()`, after all random fields have their final values. It is a derived value, not a random one.

Part 4: Controlling Randomization

Parts 2 and 3 described constraints written once, inside the class. A test also needs to change the behaviour of a class **from outside**, call by call: switch a constraint off, freeze a variable, add one extra rule for one call, run code just before or just after the solver. This part covers those controls, then failure debugging and inheritance.

4.1 pre_randomize() and post_randomize()

In plain English

`randomize()` cannot be modified, but it calls two empty "hook" functions that you can fill in:

- `pre_randomize()` runs **just before** the solver starts. Use it to prepare: set knobs, allocate arrays, switch constraints.
- `post_randomize()` runs **just after** the solver has written the new values. Use it to finish: compute fields that depend on the random values, print, update history.

Technical definition

Every class has two built-in methods:

```
function void pre_randomize();
function void post_randomize();
```

They are called automatically by `randomize()`. By default they do nothing (apart from calling the same hook in the base class). A class may define its own versions.

Property	`pre_randomize()`	`post_randomize()`
Called	Before the solver runs	After the solver has assigned the values
Called when randomization fails	Yes (it has already run)	**No**
Sees	The old values of the random variables	The new values
Type	`function void`, no arguments	`function void`, no arguments
Can consume time	No (it is a function)	No
For `rand` member objects	Their `pre_randomize()` is also called	Their `post_randomize()` is also called
With `randomize(null)` or all `rand_mode` off	Still called	Called if the check succeeds

Execution flow

```
obj.randomize()
     |
     +--> obj.pre_randomize()         user code: prepare
     |
     +--> solver                      (failure: return 0 here, nothing else happens)
     |
     +--> values assigned to obj
     |
     +--> obj.post_randomize()        user code: derive, print, record
     |
     +--> return 1
```

Typical uses

In `pre_randomize()`	In `post_randomize()`
Set state variables (knobs, limits) that constraints refer to	Compute derived, non-random fields: CRC, parity, checksum, length field
Turn constraints or variables on and off with `constraint_mode()` / `rand_mode()`	Apply effects that are easier to code procedurally: sort or shuffle an array, corrupt a field for error injection
Construct the objects of a `rand` array of handles	Record the chosen value in a history queue (to avoid repeats next time)
Print or check the state before solving (debug)	Print the transaction; assign an ID; collect coverage

Complete example

tb_pre_post.sv

```
module tb_pre_post;

  class pkt;
    rand bit [3:0] len;
    rand bit [7:0] payload[];
    bit [7:0]      checksum;          // derived: not random
    int            max_len = 4;       // knob
    int            n_calls;

    constraint len_c { len inside {[1:max_len]}; payload.size() == len; }

    function void pre_randomize();
      n_calls++;
      $display("pre  : call %0d, max_len=%0d", n_calls, max_len);
    endfunction

    function void post_randomize();
      checksum = 0;
      foreach (payload[i]) checksum ^= payload[i];
      $display("post : len=%0d payload=%p checksum=%h", len, payload, checksum);
    endfunction
  endclass

  pkt p;
  int status;

  initial begin
    p = new();
    status = p.randomize();
    $display("status=%0d", status);

    p.max_len = 0;                    // makes len_c impossible: [1:0] is an empty range
    status = p.randomize();
    $display("status=%0d", status);
  end

endmodule
```

What to expect. The sequence of lines is fixed; only the random values vary. `len` is 1 to 4, the payload has exactly `len` elements, and `checksum` is the XOR of them. In the second call `pre` prints but `post` does not, because the randomization fails. A simulator failure message may appear before the final line. Sample output; the payload values differ per seed and simulator:

```
pre  : call 1, max_len=4
post : len=3 payload='{'h5c, 'ha1, 'h0f} checksum=f2
status=1
pre  : call 2, max_len=0
status=0
```

Line by line

Line	Explanation
`bit [7:0] checksum;`	Not `rand`. A checksum is a consequence of the payload, not an independent random choice.
`len inside {[1:max_len]};`	`max_len` is a state variable. Its value at the time of the call is used.
`function void pre_randomize();`	Redefines the hook. It runs on every `randomize()` call of this object, including failing ones.

Line	Explanation
`function void post_randomize();`	Runs only after a successful solve. All `rand` fields already hold their new values, so the checksum is computed from the final payload.
`p.max_len = 0;`	The range `[1:0]` is empty, so no legal `len` exists.
Second call	`pre` runs, the solver fails, `post` is skipped, `status` is 0.

Using the hooks with inheritance

```
class err_pkt extends pkt;
  rand bit corrupt;

  function void post_randomize();
    super.post_randomize();               // base-class hook first: computes the checksum
    if (corrupt) checksum = ~checksum;    // then inject the error
  endfunction
endclass
```

If a derived class defines `pre_randomize()` or `post_randomize()`, the base-class version is **not** called automatically any more. Call `super.pre_randomize()` / `super.post_randomize()` explicitly, otherwise the base class's preparation or derived-field calculation is silently lost.

> **Note:** The standard does not declare these two functions `virtual`. They nevertheless behave polymorphically, because they are called from inside `randomize()`, which is virtual: the hooks of the actual object's class are the ones that run.

Pre-randomization versus post-randomization

Question	`pre_randomize()`	`post_randomize()`
Can it influence what the solver generates?	Yes: by changing state variables and modes	No: the solver has finished
Can it change the generated values?	Its changes to `rand` variables are overwritten by the solver	Yes, and nothing checks the constraints afterwards
Is it reached on failure?	Yes	No
Main purpose	Set up the randomization	Complete the object

> **Interview trap:** "Is `post_randomize()` called when `randomize()` fails?" No. `pre_randomize()` has already been called by then.

> **Interview trap:** "If `post_randomize()` changes a `rand` variable to an illegal value, does `randomize()` fail?" No. Constraints are not re-checked after `post_randomize()`. This is exactly how error injection is done, and it is also a way to break your own constraints by accident.

> **Common mistake:** Declaring a derived field such as `crc` as `rand` and constraining it with a function. Compute it in `post_randomize()`.

> **Common mistake:** Putting delays or blocking calls in the hooks. They are functions and cannot consume simulation time.

> **Verification application:** In a packet class, `pre_randomize()` constructs the `rand` sub-objects (headers, extension blocks), and `post_randomize()` fills in `length`, computes the CRC over the packed bytes, and, if an `inject_crc_err` knob is set, flips a bit of the CRC.

Interview questions

Q1. What are `pre_randomize()` and `post_randomize()`? Built-in void functions that `randomize()` calls automatically before the solver runs and after it has successfully assigned the values. Users redefine them to add set-up and completion code.

Q2. Give one typical use of each. `pre_randomize()`: set a constraint knob or disable a constraint depending on the test mode; allocate objects for a `rand` array of handles. `post_randomize()`: compute a CRC or parity field from the randomized data.

Q3. Can they be tasks? No. They are functions with no arguments and no return value, so they cannot contain timing controls.

Q4. If a derived class overrides `post_randomize()`, is the base-class version still executed? Only if the derived version calls `super.post_randomize()`.

Q5. Where would you calculate a CRC field, and why not with a constraint? In `post_randomize()`. The CRC is fully determined by the other fields, it is expensive or impossible for a solver to work through, and nothing needs to be solved "backwards" from it.

4.2 Turning Constraints On and Off: constraint_mode()

In plain English

A transaction class contains the constraints for normal, legal traffic. An error-injection test needs to break one of those rules; a corner-case test needs to replace a default. `constraint_mode()` is a switch on each constraint block: off means the solver ignores the block.

Syntax

```
obj.block_name.constraint_mode(0);        // task form: switch one block OFF
obj.block_name.constraint_mode(1);        // switch it ON again
obj.constraint_mode(0);                   // switch ALL blocks of the object off
obj.constraint_mode(1);                   // switch ALL blocks on

int state = obj.block_name.constraint_mode();   // function form: returns 1 (on) or 0 (off)
```

Form	Kind	Meaning
`obj.c.constraint_mode(0 or 1)`	Task call	Set the mode of block `c` in this object
`obj.constraint_mode(0 or 1)`	Task call	Set the mode of every block in this object, including inherited ones

Form	Kind	Meaning
`obj.c.constraint_mode()`	Function call, returns `int`	Query the mode of block `c`. The function form needs a block name

Rules

Rule	Detail
Default	Every constraint block is on
Scope	Per **object**. Switching a block off in one object does not affect other objects of the class
`static constraint`	The mode is shared: one call changes it for all objects
Persistence	The mode stays until it is changed again. It is not reset by `randomize()`
Granularity	A whole block. One expression inside a block cannot be switched off, which is why constraints should be split into small named blocks
Unknown block name	Compile error
An "off" block	Is ignored completely: it is not checked either

Complete example

tb_constraint_mode.sv

```
module tb_constraint_mode;

  class pkt;
    rand bit [7:0] len;
    constraint small_c { len < 10; }
    constraint big_c   { len > 200; }
  endclass

  pkt p;
  int status;

  initial begin
    p = new();
    $display("1. initial modes : small_c=%0d big_c=%0d",
             p.small_c.constraint_mode(), p.big_c.constraint_mode());

    status = p.randomize();
    $display("2. both on       : status=%0d", status);

    p.big_c.constraint_mode(0);
    status = p.randomize();
    $display("3. big_c off     : status=%0d len<10: %s", status, (p.len < 10) ? "yes" : "NO");

    p.small_c.constraint_mode(0);
    p.big_c.constraint_mode(1);
    status = p.randomize();
    $display("4. small_c off   : status=%0d len>200: %s", status, (p.len > 200) ? "yes" : "NO");

    p.constraint_mode(0);
    status = p.randomize();
    $display("5. all off       : status=%0d modes: small_c=%0d big_c=%0d",
             status, p.small_c.constraint_mode(), p.big_c.constraint_mode());
  end

endmodule
```

EXPECTED OUTPUT

```
1. initial modes : small_c=1 big_c=1
2. both on       : status=0
3. big_c off     : status=1 len<10: yes
4. small_c off   : status=1 len>200: yes
5. all off       : status=1 modes: small_c=0 big_c=0
```

A simulator failure message accompanies line 2.

Line by line

Line	Explanation
1	The function form reports that both blocks start in the "on" state.
2	`len < 10` and `len > 200` contradict each other. With both active the call fails.
3	With `big_c` off, only `small_c` applies.
4	The modes are swapped: only `big_c` applies.
5	`p.constraint_mode(0)` switches every block off. `len` is now completely unconstrained (0 to 255).

> **Common mistake:** Forgetting to switch a constraint back on. The object keeps the mode. If the same object is reused for later transactions, they are all generated without the constraint.

> **Common mistake:** Putting all constraints of a class in one big block. Then nothing can be disabled individually.

> **Verification application:** An error test: `txn.legal_len_c.constraint_mode(0);` followed by `txn.randomize() with { len > MAX_LEN; };` generates over-length packets from the same transaction class that normal tests use.

Interview questions

Q1. How do you disable a constraint? `obj.constraint_name.constraint_mode(0);`. To disable all constraints of an object: `obj.constraint_mode(0);`.

Q2. What does `constraint_mode()` return when called as a function? 1 if the named constraint block is active, 0 if it is inactive.

Q3. Does disabling a constraint in one object affect other objects? No, unless the constraint is declared `static`.

Q4. Can you disable a single expression inside a constraint block? No. The unit is the block. Split the block, or guard the expression with a state variable: `enable_x -> (expression);`.

Q5. What are the alternatives to `constraint_mode()` for removing a default? Declare the default as a `soft` constraint and override it inline; override the block in a derived class; or guard it with a knob variable.

4.3 Turning Random Variables On and Off: rand_mode()

In plain English

`rand_mode()` is the matching switch for **variables**. Switching a `rand` variable off freezes it: `randomize()` no longer changes it, and it keeps whatever value it has. It is used to hold one field at a chosen value while the rest of the object is randomized around it.

Syntax

```
obj.var_name.rand_mode(0);        // task form: this variable is no longer random
obj.var_name.rand_mode(1);        // random again
obj.rand_mode(0);                 // all random variables of the object off
obj.rand_mode(1);                 // all on

int state = obj.var_name.rand_mode();   // function form: 1 (random) or 0 (not random)
```

Rules

Rule	Detail
Applies to	Properties declared `rand` or `randc` only. Using it on a non-random property is a compile error
Default	On
Scope and persistence	Per object, until changed again
A variable that is switched off	Becomes a **state variable**. It keeps its current value, and that value is still used in every constraint that mentions it
Arrays	The whole array can be switched (`obj.arr.rand_mode(0)`), and for unpacked arrays a single element as well
Object handles	Switching off a `rand` handle stops the randomization of the object it refers to

> **Key point:** Switching a variable off does **not** switch off the constraints on it. The frozen value must still satisfy them. If it does not, `randomize()` fails.

Complete example

tb_rand_mode.sv

```
module tb_rand_mode;

  class pkt;
    rand bit [7:0] addr;
    rand bit [7:0] data;
    constraint addr_c { addr < 16; }
  endclass

  pkt p;
  int status;

  initial begin
    p = new();
    $display("1. modes: addr=%0d data=%0d", p.addr.rand_mode(), p.data.rand_mode());

    p.addr.rand_mode(0);              // freeze addr
    p.addr = 8'h05;
    repeat (3) begin
      status = p.randomize();
      $display("2. frozen at 05   : status=%0d addr=%h", status, p.addr);
    end

    p.addr = 8'hF0;                   // a frozen value that violates addr_c
    status = p.randomize();
    $display("3. frozen at F0   : status=%0d addr=%h", status, p.addr);

    p.addr_c.constraint_mode(0);      // remove the violated constraint
    status = p.randomize();
    $display("4. constraint off : status=%0d addr=%h", status, p.addr);

    p.rand_mode(0);                   // nothing is random any more
    $display("5. modes: addr=%0d data=%0d", p.addr.rand_mode(), p.data.rand_mode());
  end

endmodule
```

EXPECTED OUTPUT

```
1. modes: addr=1 data=1
2. frozen at 05   : status=1 addr=05
2. frozen at 05   : status=1 addr=05
2. frozen at 05   : status=1 addr=05
3. frozen at F0   : status=0 addr=f0
4. constraint off : status=1 addr=f0
5. modes: addr=0 data=0
```

A simulator failure message accompanies line 3. `data` (not printed) receives a new random value in every successful call.

Line by line

Line	Explanation
`p.addr.rand_mode(0);`	`addr` is now a state variable.
`p.addr = 8'h05;`	A procedural assignment chooses the frozen value. 5 satisfies `addr < 16`.
Line 2	Three successful calls. `addr` never changes; only `data` is randomized.
`p.addr = 8'hF0;`	240 violates `addr < 16`. The solver may not change `addr`, so no solution exists.
Line 3	`status` is 0.
Line 4	With the constraint switched off, the frozen value is acceptable again.

Line	Explanation
Line 5	`p.rand_mode(0)` switched every random variable off.

rand_mode() versus constraint_mode()

Property	`rand_mode()`	`constraint_mode()`
Controls	Whether a **variable** is randomized	Whether a **constraint block** is applied
Applied to	A `rand` / `randc` property, or the whole object	A constraint block, or the whole object
Off means	The variable keeps its current value and is treated as a state variable	The block is ignored by the solver
Constraints on that variable	**Still enforced** against the frozen value	Not applicable
Can cause `randomize()` to fail	Yes, if the frozen value violates an active constraint	No. Removing constraints can only enlarge the solution space
Task form	`obj.v.rand_mode(0)`, `obj.rand_mode(0)`	`obj.c.constraint_mode(0)`, `obj.constraint_mode(0)`
Function form	`obj.v.rand_mode()` returns 0/1	`obj.c.constraint_mode()` returns 0/1
Typical use	Hold a field at a directed value	Remove a rule, for example for error injection

Interview trap: "I switched off `rand_mode` for `addr` and set it to an illegal value, but `randomize()` fails. Why?" Because constraints on a non-random variable are still checked. Either set a legal value or also switch off the constraint.

Interview trap: "What does `randomize()` do when all variables have `rand_mode(0)`?" It still runs: it checks that the current values satisfy all active constraints and returns 1 or 0. Nothing is changed. This is the same as `randomize(null)` (Section 4.4).

Common mistake: Calling `rand_mode()` on a property that is not declared `rand`. That is a compile error. To make a non-random property random for one call, pass it as an argument: `obj.randomize(prop)`.

Interview questions

Q1. What does `rand_mode(0)` do? It makes a `rand` or `randc` variable inactive: `randomize()` no longer assigns it, and it behaves as a state variable holding its current value.

Q2. What is the difference between `rand_mode()` and `constraint_mode()`? `rand_mode()` enables or disables the randomization of variables; `constraint_mode()` enables or disables constraint blocks. A variable with `rand_mode` off is still checked against the constraints.

Q3. How do you fix one field to a specific value and randomize the others? Either `obj.field.rand_mode(0); obj.field = value;` and then `obj.randomize()`, or more simply `obj.randomize() with { field == value; }`. The inline form is preferred because nothing has to be switched back afterwards.

Q4. Can `rand_mode()` be used on a non-`rand` variable? No. It is a compile error.

4.4 Inline Constraints: randomize() with, and Conditional Randomization

In plain English

Sometimes one particular call needs one extra rule: "this time the address must be `0x40`". Creating a derived class or a new constraint block for that is too heavy. `randomize() with { ... }` adds constraints **for this call only**.

Syntax

```
if (!txn.randomize() with { addr == 16'h0040; len inside {[1:4]}; })
  $fatal(1, "randomize failed");

txn.randomize() with { a > b; };                  // any constraint item is allowed
txn.randomize() with { soft len == 2; };          // soft inline constraint
txn.randomize() with { foreach (d[i]) d[i] != 0; };
txn.randomize(addr, len);                         // randomize only the listed variables
txn.randomize(addr) with { addr < 100; };         // both features together
txn.randomize(null);                              // randomize nothing: check only
```

Rules for inline constraints

Rule	Detail
Lifetime	Apply to that single call
Combination	They are **added** to the active class constraints. They do not replace them
Conflict with a hard class constraint	The call fails. To override a class constraint, disable it, make it soft, or override it in a derived class
Priority	A `soft` inline constraint outranks all `soft` class constraints
Content	Everything allowed in a constraint block: expressions, `inside`, `dist`, `->`, `if-else`, `foreach`, `unique`, `solve before`, `soft`, `disable soft`
Name lookup	Names inside the braces are looked up **first in the class of the object being randomized**, then in the scope containing the call

The name-lookup trap and local::

```
task automatic send(bit [7:0] addr);                      // a local variable named addr
  bus_txn t = new();
  void'(t.randomize() with { addr == addr; });            // WRONG
  void'(t.randomize() with { addr == local::addr; });     // RIGHT
endtask
```

In the first call both `addr` names are resolved in the class `bus_txn`. The constraint is `t.addr == t.addr`, which is always true and restricts nothing. `local::` forces the name to be resolved in the scope where `randomize()` is called, so `local::addr` is the task argument. The bug is silent: no error, no warning, just an address that is not the one requested.

Randomizing only some variables: randomize(args)

Call	Random variables in this call	Everything else
`obj.randomize()`	All `rand` / `randc` properties with `rand_mode` on	State
`obj.randomize(a, b)`	Exactly `a` and `b`, **even if they are not declared `rand`**	State, including the properties that are declared `rand`
`obj.randomize(null)`	None	State

- The arguments must be properties of the object.
- All active constraints still apply. Variables not listed are treated as constants and must already satisfy them.
- `randomize(null)` turns `randomize()` into a **checker**: it returns 1 if the current values of the object satisfy all active constraints and 0 otherwise, without changing anything.

Complete example

tb_inline.sv

```
module tb_inline;

  class bus_txn;
    rand bit [7:0] addr;
    rand bit [7:0] data;
    bit [7:0]      tag;                       // not declared rand
    constraint addr_c { addr inside {[0:99]}; }
  endclass

  bus_txn t;
  int     status;

  task automatic send(bit [7:0] addr);
    status = t.randomize() with { addr == local::addr; };
    $display("2. local::addr=%0d  : status=%0d t.addr=%0d", addr, status, t.addr);
  endtask

  initial begin
    t = new();

    status = t.randomize() with { addr == 42; data == 8'hFF; };
    $display("1. inline         : status=%0d addr=%0d data=%h", status, t.addr, t.data);

    send(17);

    status = t.randomize() with { addr == 150; };
    $display("3. conflict       : status=%0d t.addr=%0d", status, t.addr);

    t.data = 8'hAA;
    status = t.randomize(addr) with { addr == 5; };
    $display("4. randomize(addr): status=%0d addr=%0d data=%h", status, t.addr, t.data);

    status = t.randomize(tag) with { tag == 8'h3C; };
    $display("5. randomize(tag) : status=%0d tag=%h addr=%0d", status, t.tag, t.addr);

    status = t.randomize(null);
    $display("6. check, addr=5  : status=%0d", status);
    t.addr = 150;
    status = t.randomize(null);
    $display("7. check, addr=150: status=%0d addr=%0d", status, t.addr);
  end

endmodule
```

EXPECTED OUTPUT

```
1. inline         : status=1 addr=42 data=ff
2. local::addr=17  : status=1 t.addr=17
3. conflict       : status=0 t.addr=17
4. randomize(addr): status=1 addr=5 data=aa
5. randomize(tag) : status=1 tag=3c addr=5
6. check, addr=5  : status=1
7. check, addr=150: status=0 addr=150
```

Simulator failure messages accompany lines 3 and 7.

Line by line

Line	Explanation
1	Two inline constraints pin both fields. The class constraint `addr_c` also holds (42 is within 0 to 99).
2	`local::addr` is the task argument (17). Without `local::`, both names would mean `t.addr` and the address would be any value from 0 to 99.
3	`addr == 150` conflicts with the hard class constraint. The call fails and `t.addr` keeps its previous value, 17.
4	Only `addr` is randomized. `data` is a state variable in this call and keeps `AA`.
5	`tag` is not declared `rand`, but naming it as an argument makes it the random variable for this call. `addr` is state and stays 5.
6	`randomize(null)` only checks. `addr` = 5 satisfies `addr_c`.
7	`addr` = 150 violates `addr_c`: the check returns 0. Nothing is modified.

Conditional randomization: the complete toolbox

"Conditional randomization" means that what is generated depends on a condition: a mode, a test phase, a previous value. SystemVerilog offers several ways. Choosing among them is a common design question.

Technique	Where the condition lives	Example	Use when
Implication / `if-else` on a **random** variable	Inside the solve	`(kind == RD) -> (len == 0);`	The condition is itself part of the random choice
`if-else` on a **state** variable (knob)	Class property set by the test	`if (cfg_short) len < 8; else len < 64;`	A test or configuration object selects a mode
Weight knobs in a `dist`	Class property	`err dist {1 := err_pct, 0 := 100 - err_pct};`	The condition is "how often"
`constraint_mode()`	Call from the test or from `pre_randomize()`	`txn.legal_c.constraint_mode(0);`	A whole rule is switched for a test or a phase
`rand_mode()`	Call from the test	`txn.kind.rand_mode(0);`	A field must stay at a directed value
Inline `with`	At the call	`randomize() with { kind == WR; }`	A one-off requirement of a single call

Technique	Where the condition lives	Example	Use when
Procedural selection	Around the call	`if (first) txn.randomize() with {...}; else txn.randomize();`	The condition depends on testbench state or history
Derived class with overriding constraints	Class hierarchy	`class err_txn extends txn;`	A reusable variant used by many tests (Section 4.7)

```
class cfg_txn;
  rand bit [7:0] len;
  rand bit       err;
  bit            cfg_short;                // knob set by the test
  int unsigned   err_pct = 5;              // knob: percentage of errors

  constraint len_c { if (cfg_short) len inside {[1:8]}; else len inside {[1:64]}; }
  constraint err_c { err dist {1 := err_pct, 0 := 100 - err_pct}; }
endclass
```

Interview trap: "Do inline constraints override class constraints?" No, they are added. A hard inline constraint that contradicts a hard class constraint causes a failure.

Interview trap: "What does `obj.randomize() with { x == x; }` do when the calling scope also has a variable `x`?" Both names refer to `obj.x`, so the constraint is always true. `local::x` is needed to reach the outer variable.

Common mistake: Using `obj.randomize(null)` and expecting random values. With `null`, nothing is randomized.

Verification application: A directed-random test body in a generator: `repeat (20) begin txn = new(); if (!txn.randomize() with { op == ALU_ADD; a + b > 255; }) $fatal(1, "randomize failed"); gen2drv_mbx.put(txn); end`. It produces twenty additions that overflow, with everything else left random.

Interview questions

Q1. What is an inline constraint? A constraint supplied at the call with `randomize() with { ... }`. It applies to that call only and is solved together with the class's own active constraints.

Q2. What happens if an inline constraint conflicts with a class constraint? If both are hard, `randomize()` returns 0. If the class constraint is soft, it is dropped and the inline constraint wins.

Q3. What is `local::` used for? Inside a `with` block, names are resolved first in the randomized object's class. `local::name` forces resolution in the calling scope, which is needed when a local variable or argument has the same name as a class property.

Q4. What does `obj.randomize(a)` do? It randomizes only the property `a`, treating all other properties as state variables for this call. `a` need not be declared `rand`.

Q5. What does `obj.randomize(null)` do? It randomizes nothing and returns 1 if the object's current values satisfy all active constraints, otherwise 0. It is used as a checker.

Q6. How would you generate 10 transactions where the first is a reset command and the rest are random? Procedurally: in the loop, for the first iteration call `randomize() with { kind == RESET; }`, and for the others plain `randomize()`. Or keep a state variable `is_first` in the class and write `is_first -> kind == RESET;`.

Q7. How can a test change the percentage of error packets without modifying the transaction class? The class uses a non-random weight variable in a `dist`, for example `err dist {1 := err_pct, 0 := 100 - err_pct};`, and the test sets `err_pct`.

4.5 std::randomize(), System Random Functions and Seeding

std::randomize(): randomizing without a class

`obj.randomize()` needs a class with `rand` properties. For a quick random value in a module, a task or a testbench method, declaring a class is excessive. The **scope randomize function** `std::randomize()` randomizes ordinary variables that are visible in the current scope.

```
int       a, b;
bit [3:0] nib;
bit       ok;

ok = std::randomize(a);                                    // any 32-bit value
ok = std::randomize(a, b) with { a inside {[1:10]}; b == a * 2; };
ok = std::randomize(nib) with { nib > 12; };
```

Property	`obj.randomize()` (class method)	`std::randomize(vars)` (scope function)
What is randomized	The object's `rand` / `randc` properties	The variables listed as arguments
`rand` keyword needed	Yes	No. Any integral variable in scope can be listed
Constraints	Class constraint blocks plus optional `with`	Only the `with` block
`randc` behaviour	Supported	Not available
`pre_randomize()` / `post_randomize()`	Called	Not called
`constraint_mode()` / `rand_mode()`	Available	Not applicable
Reuse through inheritance	Yes	No
Return value	1 or 0	1 or 0
Typical use	Transactions and configuration objects	One-off random values in procedural code

Inside a class method, an unqualified `randomize()` means `this.randomize()`. To randomize a local variable of the method, `std::randomize(local_var)` must be written explicitly.

$urandom, $urandom_range and $random

Function	Returns	Notes
`$urandom()`	Unsigned 32-bit	`$urandom(seed)` also sets the seed of the calling thread's generator
`$urandom_range(max, min)`	Unsigned, in [min, max] inclusive	`min` defaults to 0. If `max` is less than `min` the two are swapped, so `$urandom_range(1, 5)` and `$urandom_range(5, 1)` are the same
`$random()`	**Signed** 32-bit	Legacy Verilog. `$random % 10` gives -9 to 9, not 0 to 9

```
delay = $urandom_range(10, 1);        // 1 to 10
idx   = $urandom % 8;                 // 0 to 7
bad   = $random % 8;                  // -7 to 7 : a classic bug when used as an index
ok    = {$random} % 8;                // 0 to 7 : concatenation makes the value unsigned
```

Seeds and random stability (awareness level)

Random numbers come from a pseudo-random generator (RNG). Three facts matter in practice.

1. Every thread and every object has its own RNG. A value drawn in one thread does not disturb the sequence seen by another thread or object. This property is called **random stability**. It keeps a test reproducible when unrelated parts of the testbench change how many random numbers they consume.

2. Seeds are handed down hierarchically.

```
   simulation seed (command line)
        |
        v
   each initial block / top-level thread gets its own RNG, seeded from the simulation seed
        |
        +--> when a thread executes  new()  : the new OBJECT's RNG is seeded with the
        |                                     next random value of the creating thread
        |
        +--> when a thread executes  fork   : each CHILD THREAD's RNG is seeded with the
                                              next random value of the parent thread
```

Because seeds are drawn in order of creation, **inserting a new `new()` or `fork` before existing ones shifts the seeds of everything created after it** in that thread. This is why a small, harmless-looking testbench edit can change the random stimulus of a whole test, and why a failure sometimes "disappears" after adding debug code that creates an object.

3. Seeds can be set by hand.

Mechanism	Effect
Simulator command-line option	Sets the simulation seed. The option name is tool-specific: for example `+ntb_random_seed=<n>` (VCS), `-sv_seed <n>` (Questa), `-svseed <n>` (Xcelium). Check your tool's documentation
`obj.srandom(seed)`	Seeds the RNG of one object
`process::self().srandom(seed)`	Seeds the RNG of the current thread
`obj.get_randstate()` / `obj.set_randstate(str)`	Saves and restores the RNG state of an object as a string

tb_seed.sv

```
module tb_seed;

  class item;
    rand bit [15:0] v;
  endclass

  item a, b;
  bit  same = 1;

  initial begin
    a = new();
    b = new();
    a.srandom(123);                  // give both objects the same seed
    b.srandom(123);
    repeat (5) begin
      void'(a.randomize());
      void'(b.randomize());
      if (a.v != b.v) same = 0;
    end
    $display("two objects with the same seed produced the same 5 values: %s",
             same ? "yes" : "NO");
  end

endmodule
```

EXPECTED OUTPUT

```
two objects with the same seed produced the same 5 values: yes
```

Two objects of the same class, seeded identically and randomized identically, produce identical sequences. The values themselves depend on the simulator; the equality does not.

> **Key point:** To reproduce a failing random test you need the same seed, the same source code, the same simulator and version, and the same command-line options. Regression systems therefore log the seed of every run.

> **Common mistake:** Using `$random % N` as an array index or a delay. The result can be negative.

> **Common mistake:** Running a regression of 100 jobs with the default seed. All 100 jobs then generate identical stimulus. Each job must receive a different (and logged) seed.

Interview questions

Q1. What is `std::randomize()`? The scope randomize function. It randomizes the variables passed as arguments, which need not be class members or declared `rand`, optionally under inline constraints in a `with` block. It returns 1 on success.

Q2. What is the difference between `randomize()` and `std::randomize()`? `obj.randomize()` is the class method: it randomizes the object's `rand` properties under the class constraints and calls `pre_randomize()` and `post_randomize()`. `std::randomize()` works on arbitrary variables in the current scope with inline constraints only.

Q3. What range does `$urandom_range(3, 7)` return? 3 to 7 inclusive. If the first argument is smaller than the second, they are swapped.

Q4. What is random stability? The property that each thread and each object has its own random number generator, so the random values in one are not affected by random calls in another. It keeps simulations repeatable when unrelated code changes.

Q5. A test fails with seed 12345. After you add a debug object in the environment constructor, it passes with the same seed. Why? The new object is created before others in the same thread and takes a seed from the thread's RNG. All objects and threads created afterwards receive different seeds, so the stimulus changed.

Q6. How do you set the seed of one object? `obj.srandom(seed);`

4.6 Randomization Failure and Constraint Debugging

What a failure is

A randomization **fails** when no combination of values for the active random variables satisfies all active hard constraints together with the current values of the state variables. The solution space is empty.

On failure	Behaviour
Return value	0
Random variables	All keep their previous values. Nothing is partially updated
`pre_randomize()`	Has already been executed
`post_randomize()`	Not executed
Simulation	**Continues.** The language itself raises no error
Message	Most simulators print a warning or error naming the conflicting constraints; the detail and the options that control it are tool-specific

Because the simulation continues with stale values, an unchecked failure typically shows up much later as "the same packet is sent twice" or "the test never reaches the corner case". Always check the return value.

Causes of failure

Cause	Example
Two hard constraints contradict each other	`len < 10;` and `len > 200;`
A base-class constraint conflicts with one added in a derived class	Base `len inside {[1:10]}`, derived `len > 50` under a different block name
An inline constraint conflicts with a class constraint	`randomize() with { addr == 150; }` while the class has `addr < 100`
A state variable has an unexpected value	`len <= max_len` with `max_len` never initialised (0)
`rand` was forgotten	The variable is a state variable that violates its own constraint
A variable was frozen with `rand_mode(0)` at an illegal value	Section 4.3
The requested value does not fit the variable's width	`rand bit [3:0] x; constraint c { x == 20; }`
Signed/unsigned or expression-width effects	`rand bit [3:0] a[10]; a.sum() == 50;`

Cause	Example
An implicit ordering picked a dead end	Array size chosen first; function argument chosen first; `randc` chosen first
`unique` or `randc` has run out of values	Five `unique` 2-bit variables
Index out of range in a constraint	`foreach (d[i]) d[i] < d[i+1];`
An empty set	`x inside {empty_queue};` or `[hi:lo]` with reversed bounds, or all `dist` weights zero
X or Z in a state variable used by a constraint	An uninitialised `logic` knob

A debugging procedure

1. **Confirm it is a failure.** Check the return value and read the simulator's message. It normally lists the constraints (file and line) that cannot be satisfied together and the values of the state variables involved. Tools have options to make this report more detailed.
2. **Print the state.** Display all non-random variables used in constraints, and the `rand_mode()` of suspect variables, just before the call (or in `pre_randomize()`).
3. **Bisect with `constraint_mode()`.** Switch constraint blocks off one at a time and call `randomize()` again. The block whose removal makes the call succeed is part of the conflict.
4. **Check current values with `randomize(null)`.** Assign the values you expected to be legal and call `obj.randomize(null)`. If it returns 0, those values are not legal, and step 3 tells you which block rejects them.
5. **Simplify.** Copy the class into a small standalone test (like the `tb_` modules in these notes) with only the suspect constraints.
6. **Review widths and signs.** Check every literal and every arithmetic expression for truncation, wrap-around and signedness.
7. **Review the inheritance chain and inline constraints.** List every block the object actually has, including those of all base classes, and every `with` clause at the failing call.

Complete example: locating a conflict

tb_debug_fail.sv

```
module tb_debug_fail;

  class bus_txn;
    rand bit [7:0] addr;
    rand bit [7:0] len;
    int unsigned   max_len = 16;

    constraint len_c   { len inside {[1:max_len]}; }
    constraint align_c { addr[1:0] == 2'b00; }           // word aligned
    constraint odd_c   { addr[0] == 1'b1; }              // added later by somebody else
  endclass

  bus_txn t;

  initial begin
    t = new();
    if (!t.randomize()) begin
      $display("randomize FAILED. State: max_len=%0d", t.max_len);

      t.len_c.constraint_mode(0);
      $display("  without len_c   : status=%0d", t.randomize());
      t.len_c.constraint_mode(1);

      t.align_c.constraint_mode(0);
      $display("  without align_c : status=%0d", t.randomize());
      t.align_c.constraint_mode(1);

      t.odd_c.constraint_mode(0);
      $display("  without odd_c   : status=%0d", t.randomize());
      t.odd_c.constraint_mode(1);

      t.addr = 8'h10;  t.len = 4;                         // values we believe are legal
      $display("  addr=10 len=4 legal? status=%0d", t.randomize(null));
    end
  end

endmodule
```

EXPECTED OUTPUT

```
randomize FAILED. State: max_len=16
  without len_c   : status=0
  without align_c : status=1
  without odd_c   : status=1
  addr=10 len=4 legal? status=0
```

Simulator failure messages appear between these lines for each failing call.

Reading the result

Observation	Conclusion
Removing `len_c` does not help	`len_c` is not involved
Removing `align_c` helps, and removing `odd_c` helps	These two blocks conflict with each other
`randomize(null)` with a value we considered legal returns 0	The object's constraints reject an aligned address, which points at `odd_c`
Inspecting the two blocks	`addr[1:0] == 0` requires bit 0 to be 0; `addr[0] == 1` requires it to be 1

Debugging a distribution problem

Not every constraint problem is a failure. "It randomizes, but I never see value X" is as common.

Symptom	Likely cause	Fix
A mode bit is almost always 0	Implication with a wide dependent variable (Section 2.4)	`solve ... before`, or a `dist` on the mode
A listed `dist` value never appears	It is excluded by another constraint, or its `:=` weight is tiny compared with a wide range	Check the other constraints; use `:/` for the range
Values are mostly huge or negative	Unbounded or signed variable	Bound both sides; use unsigned types
A soft default never takes effect	A hard constraint or a higher-priority soft constraint conflicts with it	Review priorities
The array is always empty	The size is not constrained	Constrain `size()`
`randc` values repeat	A new object is created each time	Keep the `randc` variable in a persistent object
The same values in every regression job	Same seed	Give each job its own seed

The tool for these problems is measurement: randomize a few thousand times in a small test and print a histogram, or sample the variable in a covergroup (Module 10).

> **Interview trap:** "What happens when randomization fails? Does the simulation stop?" No. `randomize()` returns 0, the variables keep their old values, and execution continues. Only your own check (`$fatal`, `$error`) makes the failure visible.

> **Common mistake:** Trying to fix a failure with `solve ... before`. Ordering never creates solutions.

Interview questions

Q1. When does `randomize()` fail? When the active hard constraints, together with the current values of the state variables, have no solution.

Q2. How do you debug a constraint failure? Check the return value and read the simulator's conflict report; print the state variables used in constraints; disable constraint blocks one by one with `constraint_mode(0)` to find the conflicting set; use `randomize(null)` to test whether specific values are legal; reduce the problem to a small standalone class; and check widths, signs, inheritance and inline constraints.

Q3. Name five causes of randomization failure. Contradictory constraints; a conflict between base-class and derived-class or inline constraints; a state variable with an unexpected value; a variable frozen with `rand_mode(0)` at an illegal value; a forgotten `rand`; a value that does not fit the variable width; an out-of-range index in a `foreach` constraint; an implicit ordering such as array size or a function argument.

Q4. The randomization succeeds but one value never occurs. How do you investigate? Check whether another constraint excludes it; check the solution-count effect of implications and whether `solve before` or `dist` is needed; check `dist` weights and the use of `:=` versus `:/`; then measure with a histogram over many calls.

Q5. Why is `if (!obj.randomize()) $fatal(...)` used in testbenches? Because a failure is otherwise silent: the object keeps its previous contents and the test continues with wrong stimulus.

4.7 Constraint Inheritance, Overriding and Extension

In plain English

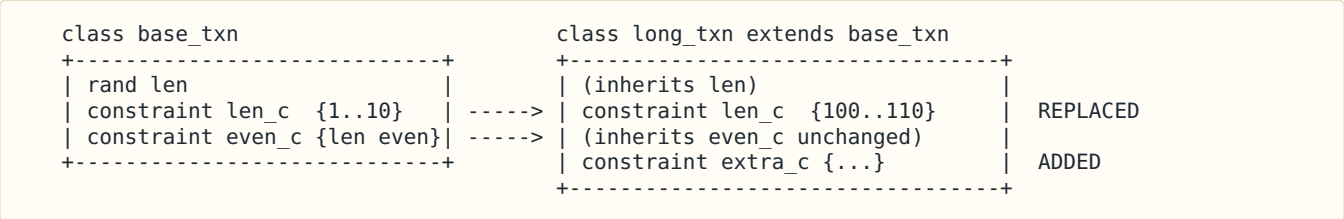
A derived class receives everything its base class has, including the random variables and the constraint blocks. The derived class can then do two things with constraints:

- **Add** a block with a **new name**. The new block applies **in addition to** all inherited ones.
- **Override** a block by declaring one with the **same name**. The new block **replaces** the inherited one.

This is how one transaction class becomes a family: `short_txn`, `long_txn`, `err_txn`, each a few lines long.

Rules

Rule	Detail
Inheritance	A derived class inherits all `rand` variables and all constraint blocks of its base classes
New block name	Adds a constraint. It must be consistent with the inherited constraints, otherwise randomization fails
Same block name	Overrides: the base-class block of that name is completely replaced in the derived class. There is no way to call the base version (no `super` for constraints)
Polymorphism	`randomize()` is virtual. Through a base-class handle that refers to a derived object, the **derived** object's constraints are used
Soft constraints	A soft constraint in a derived class has higher priority than a soft constraint in the base class
Modes	`constraint_mode()` and `rand_mode()` work on inherited blocks and variables exactly as on local ones
Abstract classes	A virtual class may declare `pure constraint name;`. Every non-abstract derived class must then provide a constraint block with that name



Complete example

tb_constraint_inherit.sv

```
module tb_constraint_inherit;

  class base_txn;
    rand bit [7:0] len;
    constraint len_c  { len inside {[1:10]}; }
    constraint even_c { len[0] == 1'b0; }
  endclass

  class long_txn extends base_txn;                  // OVERRIDE: same block name
    constraint len_c { len inside {[100:110]}; }
  endclass

  class tiny_txn extends base_txn;                  // EXTEND: new block name
    constraint tiny_c { len < 3; }
  endclass
```

```
  class bad_txn extends base_txn;                // EXTEND with a conflicting constraint
    constraint big_c { len > 50; }
  endclass

  base_txn b, h;
  long_txn l;
  tiny_txn t;
  bad_txn  x;
  int      status;

  initial begin
    b = new();  l = new();  t = new();  x = new();

    status = b.randomize();
    $display("1. base_txn : status=%0d even and in 1..10: %s", status,
             (b.len inside {2, 4, 6, 8, 10}) ? "yes" : "NO");

    status = l.randomize();
    $display("2. long_txn : status=%0d even and in 100..110: %s", status,
             (l.len inside {100, 102, 104, 106, 108, 110}) ? "yes" : "NO");

    status = t.randomize();
    $display("3. tiny_txn : status=%0d len=%0d", status, t.len);

    status = x.randomize();
    $display("4. bad_txn  : status=%0d", status);

    h = l;                                       // base handle, derived object
    status = h.randomize();
    $display("5. base handle to long_txn : status=%0d len>=100: %s", status,
             (h.len >= 100) ? "yes" : "NO");
  end

endmodule
```

EXPECTED OUTPUT

```
1. base_txn : status=1 even and in 1..10: yes
2. long_txn : status=1 even and in 100..110: yes
3. tiny_txn : status=1 len=2
4. bad_txn  : status=0
5. base handle to long_txn : status=1 len>=100: yes
```

A simulator failure message accompanies line 4.

Line by line

Line	Explanation
1	The base class: both of its blocks apply.
2	`long_txn` declares `len_c` again. The base `len_c` no longer exists for this class, so 100 to 110 does not conflict with 1 to 10. The inherited `even_c` still applies.
3	`tiny_txn` adds `tiny_c`. All three blocks apply: 1 to 10, even, less than 3. The only solution is 2, so this line is fully determined.
4	`bad_txn` adds `len > 50` under a new name. The inherited `len_c` (1 to 10) still applies. No solution: failure. The author probably meant to override and should have named the block `len_c`.
5	The handle type is `base_txn` but the object is a `long_txn`. `randomize()` is virtual, so the object's own constraints (100 to 110) are used.

Ways for a derived class or a test to change inherited behaviour

Goal	Technique
Tighten an inherited rule	Add a block with a new name
Replace an inherited rule	Declare a block with the same name
Remove an inherited rule	Override it with an empty block: `constraint len_c { }`, or call `constraint_mode(0)`
Replace a default without knowing its block name	The base class declares the default `soft`; the derived class or the inline constraint simply states the new requirement
Force derived classes to supply a rule	`pure constraint` in an abstract base class

> **Note:** The 2023 revision of the standard adds optional specifiers that let a class state explicitly whether a constraint block is new, extends or finalises an inherited one. Tool support is still arriving, and these are not expected in interviews; the name-based rules above are what you need.

Overriding versus extending

	Override (same name)	Extend (new name)
Base-class block	Replaced	Still active
Result	Only the derived block's rule applies	Both rules apply together
Risk	Accidentally discarding a rule the base class needed	Accidental contradiction with the base rule, causing failure
Use when	The derived class needs different values from the base	The derived class needs a subset of the base's values

> **Interview trap:** "A derived class adds `constraint c2 { x > 100; }` while the base has `constraint c1 { x < 50; }`. What happens?" Both apply and `randomize()` fails. Only a block named `c1` in the derived class would override the base constraint.

> **Interview trap:** "A base-class handle points to a derived-class object. Whose constraints are used?" The derived object's, because `randomize()` is virtual and constraints belong to the object, not to the handle type.

> **Common mistake:** Reusing a base-class block name by accident in a derived class. The base rule disappears without any warning.

> **Verification application:** Factory-style test writing. The generator holds a `base_txn` handle called the blueprint. A test replaces the blueprint with a derived object (`long_txn`, `err_txn`). The generator's code does not change, yet every `randomize()` now obeys the derived constraints. UVM formalises this as the factory override (Module 11).

Interview questions

Q1. Are constraints inherited? Yes. A derived class inherits all constraint blocks and random variables of its base class.

Q2. How do you override a constraint in a derived class? Declare a constraint block with the same name as the base-class block. It replaces the inherited one.

Q3. What happens if the derived class declares a constraint with a different name? It is added. The object must then satisfy both the inherited and the new constraints; if they contradict each other, randomization fails.

Q4. Can a derived class refer to the base-class constraint, like `super.c`? No. There is no `super` for constraints. To build on a base constraint, leave it inherited and add another block.

Q5. If a base handle refers to a derived object, which constraints does `randomize()` use? Those of the derived object, including overrides.

Q6. How do you remove a base-class constraint in a derived class? Override it with an empty block of the same name, or disable it with `constraint_mode(0)`, typically in the derived class's constructor.

Q7. What is a pure constraint? A constraint prototype declared with `pure constraint name;` in a virtual (abstract) class. It has no body; every concrete derived class must define a constraint of that name.

Part 5: Worked Constraint Problems

Constraint writing is tested at the whiteboard in almost every verification interview. This part contains twenty-two problems in rising order of difficulty. Each has a complete solution, the reasoning behind it, the traps, and alternative solutions where an interviewer is likely to ask for one.

How to use this part. Cover the solution, write your own, then compare. A constraint is "right" only when (1) every generated value is legal, (2) every legal value can be generated, and (3) it never fails. Requirement 2 is the one candidates forget: a constraint that always produces `1, 2, 3, 4` satisfies "ascending order" but is a poor answer.

About the checkers. Each solution contains procedural code that verifies the generated values independently of the constraint and counts violations in `errors`. Every program therefore ends with a line of the form `pNN: errors=0`, which does not depend on the seed. Writing such a checker is itself a skill interviewers look for: it shows that you can state the requirement in a second, independent way.

No.	Problem	Main technique
1	Address in two regions, aligned, with a hole	`inside`, bit-select, negation
2	Even numbers; multiples of 5 that are not multiples of 10	`%`, bit-select
3	Power of two	`$countones`, bit trick, shift
4	Exactly N bits set; N contiguous bits	`$countones`, shifted mask
5	Unique array elements without `unique`	Nested `foreach`
6	Array in ascending order	Neighbour constraint with guard
7	No two (or three) consecutive equal elements	Neighbour constraints
8	Elements that add up to a given total	`sum()` with a cast
9	Even values at even indices, odd at odd	Index used in the constraint
10	No repeated value without `randc`	History queue and `post_randomize()`
11	Fixed patterns: 0101..., 1 2 2 3 3 3...	Index arithmetic, helper function
12	Payload length depends on packet kind	Implication, `solve before`, array size
13	Exactly K elements equal to a value	Counting with `sum()`
14	Palindrome array	Mirrored index
15	Gray-code neighbours; fixed Hamming distance	`$countones` of an XOR
16	No two adjacent 1 bits	Shift and AND
17	A valid calendar date	Nested `if-else`
18	Non-overlapping memory blocks	Ordering between array elements, width control
19	Matrix with unique rows and columns	Two-dimensional `foreach`
20	ALU stimulus with a controlled overflow rate	Equivalence, `dist`, `solve before`
21	Eight queens	`unique`, signed differences
22	Push/pop sequence that never overflows a FIFO	Helper array for running state

Problem 1: Address in two regions, word-aligned, with a reserved hole

Problem. Generate a 12-bit address that lies in `0x100` to `0x1FF` or in `0x800` to `0x8FF`, is a multiple of 4, and is not in the reserved range `0x140` to `0x14F`.

tb_p01_addr_ranges.sv

```
module tb_p01_addr_ranges;

  class addr_gen;
    rand bit [11:0] addr;

    constraint region_c { addr inside {[12'h100:12'h1FF], [12'h800:12'h8FF]}; }
    constraint align_c  { addr[1:0] == 2'b00; }
    constraint hole_c   { !(addr inside {[12'h140:12'h14F]}); }
  endclass

  addr_gen g;
  int      errors;

  initial begin
    g = new();
    repeat (500) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      if (!((g.addr >= 12'h100 && g.addr <= 12'h1FF) ||
            (g.addr >= 12'h800 && g.addr <= 12'h8FF)))   errors++;
      if (g.addr % 4 != 0)                              errors++;
      if (g.addr >= 12'h140 && g.addr <= 12'h14F)        errors++;
    end
    $display("p01: errors=%0d", errors);
  end

endmodule
```

EXPECTED OUTPUT

```
p01: errors=0
```

Explanation. Three independent rules, three small blocks. `inside` lists the two regions. Alignment to 4 bytes means the two lowest bits are zero; a bit-select is cheaper for the solver than `addr % 4 == 0`. The hole is excluded with a negated `inside`, and the parentheses are required because `!` binds tighter than `inside`.

Why three blocks. An error test can now disable just `hole_c` to target the reserved range, or just `align_c` to generate misaligned accesses.

Problem 2: Even numbers, and multiples of 5 that are not multiples of 10

Problem. `x` must be even. `y` must be between 1 and 200, divisible by 5 and not divisible by 10.

tb_p02_divisible.sv

```
module tb_p02_divisible;

  class div_gen;
    rand bit [7:0] x, y;

    constraint even_c { x[0] == 1'b0; }
    constraint five_c { y inside {[1:200]}; y % 5 == 0; y % 10 != 0; }
  endclass

  div_gen g;
  int     errors;

  initial begin
    g = new();
    repeat (500) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      if (g.x % 2 != 0)                              errors++;
      if (g.y < 1 || g.y > 200)                      errors++;
      if (!(g.y % 5 == 0 && g.y % 10 != 0))          errors++;
    end
    $display("p02: errors=%0d", errors);
  end

endmodule
```

EXPECTED OUTPUT

```
p02: errors=0
```

Explanation. Even means bit 0 is zero; odd would be `x[0] == 1'b1`. For `y` the two modulus constraints together leave 5, 15, 25, ..., 195. An equivalent single constraint is `y % 10 == 5`.

Trap. With a signed variable (`rand int y`), `%` can be negative and `y % 5 == 0` also admits negative multiples. Bound the range, as done here.

Problem 3: Power of two

Problem. Generate a 16-bit value that is a power of two (1, 2, 4, ..., 32768). Show three ways.

tb_p03_power_of_two.sv

```
module tb_p03_power_of_two;

  class pow2_gen;
    rand bit [15:0] p1, p2, p3;
    rand bit [3:0]  sh;

    constraint count_c { $countones(p1) == 1; }                 // exactly one bit set
    constraint trick_c { p2 != 0; (p2 & (p2 - 1)) == 0; }       // classic bit trick
    constraint shift_c { p3 == (16'd1 << sh); }                 // 1 shifted by a random amount
  endclass

  pow2_gen g;
  int      errors;

  initial begin
    g = new();
    repeat (500) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      if (!$onehot(g.p1)) errors++;
      if (!$onehot(g.p2)) errors++;
      if (!$onehot(g.p3)) errors++;
    end
    $display("p03: errors=%0d", errors);
  end

endmodule
```

EXPECTED OUTPUT

```
p03: errors=0
```

Explanation.

Method	Why it works
`$countones(p1) == 1`	A power of two has exactly one bit set. `$onehot(p1)` is the same condition
`(p2 & (p2 - 1)) == 0`	Subtracting 1 clears the lowest set bit and sets all bits below it. ANDing with the original is zero only if there was a single set bit. Zero must be excluded separately
`p3 == (16'd1 << sh)`	Constructs the value from a random exponent 0 to 15

Trap. Writing `p2 & (p2 - 1) == 0` without the outer parentheses. `==` has higher precedence than `&`, so this is `p2 & ((p2 - 1) == 0)`, which is non-zero only for `p2` = 1.

Follow-up. "Not a power of two": `$countones(x) != 1`. "A power of two minus one (1, 3, 7, 15...)": `(x & (x + 1)) == 0; x != 0;`.

Problem 4: Exactly N bits set, and N contiguous bits set

Problem. (a) A 32-bit value with exactly five bits set, anywhere. (b) A 32-bit value with exactly five bits set, all next to each other.

tb_p04_n_bits.sv

```
module tb_p04_n_bits;

  class bits_gen;
    rand bit [31:0] any5;
    rand bit [31:0] run5;
    rand bit [4:0]  pos;

    constraint any_c { $countones(any5) == 5; }
    constraint run_c { pos <= 27; run5 == (32'h0000_001F << pos); }
  endclass

  bits_gen g;
  int      errors;

  initial begin
    g = new();
    repeat (500) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      if ($countones(g.any5) != 5)              errors++;
      if ($countones(g.run5) != 5)              errors++;
      if (((g.run5 >> g.pos) & 32'h1F) != 5'h1F) errors++;
    end
    $display("p04: errors=%0d", errors);
  end

endmodule
```

EXPECTED OUTPUT

```
p04: errors=0
```

Explanation. (a) is a direct use of `$countones`. For (b), a run of five ones is the mask `1F` shifted left by a random position. The position must not exceed 27, otherwise ones are shifted out of the top and fewer than five remain. Introducing a helper random variable (`pos`) is a standard technique: constrain something simple and derive the complicated value from it.

Follow-up. "Equal number of ones and zeros in a 16-bit value": `$countones(x) == 8`. "At most three bits differ from the previous value `prev`": `$countones(x ^ prev) <= 3` with `prev` a state variable updated in `post_randomize()`.

Problem 5: Unique array elements without the unique keyword

Problem. Fill an array of eight 4-bit elements with eight different values. You may not use `unique` or `randc`.

tb_p05_unique_manual.sv

```
module tb_p05_unique_manual;

  class uniq_gen;
    rand bit [3:0] a[8];

    constraint uniq_c {
      foreach (a[i])
        foreach (a[j])
          if (i < j) a[i] != a[j];
    }
  endclass

  uniq_gen g;
  int      errors;

  initial begin
    g = new();
    repeat (200) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      for (int i = 0; i < 8; i++)
        for (int j = i + 1; j < 8; j++)
          if (g.a[i] == g.a[j]) errors++;
    end
    $display("p05: sample a=%p", g.a);
    $display("p05: errors=%0d", errors);
  end

endmodule
```

What to expect. The first line shows eight different values from 0 to 15 (sample; values differ per seed). The second line is exactly `p05: errors=0`.

Explanation. Two nested `foreach` loops over the same array, with different loop variables, visit every pair of elements. The guard `i < j` keeps each pair once and, more importantly, skips `i == j`: without a guard the constraint would contain `a[0] != a[0]`, which is always false, and the randomization would fail. `if (i != j)` also works but generates each inequality twice.

Alternatives an interviewer may ask for.

```
// (1) The direct way, when allowed
constraint uniq_c { unique {a}; }

// (2) No constraint at all: fill with distinct values, then shuffle
function void post_randomize();
  foreach (a[i]) a[i] = i;     // any set of distinct values
  a.shuffle();
endfunction

// (3) Ascending order implies uniqueness; shuffle afterwards for a random order
constraint asc_c { foreach (a[i]) if (i > 0) a[i] > a[i-1]; }
function void post_randomize();
  a.shuffle();
endfunction
```

Solution 3 uses 7 constraints instead of 28, which matters for large arrays: the pairwise form grows with the square of the array size.

Problem 6: Array in ascending order

Problem. Generate six 8-bit elements in strictly ascending order.

tb_p06_ascending.sv

```
module tb_p06_ascending;

  class asc_gen;
    rand bit [7:0] a[6];

    constraint asc_c { foreach (a[i]) if (i > 0) a[i] > a[i-1]; }
  endclass

  asc_gen g;
  int     errors;

  initial begin
    g = new();
    repeat (200) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      for (int i = 1; i < 6; i++)
        if (g.a[i] <= g.a[i-1]) errors++;
    end
    $display("p06: sample a=%p", g.a);
    $display("p06: errors=%0d", errors);
  end

endmodule
```

What to expect. A sample of six increasing values, then exactly `p06: errors=0`.

Explanation. Each element is compared with its predecessor. The guard `if (i > 0)` removes the comparison for the first element, which has no predecessor. Use `>=` for non-decreasing order and `<` for descending order.

Alternative. Randomize without the constraint and call `a.sort()` in `post_randomize()`. Know the difference: sorting does not guarantee strictly ascending values (duplicates may remain), and it is a procedural step, so other constraints cannot refer to the sorted order.

Trap. `foreach (a[i]) a[i] < a[i+1];` indexes one past the end for the last element. Either guard with `if (i < $size(a) - 1)` or use the `a[i-1]` form.

Problem 7: No two consecutive elements equal; no three in a row

Problem. (a) An array of ten 2-bit values in which neighbouring elements are never equal. (b) An array of twelve single bits that never contains three equal bits in a row.

tb_p07_no_consecutive.sv

```
module tb_p07_no_consecutive;

  class seq_gen;
    rand bit [1:0] a[10];
    rand bit       b[12];

    constraint no_pair_c {
      foreach (a[i]) if (i > 0) a[i] != a[i-1];
    }
    constraint no_triple_c {
      foreach (b[i]) if (i > 1) !(b[i] == b[i-1] && b[i] == b[i-2]);
    }
  endclass

  seq_gen g;
  int     errors;

  initial begin
    g = new();
    repeat (200) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      for (int i = 1; i < 10; i++)
        if (g.a[i] == g.a[i-1]) errors++;
      for (int i = 2; i < 12; i++)
        if (g.b[i] == g.b[i-1] && g.b[i] == g.b[i-2]) errors++;
    end
    $display("p07: sample a=%p b=%p", g.a, g.b);
    $display("p07: errors=%0d", errors);
  end

endmodule
```

What to expect. A sample line, then exactly `p07: errors=0`.

Explanation. (a) compares each element with the previous one. (b) forbids the situation "this bit equals both of the two before it"; the guard is `i > 1` because two predecessors are needed. Note that (a) allows a value to reappear after a gap (`2, 1, 2` is legal), which distinguishes it from uniqueness.

Follow-up. "No value may repeat within any window of three elements": add `if (i > 1) a[i] != a[i-2];` to (a).

Problem 8: Elements that add up to a given total

Problem. Five elements, each between 1 and 40, whose sum is exactly 100.

tb_p08_sum.sv

```
module tb_p08_sum;

  class sum_gen;
    rand bit [7:0] a[5];

    constraint each_c { foreach (a[i]) a[i] inside {[1:40]}; }
    constraint sum_c  { a.sum() with (int'(item)) == 100; }
  endclass

  sum_gen g;
  int     errors, total;

  initial begin
    g = new();
    repeat (200) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      total = 0;
      foreach (g.a[i]) begin
        total += g.a[i];
        if (g.a[i] < 1 || g.a[i] > 40) errors++;
      end
      if (total != 100) errors++;
    end
    $display("p08: sample a=%p", g.a);
    $display("p08: errors=%0d", errors);
  end

endmodule
```

What to expect. A sample of five values between 1 and 40, then exactly `p08: errors=0`.

Explanation. `sum()` returns the element type. The `with (int'(item))` clause widens every element to 32 bits before the addition, so the total cannot wrap (Section 2.5). In this particular problem the largest possible sum is 200, which still fits in 8 bits, so the cast is not strictly needed. Write it anyway and say why: the habit prevents the bug when the limits change.

Follow-up. "Divide 100 into a random number of parts": make the array dynamic, constrain `a.size() inside {[2:10]}`, keep the other two constraints. Because the size is solved first, check that every allowed size has a solution: with elements of at most 40, size 2 cannot reach 100, so the size range must start at 3.

Problem 9: Even values at even indices, odd values at odd indices

Problem. In an array of eight elements, the element at an even index must be even and the element at an odd index must be odd.

tb_p09_even_odd_index.sv

```
module tb_p09_even_odd_index;

  class eo_gen;
    rand bit [7:0] a[8];

    constraint parity_c { foreach (a[i]) a[i] % 2 == i % 2; }
  endclass

  eo_gen g;
  int    errors;

  initial begin
    g = new();
    repeat (200) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      foreach (g.a[i])
        if ((g.a[i] % 2) != (i % 2)) errors++;
    end
    $display("p09: sample a=%p", g.a);
    $display("p09: errors=%0d", errors);
  end

endmodule
```

What to expect. A sample line, then exactly `p09: errors=0`.

Explanation. Inside a `foreach` constraint the loop variable is a constant for each unrolled copy, and it may be used freely in the expression. Here the parity of the element is tied to the parity of its index. An equivalent form with implications:

```
constraint parity_c {
  foreach (a[i]) {
    (i % 2 == 0) -> (a[i][0] == 1'b0);
    (i % 2 == 1) -> (a[i][0] == 1'b1);
  }
}
```

Follow-up. "Element value equals twice its index": `foreach (a[i]) a[i] == 2 * i;`. "Elements at indices divisible by 3 are zero, the others non-zero": `foreach (a[i]) (a[i] == 0) == (i % 3 == 0);`, an equivalence.

Problem 10: No repeated value without randc

Problem. A 4-bit variable must go through all 16 values without repetition, then start again, without using `randc`. Then modify the solution so that a value only has to differ from the last four values.

tb_p10_no_repeat.sv

```
module tb_p10_no_repeat;

  class no_repeat;
    rand bit [3:0] v;
    bit [3:0]      hist[$];                 // values already used in this cycle

    constraint fresh_c { !(v inside {hist}); }

    function void post_randomize();
      hist.push_back(v);
      if (hist.size() == 16) hist.delete(); // all values used: start a new cycle
    endfunction
  endclass

  no_repeat g;
  int       seen[16];
  int       errors;

  initial begin
    g = new();
    repeat (3) begin                        // three complete cycles
      seen = '{default: 0};
      repeat (16) begin
        if (!g.randomize()) $fatal(1, "randomize failed");
        seen[g.v]++;
      end
      foreach (seen[i]) if (seen[i] != 1) errors++;
    end
    $display("p10: errors=%0d", errors);
  end

endmodule
```

EXPECTED OUTPUT

```
p10: errors=0
```

Explanation. The object remembers what it has produced. `hist` is a state variable: at each call the solver sees its current contents as a constant set and must choose a value outside it. `post_randomize()` records the new value, and empties the history when all 16 values have been used. An `inside` test against an empty queue is false, so on the first call of each cycle nothing is excluded.

This reproduces the behaviour of `randc` with ordinary `rand`, and unlike `randc` it works for wide variables, because only the values actually used are stored.

Variant: differ from the last four values only.

```
function void post_randomize();
  hist.push_back(v);
  if (hist.size() > 4) void'(hist.pop_front());   // keep a sliding window of four
endfunction
```

Trap. Forgetting to clear the history. After 16 calls the excluded set contains every value and the 17th `randomize()` fails.

Verification application. "Do not reuse a transaction ID that is still outstanding": the scoreboard or driver keeps a queue of IDs in flight, and the generator constrains `!(id inside {busy_ids})`.

Problem 11: Fixed patterns

Problem. (a) Generate the bit pattern 0, 1, 0, 1, ... (b) Generate 1, 2, 2, 3, 3, 3, 4, 4, 4, 4, ... up to a given `n`.

tb_p11_patterns.sv

```
module tb_p11_patterns;

  class pat_gen;
    rand bit b[8];
    rand int a[];
    int      n = 4;

    // value of the pattern 1,2,2,3,3,3,... at position idx
    function automatic int val_at(int idx);
      int v   = 1;
      int end_of_group = 1;                  // number of elements up to and including group v
      while (idx >= end_of_group) begin
        v++;
        end_of_group += v;
      end
      return v;
    endfunction

    constraint alt_c  { foreach (b[i]) b[i] == i % 2; }
    constraint size_c { a.size() == n * (n + 1) / 2; }
    constraint tri_c  { foreach (a[i]) a[i] == val_at(i); }
  endclass

  pat_gen g;

  initial begin
    g = new();
    if (!g.randomize()) $fatal(1, "randomize failed");
    $write("p11: b =");
    foreach (g.b[i]) $write(" %0d", g.b[i]);
    $write("\np11: a =");
    foreach (g.a[i]) $write(" %0d", g.a[i]);
    $write("\n");
  end

endmodule
```

EXPECTED OUTPUT

```
p11: b = 0 1 0 1 0 1 0 1
p11: a = 1 2 2 3 3 3 4 4 4 4
```

Explanation. Both patterns are fully determined, so there is nothing random about the result; the question tests whether you can express a rule in terms of the index.

(a) The value is the parity of the index.

(b) The value `v` occupies `v` consecutive positions, so the total length for values 1 to `n` is `n(n+1)/2`. The helper function walks through the groups: group 1 ends before index 1, group 2 before index 3, group 3 before index 6, and so on. It is called with the loop index, which is a constant in each unrolled constraint, so the function's "arguments are solved first" behaviour (Section 3.4) causes no problem here.

A constraint-only alternative for (b). State the defining properties instead of computing the value: the sequence starts at 1, never decreases, rises by at most 1, and each value `k` occurs exactly `k` times.

```
constraint tri_c {
  a[0] == 1;
  foreach (a[i]) if (i > 0) a[i] inside {a[i-1], a[i-1] + 1};
  foreach (a[i]) a.sum() with (int'(item == a[i])) == a[i];
}
```

This is elegant but much more expensive to solve. In real code a fixed pattern is simply written in `post_randomize()` with two nested loops. Say so in the interview, then show the constraint to demonstrate that you can.

Follow-up patterns.

Pattern	Constraint
0, 0, 1, 1, 0, 0, 1, 1, ...	`a[i] == (i / 2) % 2`
0, 1, 2, 0, 1, 2, ...	`a[i] == i % 3`
1, 0, 2, 0, 3, 0, ...	`a[i] == ((i % 2 == 0) ? i / 2 + 1 : 0)`
7, 6, 5, ..., 0	`a[i] == $size(a) - 1 - i`

Problem 12: Payload length depends on the packet kind

Problem. A packet has a `kind` (SHORT, MEDIUM or LONG). SHORT packets have 1 to 15 payload bytes, MEDIUM 16 to 255, LONG 256 to 1500. The three kinds must occur about equally often, and the payload array must have exactly `len` bytes.

tb_p12_kind_len.sv

```
module tb_p12_kind_len;

  typedef enum {SHORT, MEDIUM, LONG} kind_e;

  class pkt;
    rand kind_e     kind;
    rand bit [10:0] len;
    rand bit [7:0]  payload[];

    constraint len_c {
      (kind == SHORT)  -> len inside {[1:15]};
      (kind == MEDIUM) -> len inside {[16:255]};
      (kind == LONG)   -> len inside {[256:1500]};
    }
    constraint order_c { solve kind before len; }
    constraint size_c  { payload.size() == len; }
  endclass

  pkt p;
  int errors;
  int count[kind_e];

  initial begin
    p = new();
    repeat (300) begin
      if (!p.randomize()) $fatal(1, "randomize failed");
      count[p.kind]++;
      case (p.kind)
        SHORT  : if (!(p.len inside {[1:15]}))     errors++;
        MEDIUM : if (!(p.len inside {[16:255]}))   errors++;
        LONG   : if (!(p.len inside {[256:1500]})) errors++;
      endcase
      if (p.payload.size() != p.len) errors++;
    end
    $display("p12: SHORT=%0d MEDIUM=%0d LONG=%0d", count[SHORT], count[MEDIUM], count[LONG]);
    $display("p12: errors=%0d", errors);
  end

endmodule
```

What to expect. The first line shows three counts each close to 100. The second line is exactly `p12: errors=0`.

Explanation. Three implications describe the table in the specification. Since `kind` is an enum with exactly these three members, the three cases cover everything.

Without `order_c` there are 15 + 240 + 1245 = 1500 solutions, and SHORT packets would be only 1 percent of the traffic. `solve kind before len` makes the solver pick the kind first with probability 1/3 each. This is the standard use of `solve` ... `before`.

`payload.size() == len` ties the array size to the length field, so `len` effectively decides the size.

Alternative to the ordering. `constraint kind_c { kind dist {SHORT := 1, MEDIUM := 1, LONG := 1}; }`. With a `dist` the test can also change the mix by using weight variables.

Follow-up. "Within LONG packets, make maximum-size packets common": add `(kind == LONG) -> len dist {1500 := 30, [256:1499] :/ 70};`.

Problem 13: Exactly K elements equal to a given value

Problem. In an array of ten 4-bit elements, exactly three must be equal to 5.

tb_p13_count_value.sv

```
module tb_p13_count_value;

  class cnt_gen;
    rand bit [3:0] a[10];

    constraint three_fives_c { a.sum() with (int'(item == 5)) == 3; }
  endclass

  cnt_gen g;
  int     errors, fives;

  initial begin
    g = new();
    repeat (200) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      fives = 0;
      foreach (g.a[i]) if (g.a[i] == 5) fives++;
      if (fives != 3) errors++;
    end
    $display("p13: sample a=%p", g.a);
    $display("p13: errors=%0d", errors);
  end

endmodule
```

What to expect. A sample containing exactly three 5s, then exactly `p13: errors=0`.

Explanation. `item == 5` is 1 for a matching element and 0 otherwise. Summing these gives the number of matches. The cast `int'( ... )` is essential: the comparison result is a single bit, and a sum of 1-bit values is computed in 1 bit, which would only give the parity of the count. Without the cast, `== 3` could never be true (a 1-bit value is 0 or 1) and the randomization would fail.

Follow-ups. "At least two elements greater than 10": `a.sum() with (int'(item > 10)) >= 2`. "No more than half the elements are zero": `a.sum() with (int'(item == 0)) <= $size(a) / 2`.

Problem 14: Palindrome array

Problem. Generate an array of 5 to 9 decimal digits that reads the same forwards and backwards.

tb_p14_palindrome.sv

```
module tb_p14_palindrome;

  class pal_gen;
    rand bit [3:0] a[];

    constraint size_c   { a.size() inside {[5:9]}; }
    constraint digit_c  { foreach (a[i]) a[i] < 10; }
    constraint mirror_c { foreach (a[i]) a[i] == a[a.size() - 1 - i]; }
  endclass

  pal_gen g;
  int     errors, n;

  initial begin
    g = new();
    repeat (200) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      n = g.a.size();
      if (n < 5 || n > 9) errors++;
      for (int i = 0; i < n; i++)
        if (g.a[i] != g.a[n - 1 - i] || g.a[i] > 9) errors++;
    end
    $display("p14: sample a=%p", g.a);
    $display("p14: errors=%0d", errors);
  end

endmodule
```

What to expect. A sample such as `'{'h3, 'h7, 'h0, 'h7, 'h3}`, then exactly `p14: errors=0`.

Explanation. Element `i` must equal its mirror image, element `size - 1 - i`. The size is solved before the elements, so `a.size()` is a known constant when the `foreach` is unrolled. Each pair is constrained twice (once from each end) and the middle element of an odd-length array is compared with itself; both are harmless. To halve the work, guard with `if (i < a.size() / 2)`.

Problem 15: Gray-code neighbours and fixed Hamming distance

Problem. (a) Generate eight different 4-bit values such that each differs from the previous one in exactly one bit. (b) Generate two 8-bit values that differ in exactly two bit positions.

tb_p15_gray.sv

```
module tb_p15_gray;

  class gray_gen;
    rand bit [3:0] g[8];
    rand bit [7:0] x, y;

    constraint step_c { foreach (g[i]) if (i > 0) $countones(g[i] ^ g[i-1]) == 1; }
    constraint uniq_c { unique {g}; }
    constraint dist_c { $countones(x ^ y) == 2; }
  endclass

  gray_gen h;
  int      errors;

  initial begin
    h = new();
    repeat (200) begin
      if (!h.randomize()) $fatal(1, "randomize failed");
      for (int i = 1; i < 8; i++)
        if ($countones(h.g[i] ^ h.g[i-1]) != 1) errors++;
      for (int i = 0; i < 8; i++)
        for (int j = i + 1; j < 8; j++)
          if (h.g[i] == h.g[j]) errors++;
      if ($countones(h.x ^ h.y) != 2) errors++;
    end
    $display("p15: sample g=%p", h.g);
    $display("p15: errors=%0d", errors);
  end

endmodule
```

What to expect. A sample sequence in which neighbours differ in one bit, then exactly `p15: errors=0`.

Explanation. XOR marks the bit positions in which two values differ; `$countones` counts them. The number of differing bits is called the Hamming distance. Requiring a distance of 1 between neighbours is the defining property of a Gray-code sequence. The `unique` constraint prevents the sequence from stepping back to a value it has already visited.

Verification application. Checking a clock-domain-crossing FIFO pointer, which must be Gray-coded: a monitor uses exactly this expression, `$countones(ptr ^ prev_ptr) <= 1`, as a check.

Problem 16: No two adjacent 1 bits

Problem. Generate a 16-bit value with exactly four bits set, no two of which are next to each other.

tb_p16_no_adjacent_ones.sv

```
module tb_p16_no_adjacent_ones;

  class bits_gen;
    rand bit [15:0] x;

    constraint spread_c { (x & (x >> 1)) == 0; }
    constraint four_c   { $countones(x) == 4; }
  endclass

  bits_gen g;
  int      errors;

  initial begin
    g = new();
    repeat (300) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      for (int i = 0; i < 15; i++)
        if (g.x[i] && g.x[i+1]) errors++;
      if ($countones(g.x) != 4) errors++;
    end
    $display("p16: sample x=%b", g.x);
    $display("p16: errors=%0d", errors);
  end

endmodule
```

What to expect. A sample such as `0100101000010001`, then exactly `p16: errors=0`.

Explanation. Shifting `x` right by one places each bit on top of its lower neighbour. If the AND of the original and the shifted value is zero, no bit and its neighbour are both 1. The equivalent per-bit form is `foreach (x[i]) if (i > 0) !(x[i] && x[i-1]);`, which is clearer but produces 15 constraints instead of one.

Follow-up. "At least two zeros between any two ones": `(x & (x >> 1)) == 0; (x & (x >> 2)) == 0;`.

Problem 17: A valid calendar date

Problem. Generate a valid date between the years 2000 and 2099.

tb_p17_date.sv

```
module tb_p17_date;

  class date_gen;
    rand bit [11:0] year;
    rand bit [3:0]  month;
    rand bit [4:0]  day;

    constraint year_c  { year inside {[2000:2099]}; }
    constraint month_c { month inside {[1:12]}; }
    constraint day_c {
      day >= 1;
      if (month inside {4, 6, 9, 11})
        day <= 30;
      else if (month == 2) {
        if (year % 4 == 0) day <= 29;
        else               day <= 28;
      }
      else
        day <= 31;
    }
  endclass

  function automatic int days_in(int y, int m);
    if (m inside {4, 6, 9, 11}) return 30;
    if (m == 2)                 return (y % 4 == 0) ? 29 : 28;
    return 31;
  endfunction

  date_gen g;
  int      errors;

  initial begin
    g = new();
    repeat (500) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      if (g.year < 2000 || g.year > 2099)                    errors++;
      if (g.month < 1 || g.month > 12)                       errors++;
      if (g.day < 1 || g.day > days_in(g.year, g.month))     errors++;
    end
    $display("p17: sample %0d-%0d-%0d", g.year, g.month, g.day);
    $display("p17: errors=%0d", errors);
  end

endmodule
```

What to expect. A sample date, then exactly `p17: errors=0`.

Explanation. A nested `if`-`else` constraint written like the rule everyone knows. Note the braces around the inner `if`-`else`: they make clear which `if` each `else` belongs to. Between 2000 and 2099 every year divisible by 4 is a leap year (2000 is divisible by 400), so the simple test is correct for this range; for a wider range the full rule `(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)` is needed.

Distribution note. With no ordering, every valid date is equally likely, which is the natural result (February is slightly rarer than January because it has fewer days). `solve month before day` would instead make every month equally likely.

Problem 18: Non-overlapping memory blocks

Problem. Allocate four memory blocks inside a 1024-byte region. Each block is 16 to 128 bytes long, starts on a 16-byte boundary, lies completely inside the region, and no two blocks overlap.

tb_p18_mem_blocks.sv

```
module tb_p18_mem_blocks;

  class alloc_gen;
    rand bit [9:0] base[4];        // start address of each block
    rand bit [7:0] blen[4];        // length of each block

    constraint len_c   { foreach (blen[i]) blen[i] inside {[16:128]}; }
    constraint align_c { foreach (base[i]) base[i][3:0] == 4'h0; }
    constraint fit_c   { foreach (base[i]) int'(base[i]) + int'(blen[i]) <= 1024; }
    constraint order_c {
      foreach (base[i])
        if (i > 0) int'(base[i]) >= int'(base[i-1]) + int'(blen[i-1]);
    }
  endclass

  alloc_gen g;
  int       errors;

  initial begin
    g = new();
    repeat (200) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      for (int i = 0; i < 4; i++) begin
        if (g.blen[i] < 16 || g.blen[i] > 128) errors++;
        if (g.base[i] % 16 != 0)               errors++;
        if (g.base[i] + g.blen[i] > 1024)      errors++;
        for (int j = i + 1; j < 4; j++)        // overlap test for every pair
          if (g.base[i] < g.base[j] + g.blen[j] && g.base[j] < g.base[i] + g.blen[i])
            errors++;
      end
    end
    $display("p18: sample base=%p blen=%p", g.base, g.blen);
    $display("p18: errors=%0d", errors);
  end

endmodule
```

What to expect. A sample allocation with increasing base addresses, then exactly `p18: errors=0`.

Explanation. Requiring "no two blocks overlap" for every pair needs a pairwise constraint. A simpler and cheaper approach is to **order** the blocks: block `i` must start at or after the end of block `i - 1`. Ordered blocks cannot overlap. If the blocks must appear in random order, shuffle the two arrays consistently in `post_randomize()`.

The width trap in this problem. Without the casts, the ordering constraint would be `base[i] >= base[i-1] + blen[i-1]`. The widest operand is 10 bits, so the addition is done in 10 bits. A block that ends exactly at 1024 (for example base 1008, length 16) gives 1024, which wraps to 0 in 10 bits. The constraint then reads `base[i] >= 0`, which is always true, and the next block can be placed anywhere, including on top of earlier blocks. The casts to `int` make the arithmetic 32 bits wide. This is a realistic example of a constraint that passes most runs and fails rarely.

Problem 19: A matrix with unique rows and unique columns

Problem. Fill a 4 × 4 matrix with the values 1 to 4 so that no value repeats in any row or in any column (a Latin square).

tb_p19_latin_square.sv

```
module tb_p19_latin_square;

  class latin_gen;
    rand bit [2:0] m[4][4];

    constraint range_c { foreach (m[i, j]) m[i][j] inside {[1:4]}; }
    constraint uniq_c {
      foreach (m[i, j])
        foreach (m[r, c]) {
          (r == i && c > j) -> m[i][j] != m[r][c];     // same row, later column
          (c == j && r > i) -> m[i][j] != m[r][c];     // same column, later row
        }
    }
  endclass

  latin_gen g;
  int       errors;

  initial begin
    g = new();
    repeat (50) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      for (int i = 0; i < 4; i++)
        for (int j = 0; j < 4; j++) begin
          if (g.m[i][j] < 1 || g.m[i][j] > 4) errors++;
          for (int k = j + 1; k < 4; k++) if (g.m[i][j] == g.m[i][k]) errors++;
          for (int k = i + 1; k < 4; k++) if (g.m[i][j] == g.m[k][j]) errors++;
        end
    end
    foreach (g.m[i]) $display("p19: sample row %0d = %p", i, g.m[i]);
    $display("p19: errors=%0d", errors);
  end

endmodule
```

What to expect. Four sample rows, each a permutation of 1 to 4, with no repeats down any column. Then exactly `p19: errors=0`.

Explanation. The outer `foreach` picks an element `m[i][j]`; the inner `foreach` picks a second element `m[r][c]`. Two implications select the pairs that matter: same row with a later column, and same column with a later row. Since all four loop variables are constants after unrolling, the guards are resolved at that time and only the relevant inequalities remain (24 for rows and 24 for columns).

Simpler form for the rows. Where the tool accepts array slices in `unique`, `foreach (m[i]) unique {m[i]};` handles the rows. Columns still need the explicit form, because a column is not a slice of the array.

Follow-ups. "All sixteen elements different": `unique {m};` with a wide enough element type. "Zero on the main diagonal": `foreach (m[i, j]) (i == j) -> m[i][j] == 0;`. "Symmetric matrix": `foreach (m[i, j]) m[i][j] == m[j][i];`.

Problem 20: ALU stimulus with a controlled overflow rate

Problem. For the course ALU, generate only additions and subtractions. About 80 percent of the transactions must produce a carry (for `ALU_ADD`, `a + b > 255`) or a borrow (for `ALU_SUB`, `a < b`). This example needs `alu_pkg.sv` from Module 1.

tb_p20_alu_overflow.sv

```
module tb_p20_alu_overflow;
  import alu_pkg::*;

  class alu_txn;
    rand bit [7:0] a, b;
    rand alu_op_e  op;
    rand bit       ovf;                    // helper: does this transaction overflow?

    constraint op_c       { op inside {ALU_ADD, ALU_SUB}; }
    constraint ovf_dist_c { ovf dist {1 := 80, 0 := 20}; }
    constraint ovf_c {
      if (op == ALU_ADD) ovf == (a + b > 255);
      else               ovf == (a < b);
    }
    constraint order_c    { solve op, ovf before a, b; }
  endclass

  alu_txn txn;
  int     errors, n_ovf;
  bit     really;

  initial begin
    txn = new();
    repeat (1000) begin
      if (!txn.randomize()) $fatal(1, "randomize failed");
      if (!(txn.op inside {ALU_ADD, ALU_SUB})) errors++;
      really = (txn.op == ALU_ADD) ? (int'(txn.a) + int'(txn.b) > 255) : (txn.a < txn.b);
      if (really != txn.ovf) errors++;
      if (really) n_ovf++;
    end
    $display("p20: carry/borrow in %0d of 1000", n_ovf);
    $display("p20: errors=%0d", errors);
  end

endmodule
```

What to expect. The first line reports a count near 800; the exact ratio can deviate somewhat between simulators because the `dist` variable also takes part in other constraints. The second line is exactly `p20: errors=0`.

Explanation.

- `ovf` is a **helper random variable**. It is not driven to the DUT. It gives the "interesting property" of the transaction a name, so that the property can be weighted with a `dist` and sampled in coverage.
- `ovf == (a + b > 255)` is an **equivalence**: `ovf` is 1 exactly when the sum overflows. An implication would not be enough, because it would allow an overflow with `ovf` = 0.
- In `a + b > 255` the literal 255 is 32 bits wide, so the sum is computed in 32 bits and does not wrap.
- `solve op, ovf before a, b` makes sure the operation and the overflow decision are taken first, and the operands are then chosen to fit.

Verification application. This is how coverage holes are closed in practice: identify the uncovered condition, give it a helper variable, weight it.

Problem 21: Eight queens

Problem. Place eight queens on a chessboard so that none attacks another.

tb_p21_queens.sv

```
module tb_p21_queens;

  class queens_gen;
    rand bit [2:0] col[8];                  // col[r] = column of the queen in row r

    constraint cols_c { unique {col}; }     // no two queens in the same column
    constraint diag_c {
      foreach (col[i])
        foreach (col[j])
          if (i < j) {
            int'(col[i]) - int'(col[j]) != i - j;     // not on the same diagonal
            int'(col[i]) - int'(col[j]) != j - i;     // not on the same anti-diagonal
          }
    }
  endclass

  queens_gen g;
  int        errors, dc;

  initial begin
    g = new();
    if (!g.randomize()) $fatal(1, "randomize failed");
    for (int i = 0; i < 8; i++)
      for (int j = i + 1; j < 8; j++) begin
        dc = int'(g.col[i]) - int'(g.col[j]);
        if (dc == 0 || dc == i - j || dc == j - i) errors++;
      end
    foreach (g.col[r]) begin
      for (int c = 0; c < 8; c++) $write("%s", (g.col[r] == c) ? " Q" : " .");
      $write("\n");
    end
    $display("p21: errors=%0d", errors);
  end

endmodule
```

What to expect. An 8 × 8 board with one `Q` in every row and every column and no two on a common diagonal (one of the 92 solutions; which one depends on the seed), then exactly `p21: errors=0`.

Explanation. The data structure does half the work. Using one array element per row, holding that row's column, makes "one queen per row" true by construction. `unique {col}` gives one queen per column. Two queens in rows `i` and `j` are on a common diagonal when the column difference equals the row difference, in either direction.

The sign trap. `col[i] - col[j]` with 3-bit unsigned elements would be computed as an unsigned value, and a "negative" difference would wrap to a large positive number that never equals `i - j`. Casting each element to `int` first makes the subtraction signed. Whenever a constraint needs a difference that can be negative, cast to a signed type.

Problem 22: A push/pop sequence that never overflows or underflows a FIFO

Problem. Generate a sequence of 20 operations, each a push or a pop, for a FIFO of depth 4 that starts empty. The sequence must never pop from an empty FIFO and never push into a full one.

tb_p22_fifo_ops.sv

```
module tb_p22_fifo_ops;

  class fifo_seq;
    localparam int N     = 20;
    localparam int DEPTH = 4;

    rand bit push[N];                       // 1 = push, 0 = pop
    rand int level[N + 1];                  // helper: fill level before each operation

    constraint start_c { level[0] == 0; }
    constraint step_c  {
      foreach (push[i]) level[i + 1] == level[i] + (push[i] ? 1 : -1);
    }
    constraint bound_c { foreach (level[i]) level[i] inside {[0:DEPTH]}; }
  endclass

  fifo_seq g;
  int      errors, fill;

  initial begin
    g = new();
    repeat (100) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      fill = 0;
      foreach (g.push[i]) begin
        if (g.push[i])  begin if (fill == 4) errors++; fill++; end
        else            begin if (fill == 0) errors++; fill--; end
      end
    end
    $write("p22: sample ops =");
    foreach (g.push[i]) $write(" %s", g.push[i] ? "PU" : "po");
    $write("\n");
    $display("p22: errors=%0d", errors);
  end

endmodule
```

What to expect. A sample sequence that always begins with a push (the FIFO starts empty), then exactly `p22: errors=0`.

Explanation. Whether operation `i` is legal depends on everything that happened before it, that is, on a **running state**. Constraints have no loops with memory, so the state is made explicit as a helper array: `level[i]` is the fill level before operation `i`. One constraint links consecutive levels through the operation, and one keeps every level within 0 to 4. The solver then chooses all twenty operations **together**, so every legal sequence can be generated, including ones that fill the FIFO completely and drain it again.

Compare this with the procedural alternative: choose each operation in a loop, looking at the current level. That works, but it is a greedy, one-step-at-a-time choice. With the constraint formulation you can add global requirements that a loop cannot guarantee, for example "the FIFO must be full at least twice" (`level.sum() with (int'(item == DEPTH)) >= 2`) or "the sequence ends with an empty FIFO" (`level[N] == 0`).

Verification application. This is the pattern behind scenario generation for any stateful protocol: credits, outstanding transaction counts, buffer occupancy. Model the state as a helper array, constrain the transitions, bound the state.

What the problems have in common

Technique	Problems
Translate the requirement into a property of **index and neighbours**	6, 7, 9, 11, 14
Use **bit-level** operators instead of arithmetic	1, 2, 3, 4, 15, 16
Count with `sum() with (int'(...))`	8, 13
Introduce a **helper random variable**	3, 4, 20, 22
Keep **history in a state variable**, update it in `post_randomize()`	10
Control the **distribution** with `solve before` or `dist`	12, 20
Choose the **data structure** so that part of the requirement holds by construction	18, 21
Guard against **width and sign** effects	8, 13, 18, 21
Always write an independent **checker**	all

Part 6: Module 5 Revision and Practice

6.1 Revision Summary

The mechanism in one page

Item	Summary
`rand`	Random; uniform over the legal values; values may repeat
`randc`	Random-cyclic; no repeat until all legal values are used; history is per object; solved first; integral types only
`randomize()`	Built-in virtual method; returns 1 on success, 0 on failure; on failure nothing changes
State variable	Any non-random variable in a constraint; its current value is a constant for the solver
Constraint block	Named class member; a set of expressions that must all be true
Solver	Considers all active constraints together; picks one solution; all solutions equally likely unless `dist` or `solve before` says otherwise
`pre_randomize()`	Runs before the solver, also when the call later fails
`post_randomize()`	Runs after a successful solve only; constraints are not re-checked afterwards

Constraint constructs

Construct	Meaning	Remember
`a < b;`	Relational	Bidirectional. No chained comparisons (`1 < x < 5` is wrong)
`a + b == 10;`	Arithmetic	Normal width and sign rules apply: overflow, unsigned subtraction
`x inside {[1:5], 9};`	Set membership	Uniform over the values, not over the items
`A -> B;`	Implication	Same as `!A || B`. Also means "not B implies not A"
`if (A) B; else C;`	Conditional	Same as `(A -> B) && (!A -> C)`. Braces, not `begin` ... `end`
`A <-> B;` or `A == B;`	Equivalence	Both directions
`foreach (a[i]) ...`	Per-element	Unrolled after the size is known. Guard neighbour indices
`a.sum() with (int'(item))`	Reduction	The sum has the element's width unless cast
`unique {a, b, arr};`	All different	Within one call only
`x dist {v := w, [r] :/ w};`	Weighted distribution	`:=` weight per value; `:/` weight shared by the range
`soft x == 5;`	Default	Dropped silently on conflict; inline > derived > base; later > earlier
`solve a before b;`	Ordering	Changes probabilities only, never the legal set
`f(x)` in a constraint	Function call	Arguments are solved first; not bidirectional

Controls

Control	Syntax	Effect
Disable a constraint	`obj.c.constraint_mode(0);`	The block is ignored
Freeze a variable	`obj.v.rand_mode(0);`	The variable becomes state; constraints on it still apply
Inline constraint	`obj.randomize() with { ... };`	Added to the class constraints for this call
Outer-scope name	`local::name`	Resolves a name in the calling scope inside `with`
Partial randomization	`obj.randomize(a, b);`	Only `a` and `b` are random in this call
Check only	`obj.randomize(null);`	Returns 1 if the current values are legal
Scope randomization	`std::randomize(v) with { ... };`	Randomizes ordinary variables
Seed an object	`obj.srandom(seed);`	Sets that object's random generator

The six comparisons

Compare	In one sentence
`rand` vs `randc`	`rand` picks independently each time and can repeat; `randc` cycles through every legal value once before repeating
Constraint expression vs `dist`	An expression says what is legal, with equal probability; `dist` says what is legal and how often
Implication vs `if-else`	Same logic and both bidirectional; `if-else` has a built-in "otherwise" branch, `->` is a compact expression
Soft vs hard	A hard constraint must hold or the call fails; a soft constraint is a default that is dropped when it conflicts
`rand_mode()` vs `constraint_mode()`	`rand_mode` switches variables, and the frozen value must still be legal; `constraint_mode` switches rules and can never cause a failure
`pre_randomize()` vs `post_randomize()`	`pre` prepares the solve and always runs; `post` completes the object and runs only on success

Implicit orderings (the only places where the solver "decides first")

1. `randc` variables before `rand` variables.
2. The size of a dynamic array or queue before its elements.
3. The arguments of a function called in a constraint before the function's result.

All three can make a solvable problem fail. Explicit `solve...before` cannot.

The traps, in one list

- `rand` forgotten: the variable is a constant.
- Return value of `randomize()` not checked.
- `assert(obj.randomize())` with assertions disabled.
- A new object per iteration destroys `randc`.
- Dynamic array without a size constraint stays empty; without an upper bound it may become huge.
- `1 < x < 5`; `a - b > 0` on unsigned values; `x < 10` on a signed variable.

- `a + b == 8'd10` wraps; `arr.sum()` wraps; `sum() with (item == 5)` is one bit wide.
- `!x inside {...}` needs parentheses; `x & (x - 1) == 0` needs parentheses.
- `d[i+1]` or `d[i-1]` without a guard.
- `x inside {[1:100], 200}` does not give 200 half the probability.
- A soft constraint is not a probability; inline constraints do not override hard class constraints.
- `rand_mode(0)` does not remove the constraints on the variable.
- `with { addr == addr; }` constrains nothing.
- A derived-class constraint with a new name is added, not substituted.
- `post_randomize()` is skipped on failure and can break constraints unnoticed.

6.2 Interview Questions and Answers

The questions at the end of each section cover the direct definitions. The questions below are additional.

Basic level

Q1. What are the three things needed for class-based randomization? Properties declared `rand` or `randc`, constraint blocks that restrict them, and a call to `randomize()` on a constructed object.

Q2. Where can `rand` be used? On class properties. Ordinary module variables and local variables cannot be declared `rand`; they are randomized with `std::randomize()`.

Q3. What is the return type of `randomize()`? `int`: 1 for success, 0 for failure.

Q4. Write a constraint that keeps an 8-bit value between 10 and 50, excluding 25. `constraint c { x inside {[10:50]}; x != 25; }`

Q5. How do you randomize a variable without declaring a class? `std::randomize(var) with { ... };` or, for a simple range, `var = $urandom_range(max, min);`.

Q6. What is the default distribution of a `rand` variable? Uniform over its legal values when no other variable is involved. In general, all solutions (combinations of all variables) are equally likely.

Q7. What is the difference between `$urandom_range(10)` and `$urandom % 10`? The first returns 0 to 10 inclusive; the second returns 0 to 9.

Q8. Can constraints be written outside the class? Yes, as an external constraint: declare `constraint c;` in the class and define `constraint cls::c { ... }` outside.

Q9. Can a constraint block be empty? Yes. It restricts nothing. An empty block is also used in a derived class to cancel an inherited block of the same name.

Q10. What happens to non-`rand` properties when `randomize()` is called? They are not changed. If they appear in constraints their current values are used as constants.

Intermediate level

Q11. How many solutions does `rand bit [2:0] a, b; constraint c { a + b == 3'd7; }` have, and how many does `a + b == 7` have? With the 3-bit literal the sum is computed in 3 bits: for each of the 8 values of `a` exactly one `b` works, so 8 solutions. With the 32-bit literal `7` the sum is computed in 32 bits: also 8 solutions

(0+7, 1+6, ..., 7+0). They coincide for 7. For a target of 3 they differ: `== 3'd3` has 8 solutions (including 4+7 = 11, which is 3 in 3 bits), while `== 3` has only 4.

Q12. Why is a `rand bit` field preferred to a `rand logic` field? The solver produces two-state values in both cases. `bit` is faster, uses less memory, and cannot be accidentally left at X when used as a state variable in a constraint.

Q13. Can you constrain a non-random variable? You can write the constraint, and it will be **checked**: the call fails if the variable's value violates it. But the solver cannot change the variable.

Q14. How do you make sure a `dist` is exact? You cannot guarantee exact proportions over a finite number of calls; weights are probabilities. They are also only exact when no other constraint interferes with the variable. Measure with coverage.

Q15. How do you randomize a variable to any value except those in a queue? `constraint c { !(x inside {used_q}); }`

Q16. What is the difference between `foreach` in a constraint and `foreach` in procedural code? In a constraint it is unrolled into a set of simultaneous constraints, one per element; nothing is executed in sequence. In procedural code it is a loop that executes its body repeatedly.

Q17. Can you call `randomize()` inside `post_randomize()`? On other objects, yes, and it is sometimes done to randomize sub-items in a second stage. Calling `this.randomize()` there would recurse endlessly.

Q18. How do you constrain a queue to hold between 3 and 6 elements, all different, in descending order? `constraint c { q.size() inside {[3:6]}; foreach (q[i]) if (i > 0) q[i] < q[i-1]; }`. Strict descending order already makes the elements different.

Q19. Is `constraint c { a == b; }` the same as assigning `a = b` in `post_randomize()`? No. The constraint is bidirectional: other constraints on `a` also restrict `b`, and the solver chooses both consistently. The assignment overwrites `a` after solving, without checking any constraint on `a`.

Q20. How do you get 70 percent reads and 30 percent writes? `constraint c { write dist {0 := 70, 1 := 30}; }`

Q21. What does `constraint c { x inside {a_queue}; }` do when the queue is empty? The set is empty, so the constraint cannot be satisfied and `randomize()` fails.

Q22. How do `static` random variables behave? `static rand` properties are shared by all objects of the class. Each `randomize()` call on any object assigns the shared copy a new value.

Advanced and scenario-based

Q23. A regression shows that a configuration bit `fast_mode` is 1 in fewer than 1 percent of tests although nothing constrains it directly. Explain and fix. Some constraint of the form `fast_mode -> (something restrictive)` gives `fast_mode` = 1 far fewer solutions than `fast_mode` = 0, and solutions are equally likely. Fix with `solve fast_mode before <the dependent variables>;` or with an explicit `dist` on `fast_mode`.

Q24. Your generator uses `randc` for a channel number, but coverage shows repeated channels before all are used. Why? The generator creates a new transaction object per item, so each object's cyclic history starts empty. Keep the `randc` variable in the generator (or in one long-lived blueprint object) and copy its value into each transaction.

Q25. A test adds `randomize() with { len == 2000; }` and the call fails although the test writer disabled `len_c`. What do you check? Whether another block, possibly in a base class, also restricts `len`; whether `len` is

wide enough to hold 2000; whether the array whose size is tied to `len` has its own size limit; whether the `constraint_mode(0)` call was made on the same object that is randomized (a generator may construct a new object after the call).

Q26. How would you generate a transaction that depends on the previous one, for example an address that is the previous address plus 4? Keep the previous value in a non-random property and refer to it: `constraint c { addr == prev_addr + 4; }`, updating `prev_addr = addr;` in `post_randomize()`. The object must be reused, or the value must be passed to each new object by the generator.

Q27. When would you not use constraints at all? For fully determined values (fixed patterns, checksums, derived fields), which belong in procedural code or `post_randomize()`; and for very large problems where a procedural algorithm is far cheaper than solving, such as shuffling a large array or picking a random subset.

Q28. The same seed gives different stimulus on two simulators. Is that a bug? No. The standard defines the constraint semantics and the stability rules, not the random number algorithm or the solver. Reproducibility holds for the same seed on the same simulator version with the same code.

Q29. A constraint solver takes minutes per call. What do you look for? Wide variables in multiplications, divisions or modulus; large arrays with pairwise (`foreach` in `foreach`) constraints or `unique`; unbounded array sizes; many variables coupled into one large problem that could be split into stages (randomize the header first, then the payload); bit-level functions of wide vectors.

Q30. Explain how a test can change the behaviour of a transaction class without editing it. Give four mechanisms. Inline constraints (`randomize() with`), `constraint_mode()` and `rand_mode()`, knob properties used as state variables or `dist` weights, and a derived class that overrides or adds constraint blocks (installed through the generator's blueprint handle or the UVM factory).

Q31. `randomize()` succeeds, but the DUT receives an illegal value. List possible causes. `post_randomize()` modified the value; the field is not `rand` and was set elsewhere; the constraint was disabled or overridden; the legal-value constraint is `soft` and was dropped; a width or sign effect makes the constraint accept wrapped values; the driver, not the generator, corrupts the value.

Q32. Why are helper random variables useful? Give an example. They name a property of the stimulus so that it can be constrained, weighted and covered directly. Example: `rand bit ovf;` with `ovf == (a + b > 255)` lets the test set the overflow rate with a `dist` on `ovf`.

6.3 Output-Prediction Questions

All of these have exact answers that do not depend on the seed. Work them out before reading the answer.

P1.

```
class A;
  rand bit [3:0] x;
  constraint c { x > 12; x < 14; }
endclass
// A a = new();  void'(a.randomize());  $display("%0d", a.x);
```

Answer: `13`. Only one value satisfies both conditions.

P2.

```
class B;
  rand bit [2:0] x;
  int y = 9;
  constraint c { x == y; }
endclass
// B b = new();  $display("%0d %0d", b.randomize(), b.x);
```

Answer: `0 0`. A 3-bit variable cannot equal 9 (the comparison is done in 32 bits, so there is no truncation of 9). The call fails and `x` keeps its initial value 0. (The arguments are evaluated left to right in practice; either order gives the same result here because `x` never changes.)

P3.

```
class C;
  rand int x;
  constraint a { soft x == 1; }
  constraint b { soft x == 2; }
endclass
// C c = new();
// void'(c.randomize());                          $display("%0d", c.x);
// void'(c.randomize() with { soft x == 3; });    $display("%0d", c.x);
// void'(c.randomize() with { x < 2; });          $display("%0d", c.x);
```

Answer: `2`, `3`, `1`. First call: the two soft constraints conflict and the later block has priority. Second call: the inline soft constraint has the highest priority. Third call: the hard `x < 2` removes `soft x == 2`; `soft x == 1` does not conflict and is honoured.

P4.

```
class D;
  rand bit [7:0] a, b;
  constraint c1 { a == 10; }
  constraint c2 { b == a + 1; }
endclass
// D d = new();  int s;
// s = d.randomize();                              $display("%0d %0d %0d", s, d.a, d.b);
// d.a.rand_mode(0);  d.a = 20;  s = d.randomize(); $display("%0d %0d %0d", s, d.a, d.b);
// d.c1.constraint_mode(0);      s = d.randomize(); $display("%0d %0d %0d", s, d.a, d.b);
```

Answer: `1 10 11`, then `0 20 11`, then `1 20 21`. In the second call `a` is frozen at 20, which violates `c1`, so the call fails and `b` keeps 11. In the third call `c1` is off; `c2` uses the frozen `a`.

P5.

```
class E;
  rand bit [3:0] v;
  constraint c { v == 7; }
  function void pre_randomize();  $display("pre  v=%0d", v);          endfunction
  function void post_randomize(); $display("post v=%0d", v); v = 15;  endfunction
endclass
// E e = new();
// void'(e.randomize());  $display("after v=%0d", e.v);
// void'(e.randomize());
```

Answer:

EXPECTED OUTPUT

```
pre  v=0
post v=7
after v=15
pre  v=15
post v=7
```

`pre_randomize()` sees the old value, `post_randomize()` sees the solved value and then overwrites it. The overwritten value 15 violates the constraint, and nothing complains.

P6.

```
class P;
  rand bit [3:0] x;
  constraint c { x < 4; }
  constraint d { x > 1; }
endclass
class Q extends P;
  constraint c { x > 12; }
  constraint e { x % 2 == 0; }
endclass
// P h;  Q q = new();  h = q;  void'(h.randomize());  $display("%0d", h.x);
```

Answer: `14`. The object is a `Q`. Its constraints are the overriding `c` (`x > 12`), the inherited `d` (`x > 1`) and the added `e` (even). Of 13, 14, 15 only 14 is even.

P7.

```
class F;
  rand bit [3:0] x;
  constraint c { x inside {[2:5]}; }
endclass
// module scope:  F f = new();  int x = 9;
// f.x = 3;  $display("%0d", f.randomize(null));
// f.x = 9;  $display("%0d", f.randomize(null));
// $display("%0d", f.randomize() with { x == local::x; });
// $display("%0d", f.randomize() with { x == x; });
```

Answer: `1`, `0`, `0`, `1`. The first two are checks of the current value. The third requires `f.x == 9`, which conflicts with the class constraint. In the fourth both names mean `f.x`, so the inline constraint is always true.

P8. Probabilities.

```
rand bit       a;
rand bit [2:0] b;
constraint c { a -> b < 2; }
```

Give P(a = 1) and P(b = 0), first without ordering and then with `solve a before b`.

Answer: Solutions: `a` = 0 with eight values of `b`; `a` = 1 with `b` = 0 or 1. Ten solutions. Without ordering: P(a = 1) = 2/10 = 1/5, and P(b = 0) = 2/10 = 1/5. With `solve a before b`: P(a = 1) = 1/2, and P(b = 0) = 1/2 × 1/8 + 1/2 × 1/2 = 5/16.

P9. Weights.

```
x dist { [1:3] := 2, 4 :/ 2, [5:6] :/ 2 };
```

Give the probability of 1, of 4 and of 5.

Answer: Weights: 1, 2, 3 have 2 each; 4 has 2; 5 and 6 share 2, so 1 each. Total 10. P(1) = 0.2, P(4) = 0.2, P(5) = 0.1.

P10.

```
class G;
  rand int q[$];
  rand int d[];
  constraint c { foreach (q[i]) q[i] == i * 2; }
endclass
// G g = new();  g.q = '{5, 5, 5};
// void'(g.randomize());
// $display("%0d %0d %0d %0d | %0d", g.q.size(), g.q[0], g.q[1], g.q[2], g.d.size());
```

Answer: `3 0 2 4 | 0`. Neither size is constrained, so both arrays keep their sizes: the queue keeps three elements, which are then constrained to 0, 2, 4, and the dynamic array stays empty.

6.4 Find-the-Error and Debugging Questions

Errors the compiler or solver reports

E1. `constraint c { a = 5; }` Assignment in a constraint. Use `a == 5;`.

E2. `randc bit [3:0] id; constraint c { id dist {0 := 1, [1:15] := 2}; }` A `dist` cannot be applied to a `randc` variable. Make it `rand`, or drop the `dist`.

E3. `constraint c { solve a before b; solve b before a; }` Circular ordering.

E4. `constraint c { for (int i = 0; i < 4; i++) d[i] > 0; }` `for` is not allowed in a constraint. Use `foreach (d[i]) d[i] > 0;`.

E5. `constraint c { if (mode) begin len == 4; addr == 0; end }` Constraint sets are grouped with braces: `if (mode) { len == 4; addr == 0; }`.

E6. `int tag;` ... `obj.tag.rand_mode(0);` `rand_mode()` on a property that is not `rand` is a compile error.

E7. `rand string name;` Strings cannot be random variables. Randomize an index or an enum and map it to a string.

E8. `rand bit [1:0] v[5]; constraint c { unique {v}; }` Compiles, but always fails at run time: five distinct values do not exist in 2 bits.

E9. `function void post_randomize(); #10; crc = calc(); endfunction` A delay inside a function. The hooks cannot consume time.

E10. `constraint c { foreach (d[i]) d[i] < d[i+1]; }` Index out of range for the last element. Guard it: `if (i < d.size() - 1)`.

E11. `constraint c { soft id == 3; }` with `randc bit [3:0] id;` Soft constraints are not allowed on `randc` variables.

E12. `class T; rand bit [7:0] a; endclass` ... `T t; void'(t.randomize());` Null handle: `t = new();` is missing. Run-time error.

Bugs that compile and run

D1. Channels repeat although `ch` is `randc`.

```
repeat (8) begin
  cfg c = new();
  void'(c.randomize());
  drive(c.ch);
end
```

A new object each time. Construct `c` once, outside the loop.

D2. The payload is always empty.

```
rand bit [7:0] payload[];
constraint c { foreach (payload[i]) payload[i] != 0; }
```

No size constraint, so the size stays 0. Add `payload.size() inside {[1:64]};`.

D3. Randomization always fails.

```
rand bit [7:0] w[4];
constraint c { w.sum() == 300; }
```

The sum is 8 bits wide and can never be 300. Use `w.sum() with (int'(item)) == 300`.

D4. With one simulator option the test sends all-zero transactions.

```
assert (txn.randomize());
```

With assertions disabled the call is not executed. Use `if (!txn.randomize()) $fatal(1, "...");`.

D5. The requested address is ignored.

```
task send(bit [15:0] addr);
  void'(txn.randomize() with { addr == addr; });
endtask
```

Both names resolve to `txn.addr`. Write `addr == local::addr`.

D6. Delays are sometimes enormous.

```
rand int delay;
constraint c { delay < 20; }
// ...  repeat (txn.delay) @(posedge clk);
```

`delay` is signed, so almost every value is negative; `repeat` with a negative count executes zero times, and other uses treat it as a huge unsigned number. Use `rand int unsigned delay;` with `delay inside {[0:19]};`.

D7. The error test fails to randomize.

```
class err_txn extends base_txn;          // base has: constraint len_c { len <= 64; }
  constraint long_c { len > 64; }
endclass
```

The new block is added to the inherited one. Name it `len_c` to override.

D8. Every tenth transaction is a duplicate of the previous one.

```
void'(txn.randomize() with { a > b; });
gen2drv_mbx.put(txn.copy());
```

The return value is discarded. When the call occasionally fails (here because another constraint sometimes conflicts with `a > b`), the old values are sent again. Check the status.

D9. `fast` is practically never 1.

```
rand bit        fast;
rand bit [31:0] timeout;
constraint c { fast -> timeout == 0; }
```

One solution with `fast` = 1 against four billion with `fast` = 0. Add `solve fast before timeout;`.

D10. The sum check passes in simulation but the values look wrong.

```
rand bit [7:0] a, b;
constraint c { a + b == 8'd200; }
```

The 8-bit literal makes the addition 8 bits wide, so `a` = 250, `b` = 206 is accepted (456 modulo 256 is 200). Write `a + b == 200`.

6.5 Coding Exercises with Solutions

Exercise 1: the ALU transaction class

Write the course transaction class `alu_txn` with random operands and opcode, a weighted opcode mix, the rule that `b` is 0 for `ALU_NOT`, and the methods `print`, `copy` and `compare`. Randomize it five times, and show that a copy compares equal. This example needs `alu_pkg.sv` from Module 1.

ex01_alu_txn.sv

```
module ex01_alu_txn;
  import alu_pkg::*;

  class alu_txn;
    rand bit [7:0] a, b;
    rand alu_op_e  op;
    logic [8:0]    result;
    int unsigned   id;

    constraint op_dist_c {
      op dist { ALU_ADD := 3, ALU_SUB := 3,
                ALU_AND := 2, ALU_OR := 2, ALU_XOR := 2, ALU_NOT := 2,
                ALU_SHL := 1, ALU_SHR := 1 };
    }
    constraint not_b_c { (op == ALU_NOT) -> (b == 0); }

    function void print(string tag = "");
      $display("%s id=%0d op=%-7s a=%3d b=%3d result=%h", tag, id, op.name(), a, b, result);
    endfunction

    function alu_txn copy();
      alu_txn t = new();
      t.a = a;  t.b = b;  t.op = op;  t.result = result;  t.id = id;
      return t;
    endfunction
```

```
    function bit compare(alu_txn rhs);
      return (a == rhs.a) && (b == rhs.b) && (op == rhs.op) && (result === rhs.result);
    endfunction
  endclass

  alu_txn txn, cpy;
  int     errors;

  initial begin
    txn = new();
    for (int i = 0; i < 5; i++) begin
      if (!txn.randomize()) $fatal(1, "randomize failed");
      txn.id = i;
      txn.print("[gen]");
      if (txn.op == ALU_NOT && txn.b != 0) errors++;
    end
    cpy = txn.copy();
    $display("copy equals original : %0d", txn.compare(cpy));
    cpy.a = ~cpy.a;
    $display("after changing copy  : %0d", txn.compare(cpy));
    $display("ex01: errors=%0d", errors);
  end

endmodule
```

What to expect. Five `[gen]` lines with `id` 0 to 4, random operands and opcodes, and `result=xxx` (it is a four-state field that nothing has assigned). The last three lines are exact:

EXPECTED OUTPUT

```
copy equals original : 1
after changing copy  : 0
ex01: errors=0
```

`compare` uses `===` for `result`, so two unassigned (X) results compare equal instead of producing an unknown.

Exercise 2: measure a distribution

Write a small reusable way to print a histogram of a random variable, and use it to verify `len dist {1 := 30, [2:9] :/ 40, 10 := 30}`.

ex02_histogram.sv

```
module ex02_histogram;

  class len_gen;
    rand bit [3:0] len;
    constraint c { len dist {1 := 30, [2:9] :/ 40, 10 := 30}; }
  endclass

  len_gen g;
  int     hist[int];          // associative array: value -> count
  int     n = 2000;
  int     errors;

  initial begin
    g = new();
    repeat (n) begin
      if (!g.randomize()) $fatal(1, "randomize failed");
      hist[g.len]++;
      if (!(g.len inside {[1:10]})) errors++;
    end
    foreach (hist[v])
      $display("len=%2d : %4d  (%0.1f%%)", v, hist[v], 100.0 * hist[v] / n);
    $display("ex02: errors=%0d", errors);
  end

endmodule
```

What to expect. Ten lines for `len` 1 to 10. `len=1` and `len=10` are each near 30 percent (about 600 of 2000). Each of 2 to 9 is near 5 percent (40 percent shared by eight values, about 100 each). The last line is exactly `ex02: errors=0`. An associative array is used so that only values that actually occur are stored and printed.

Exercise 3: a packet with a checksum and error injection

Write a packet class with a random payload of 1 to 16 bytes, a checksum equal to the XOR of all payload bytes, and a random `bad_csum` flag (10 percent) that corrupts the checksum.

ex03_packet.sv

```
module ex03_packet;

  class packet;
    rand bit [7:0] payload[];
    rand bit       bad_csum;
    bit [7:0]      csum;

    constraint size_c { payload.size() inside {[1:16]}; }
    constraint err_c  { bad_csum dist {1 := 10, 0 := 90}; }

    function bit [7:0] calc_csum();
      bit [7:0] c = 0;
      foreach (payload[i]) c ^= payload[i];
      return c;
    endfunction

    function void post_randomize();
      csum = calc_csum();
      if (bad_csum) csum = ~csum;
    endfunction
  endclass

  packet p;
  int    errors, n_bad;

  initial begin
    p = new();
    repeat (1000) begin
      if (!p.randomize()) $fatal(1, "randomize failed");
      if (p.payload.size() < 1 || p.payload.size() > 16)           errors++;
      if (!p.bad_csum && p.csum != p.calc_csum())                  errors++;
      if ( p.bad_csum && p.csum == p.calc_csum())                  errors++;
      if (p.bad_csum) n_bad++;
    end
    $display("corrupted packets: %0d of 1000", n_bad);
    $display("ex03: errors=%0d", errors);
  end

endmodule
```

What to expect. About 100 corrupted packets, then exactly `ex03: errors=0`. The checksum is not `rand`: it is computed in `post_randomize()` from the final payload, and the error is injected after the correct value has been calculated.

Exercise 4: a burst that must not cross a 4 KB boundary

A bus burst has a start address, a number of beats (1 to 16) and a beat size of 1, 2, 4 or 8 bytes. Rules: the start address is aligned to the beat size, and the whole burst lies within one 4096-byte page.

ex04_burst_4k.sv

```
module ex04_burst_4k;

  class burst;
    rand bit [31:0] addr;
    rand bit [4:0]  beats;         // number of beats
    rand bit [1:0]  size_log2;     // bytes per beat = 1 << size_log2

    constraint beats_c { beats inside {[1:16]}; }
    constraint align_c { (addr & ((32'd1 << size_log2) - 1)) == 0; }
    constraint page_c  { int'(addr[11:0]) + int'(beats) * (1 << size_log2) <= 4096; }
  endclass

  burst b;
  int   errors, nbytes;

  initial begin
    b = new();
    repeat (500) begin
      if (!b.randomize()) $fatal(1, "randomize failed");
      nbytes = 1 << b.size_log2;
      if (b.beats < 1 || b.beats > 16)                          errors++;
      if (b.addr % nbytes != 0)                                 errors++;
      if ((b.addr / 4096) != ((b.addr + b.beats * nbytes - 1) / 4096)) errors++;
    end
    $display("ex04: sample addr=%h beats=%0d bytes/beat=%0d", b.addr, b.beats, nbytes);
    $display("ex04: errors=%0d", errors);
  end

endmodule
```

What to expect. A sample burst, then exactly `ex04: errors=0`. The page rule uses only the offset inside the page (`addr[11:0]`) and widens everything to `int`, so no intermediate result can wrap. The checker states the same rule differently: the first and the last byte of the burst are in the same page.

Exercise 5: a random subset in random order

Given a fixed list of eight legal register addresses, choose four different ones in random order.

ex05_subset.sv

```
module ex05_subset;

  class pick;
    bit [7:0]      legal[8] = '{8'h00, 8'h04, 8'h08, 8'h10, 8'h14, 8'h20, 8'h40, 8'h80};
    rand bit [7:0] chosen[4];

    constraint member_c { foreach (chosen[i]) chosen[i] inside {legal}; }
    constraint uniq_c   { unique {chosen}; }
  endclass

  pick p;
  int  errors;

  initial begin
    p = new();
    repeat (200) begin
      if (!p.randomize()) $fatal(1, "randomize failed");
      foreach (p.chosen[i]) begin
        if (!(p.chosen[i] inside {p.legal})) errors++;
        for (int j = i + 1; j < 4; j++)
          if (p.chosen[i] == p.chosen[j]) errors++;
      end
    end
    $display("ex05: sample chosen=%p", p.chosen);
    $display("ex05: errors=%0d", errors);
  end

endmodule
```

What to expect. A sample of four different addresses from the list, then exactly `ex05: errors=0`. `legal` is a state array; `inside {legal}` restricts each choice to it and `unique` makes the choices different.

Further practice (no solutions given)

1. Constrain a 16-bit value so that its upper byte is the bitwise inverse of its lower byte.
2. Generate an array of ten elements in which the first half is ascending and the second half is descending.
3. Generate three 8-bit values that are the side lengths of a valid triangle (each side shorter than the sum of the other two). Watch the widths.
4. Generate a 10-element array where every element is either 0 or greater than its index.
5. Write a class that produces a different one-hot 8-bit value on each of eight successive calls, in random order, without `randc`.
6. Generate a random 32-bit value in which exactly one byte is `0xFF` and no other byte is `0xFF`.
7. Write a constraint for a 3 × 3 matrix in which every row and every column sums to 15 and all nine elements are different (a magic square). Measure how long your simulator takes.
8. Add a knob to the ALU transaction of Exercise 1 that lets a test select "boundary operands only" (each operand is one of 0, 1, 127, 128, 254, 255) without editing the class.
9. Take Problem 22 and add: the FIFO must become full at least twice, and the sequence must end with an empty FIFO. Is 20 operations still enough?
10. For `rand bit [3:0] a, b; constraint c { a < b; }`, compute by hand the probability of each value of `a`, then confirm it with a histogram over 12,000 calls.

6.6 Concepts You Must Understand Before Moving Forward

Basics

- [ ] I can explain why constrained-random verification is used and what it needs besides random stimulus.
- [ ] I can state the difference between `rand` and `randc` and explain why `randc` fails with a new object per item.
- [ ] I know what `randomize()` returns, and what happens to the variables and to `post_randomize()` on failure.
- [ ] I can explain what a state variable is.
- [ ] I know when a `rand` dynamic array is resized and when it is not.

Constraints

- [ ] I can explain "declarative", "bidirectional" and "solved simultaneously" with an example.
- [ ] I can explain why a variable may be non-uniform although all solutions are equally likely.
- [ ] I can spot the chained comparison, overflow, unsigned subtraction and signed-variable traps.
- [ ] I can write `inside`, implication, `if-else`, `foreach`, `sum()` and `unique` constraints from memory.
- [ ] I can explain why `a -> b` also restricts `a`.
- [ ] I know why `sum()` needs `with (int'(item))`.

Distribution

- [ ] I can compute probabilities for `:=` and `:/`.
- [ ] I can explain soft-constraint priority and predict which soft constraint survives.
- [ ] I can work out a probability table with and without `solve ... before`.
- [ ] I can explain why a function call in a constraint is not bidirectional.

Control and debug

- [ ] I know the task and function forms of `constraint_mode()` and `rand_mode()` and the difference between the two.
- [ ] I can explain inline constraints, `local::`, `randomize(args)` and `randomize(null)`.
- [ ] I know what `std::randomize()` is for.
- [ ] I can explain seeds, random stability and how to reproduce a failure.
- [ ] I can list the causes of a randomization failure and describe a debugging procedure.
- [ ] I can explain overriding versus extending a constraint in a derived class.

Problem solving

- [ ] I can solve at least fifteen of the twenty-two problems of Part 5 without looking, and write a checker for each.

6.7 What Comes Next

Module 6 uses everything in this module to build the transaction class properly: properties, constructor, constraints, and the `print`, `copy` and `compare` methods of `alu_txn`. It then steps back to verification planning:

how a specification becomes a list of test scenarios, how directed and constrained-random tests are combined, and how functional coverage and a self-checking testbench close the loop.